



(19) 대한민국특허청(KR)  
(12) 등록특허공보(B1)

(45) 공고일자 2017년05월26일  
(11) 등록번호 10-1740839  
(24) 등록일자 2017년05월22일

(51) 국제특허분류(Int. Cl.)  
G06F 9/30 (2017.01)  
(52) CPC특허분류  
G06F 9/3001 (2013.01)  
G06F 9/30036 (2013.01)  
(21) 출원번호 10-2015-7016920  
(22) 출원일자(국제) 2013년12월04일  
심사청구일자 2015년11월23일  
(85) 번역문제출일자 2015년06월25일  
(65) 공개번호 10-2015-0110497  
(43) 공개일자 2015년10월02일  
(86) 국제출원번호 PCT/IB2013/060637  
(87) 국제공개번호 WO 2014/115001  
국제공개일자 2014년07월31일  
(30) 우선권주장  
13/748,495 2013년01월23일 미국(US)  
(56) 선행기술조사문헌  
US6643821 A\*  
US20110314263 A1\*  
US20080046682 A1\*  
US20020095642 A1\*  
\*는 심사관에 의하여 인용된 문헌

(73) 특허권자  
인터내셔널 비즈니스 머신즈 코포레이션  
미국 10504 뉴욕주 아몬크 뉴오차드 로드  
(72) 발명자  
브래드버리, 조나단, 데이비드  
미국 뉴욕 12601-5400, 포킵시, 사우스 로드  
2455, 아이비엠 코포레이션  
슈워츠, 에릭, 마크  
미국 뉴욕 12601-5400, 포킵시, 사우스 로드  
2455, 엠/피 빌딩 705 2층, 아이비엠 코포레이션  
(74) 대리인  
허정훈

전체 청구항 수 : 총 17 항

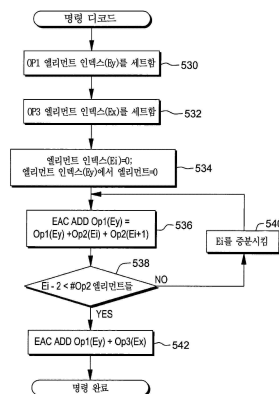
심사관 : 김경완

(54) 발명의 명칭 벡터 체크섬 명령

(57) 요약

벡터 체크섬 명령(Vector Checksum instruction). 제2 오퍼랜드로부터의 엘리먼트들은 제1 결과를 획득하기 위해 하나씩 서로 더해진다. 상기 더하기는 하나 또는 그 이상의 엔드 어라운드 캐리 애드 연산들(end around carry add operations)을 수행하는 단계를 포함한다. 상기 제1 결과는 상기 명령의 제1 오퍼랜드의 엘리먼트 내에 배치된다. 엘리먼트의 각 더하기 후에, 상기 합 of 선정된 위치로부터의 캐리가, 만일 있다면, 이는 상기 제1 오퍼랜드의 엘리먼트 내 선택된 위치에 더해진다.

대표도 - 도5b



(52) CPC특허분류  
*G06F 9/30098* (2013.01)

---

## 명세서

### 청구범위

#### 청구항 1

중앙처리장치에서 기계어 명령을 실행하기 위한 방법을 수행하기 위해 처리 회로에 의한 실행을 위한 명령들을 저장하는 비-일시적인 컴퓨터 읽기가능 저장 매체로서, 상기 방법은: 실행을 위한 기계어 명령을, 처리 회로에 의해서, 획득하는 단계 및 상기 기계어 명령을 실행하는 단계를 포함하고, 상기 기계어 명령은 컴퓨터 아키텍처에 따라 컴퓨터 실행을 위해 정의되며, 상기 기계어 명령은:

오퍼코드를 제공하기 위한 적어도 하나의 오퍼코드 필드 — 상기 오퍼코드는 벡터 체크섬 동작(a Vector Checksum operation)을 식별함 —;

제1 레지스터를 지정하기 위해 사용될 제1 레지스터 필드 — 상기 제1 레지스터는 제1 오퍼랜드를 포함함 —;

제2 레지스터를 지정하기 위해 사용될 제2 레지스터 필드 — 상기 제2 레지스터는 제2 오퍼랜드를 포함함 —; 그리고

하나 또는 그 이상의 레지스터들을 지정하는 데에 사용될 확장 필드(an extension field) — 상기 제1 레지스터 필드는 상기 제1 레지스터를 지정하기 위해 상기 확장 필드의 제1 부분과 결합되며, 그리고 상기 제2 레지스터 필드는 상기 제2 레지스터를 지정하기 위해 상기 확장 필드의 제2 부분과 결합됨 — 를 포함하고; 그리고 상기 실행하는 단계는:

제1 결과를 획득하기 위해 상기 제2 오퍼랜드의 복수의 엘리먼트들을 모두 더하는 단계(adding) — 상기 더하는 단계는 하나 또는 그 이상의 엔드 어라운드 캐리 애드 연산들(one or more end around carry add operations)을 수행하는 단계를 포함함 —;

엔드 어라운드 캐리 애드 연산을 수행하는 단계와 합을 생산하는 단계(producing a sum)에 기초하여, 상기 합을 선정된(chosen) 위치로부터의 캐리(carry)가 있다면, 이를, 상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내의 선택된 위치에 더하는 단계; 및

상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내에 상기 제1 결과를 배치하는 단계(placing)를 포함하는,

비-일시적인 컴퓨터 읽기가능 저장 매체.

#### 청구항 2

제1항에 있어서, 상기 선정된 위치는 비트 위치 0(bit position zero)이고, 상기 선택된 위치는 비트 위치 31이며, 상기 제1 오퍼랜드의 선택된 엘리먼트는 상기 제1 오퍼랜드의 엘리먼트 1(element one)인,

비-일시적인 컴퓨터 읽기가능 저장 매체.

#### 청구항 3

제2항에 있어서, 상기 실행하는 단계는 상기 제1 오퍼랜드의 하나 또는 그 이상의 기타 엘리먼트들에 0들을 배치하는 단계를 더 포함하는,

비-일시적인 컴퓨터 읽기가능 저장 매체.

#### 청구항 4

제3항에 있어서, 상기 제1 오퍼랜드는 4워드 크기의 엘리먼트들(four word-sized elements)을 포함하고, 상기 배치하는 단계는 상기 제1 오퍼랜드의 엘리먼트들 0, 2 및 3에 0들을 배치하는 단계를 포함하는,

비-일시적인 컴퓨터 읽기가능 저장 매체.

#### 청구항 5

삭제

**청구항 6**

제1항에 있어서, 상기 제2 오퍼랜드의 복수의 엘리먼트들은 복수의 워드-크기의 엘리먼트들(word-sized elements)을 포함하는,

비-일시적인 컴퓨터 읽기가능 저장 매체.

**청구항 7**

제1항에 있어서, 상기 기계어 명령은 제3 레지스터들을 지정하기 위해 사용될 제3 레지스터 필드(a third register field)를 더 포함하고, 상기 제3 레지스터는 제3 오퍼랜드를 포함하며, 그리고 상기 실행하는 단계는 제2 결과를 획득하기 위해 상기 제3 오퍼랜드의 선택된 엘리먼트에 상기 제1 결과를 더하는 단계를 더 포함하고, 상기 제1 결과를 더하는 단계는 엔드 어라운드 캐리 애드 연산(an end around carry add operation)을 사용하는,

비-일시적인 컴퓨터 읽기가능 저장 매체.

**청구항 8**

제1항에 있어서, 상기 실행하는 단계는 제2 결과를 획득하기 위해 제3 오퍼랜드의 선택된 엘리먼트에 상기 제1 결과를 더하는 단계를 더 포함하고, 상기 제1 결과를 더하는 단계는 엔드 어라운드 캐리 애드 연산(an end around carry add operation)을 사용하며, 그리고 상기 배치하는 단계는 상기 제1 오퍼랜드의 선택된 엘리먼트에 상기 제2 결과를 배치하는 단계를 포함하는,

비-일시적인 컴퓨터 읽기가능 저장 매체.

**청구항 9**

제8항에 있어서, 상기 제3 오퍼랜드는 상기 기계어 명령에 포함되는,

비-일시적인 컴퓨터 읽기가능 저장 매체.

**청구항 10**

중앙처리장치에서 기계어 명령을 실행하기 위한 컴퓨터 시스템에 있어서, 상기 컴퓨터 시스템은:

메모리; 및

상기 메모리와 통신하는 프로세서를 포함하되, 상기 컴퓨터 시스템은 한 방법을 수행하도록 구성되고, 상기 방법은:

실행을 위한 기계어 명령(machine instruction)을 프로세서에 의해 획득하는 단계 및 상기 기계어 명령을 실행하는 단계를 포함하고, 상기 기계어 명령은 컴퓨터 아키텍처에 따라 컴퓨터 실행을 위해 정의되며, 상기 기계어 명령은:

오퍼코드를 제공하기 위한 적어도 하나의 오퍼코드 필드 — 상기 오퍼코드는 벡터 체크섬 동작(a Vector Checksum operation)을 식별함 —;

제1 레지스터를 지정하기 위해 사용될 제1 레지스터 필드 — 상기 제1 레지스터는 제1 오퍼랜드를 포함함 —;

제2 레지스터를 지정하기 위해 사용될 제2 레지스터 필드 — 상기 제2 레지스터는 제2 오퍼랜드를 포함함 —;

하나 또는 그 이상의 레지스터들을 지정하는 데에 사용될 확장 필드(an extension field) — 상기 제1 레지스터 필드는 상기 제1 레지스터를 지정하기 위해 상기 확장 필드의 제1 부분과 결합되며, 그리고 상기 제2 레지스터 필드는 상기 제2 레지스터를 지정하기 위해 상기 확장 필드의 제2 부분과 결합됨 — 를 포함하고; 그리고 상기 실행하는 단계는:

제1 결과를 획득하기 위해 상기 제2 오퍼랜드의 복수의 엘리먼트들을 모두 더하는 단계(adding) — 상기 더하는 단계는 하나 또는 그 이상의 엔드 어라운드 캐리 애드 연산들(one or more end around carry add operations)을 수행하는 단계를 포함함 —;

엔드 어라운드 캐리 애드 연산을 수행하는 단계와 합을 생산하는 단계(producing a sum)에 기초하여, 상기 합을

선정된(chosen) 위치로부터의 캐리(carry)가 있다면, 이를, 상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내의 선택된 위치에 더하는 단계; 및

상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내에 상기 제1 결과를 배치하는 단계(placing)를 포함하는, 컴퓨터 시스템.

#### 청구항 11

제10항에 있어서, 상기 합의 선정된(chosen) 위치는 비트 위치 0(bit position zero)이고, 상기 선택된 위치는 비트 위치 31이며, 상기 제1 오퍼랜드의 선택된 엘리먼트는 상기 제1 오퍼랜드의 엘리먼트 1(element one)인, 컴퓨터 시스템.

#### 청구항 12

삭제

#### 청구항 13

제10항에 있어서, 상기 제2 오퍼랜드의 복수의 엘리먼트들은 복수의 워드-크기의 엘리먼트들(word-sized elements)을 포함하는,

컴퓨터 시스템.

#### 청구항 14

제10항에 있어서, 상기 기계어 명령은 제3 레지스터들을 지정하기 위해 사용될 제3 레지스터 필드(a third register field)를 더 포함하고, 상기 제3 레지스터는 제3 오퍼랜드를 포함하며, 그리고 상기 실행하는 단계는 제2 결과를 획득하기 위해 상기 제3 오퍼랜드의 선택된 엘리먼트에 상기 제1 결과를 더하는 단계를 더 포함하고, 상기 제1 결과를 더하는 단계는 엔드 어라운드 캐리 애드 연산(an end around carry add operation)을 사용하는,

컴퓨터 시스템.

#### 청구항 15

제10항에 있어서, 상기 실행하는 단계는 제2 결과를 획득하기 위해 제3 오퍼랜드의 선택된 엘리먼트에 상기 제1 결과를 더하는 단계를 더 포함하고, 상기 제1 결과를 더하는 단계는 엔드 어라운드 캐리 애드 연산(an end around carry add operation)을 사용하며, 그리고 상기 배치하는 단계는 상기 제1 오퍼랜드의 선택된 엘리먼트에 상기 제2 결과를 배치하는 단계를 포함하는,

컴퓨터 시스템.

#### 청구항 16

제15항에 있어서, 상기 제3 오퍼랜드는 상기 기계어 명령에 포함되는, 컴퓨터 시스템.

#### 청구항 17

중앙처리장치에서 기계어 명령을 실행하기 위한 방법에 있어서, 상기 방법은:

실행하기 위한 기계어 명령(machine instruction)을 프로세서에 의해 획득하는 단계 및 상기 기계어 명령을 실행하는 단계를 포함하고, 상기 기계어 명령은 컴퓨터 아키텍처에 따라 컴퓨터 실행을 위해 정의되며, 상기 기계어 명령은:

오퍼코드를 제공하기 위한 적어도 하나의 오퍼코드 필드 — 상기 오퍼코드는 벡터 체크섬 동작(a Vector Checksum operation)을 식별함 —;

제1 레지스터를 지정하기 위해 사용될 제1 레지스터 필드 — 상기 제1 레지스터는 제1 오퍼랜드를 포함함 —;

제2 레지스터를 지정하기 위해 사용될 제2 레지스터 필드 — 상기 제2 레지스터는 제2 오퍼랜드를 포함함 —;

하나 또는 그 이상의 레지스터들을 지정하는 데에 사용될 확장 필드(an extension field) - 상기 제1 레지스터 필드는 상기 제1 레지스터를 지정하기 위해 상기 확장 필드의 제1 부분과 결합되며, 그리고 상기 제2 레지스터 필드는 상기 제2 레지스터를 지정하기 위해 상기 확장 필드의 제2 부분과 결합됨 - 를 포함하고; 그리고 상기 실행하는 단계는:

제1 결과를 획득하기 위해 상기 제2 오퍼랜드의 복수의 엘리먼트들을 모두 더하는 단계(adding) - 상기 더하는 단계는 하나 또는 그 이상의 엔드 어라운드 캐리 애드 연산들(one or more end around carry add operations)을 수행하는 단계를 포함함 -;

엔드 어라운드 캐리 애드 연산을 수행하는 단계와 합을 생산하는 단계(producing a sum)에 기초하여, 상기 합의 선정된(chosen) 위치로부터의 캐리(carry)가 있다면, 이를, 상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내의 선택된 위치에 더하는 단계; 및

상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내에 상기 제1 결과를 배치하는 단계(placing)를 포함하는, 방법.

#### 청구항 18

제17항에 있어서, 상기 합의 선정된(chosen) 위치는 비트 위치 0(bit position zero)이고, 상기 선택된 위치는 비트 위치 31이며, 상기 제1 오퍼랜드의 선택된 엘리먼트는 상기 제1 오퍼랜드의 엘리먼트 1(element one)인, 방법.

#### 청구항 19

삭제

#### 청구항 20

제17항에 있어서, 상기 기계어 명령은 제3 레지스터들을 지정하기 위해 사용될 제3 레지스터 필드(a third register field)를 더 포함하고, 상기 제3 레지스터는 제3 오퍼랜드를 포함하며, 그리고 상기 실행하는 단계는 제2 결과를 획득하기 위해 상기 제3 오퍼랜드의 선택된 엘리먼트에 상기 제1 결과를 더하는 단계를 더 포함하고, 상기 제1 결과를 더하는 단계는 엔드 어라운드 캐리 애드 연산(an end around carry add operation)을 사용하는,

방법.

### 발명의 설명

### 기술 분야

[0001] 본 발명의 하나 또는 그 이상의 특징들은, 일반적으로, 컴퓨팅 환경 내의 처리와 관련되며, 특히 그러한 환경 내의 벡터 처리(vector processing)와 관련된다.

### 배경 기술

[0002] 컴퓨팅 환경 내의 처리는 하나 또는 그 이상의 중앙 처리 유닛들(CPU들)의 연산을 제어하는 것을 포함한다. 보통으로, CPU의 연산은 스토리지 내의 명령들에 의해서 제어된다. 명령들은 다른 포맷들을 가질 수 있으며, 종종 다양한 연산들을 수행하는 데에 사용되는 레지스터들을 명시한다(specify).

[0003] CPU의 아키텍처에 따라서, 다양한 종류의 레지스터들이 사용될 수 있는데, 이들은, 예를 들어, 범용 레지스터들, 특수목적 레지스터들, 부동 소수점 레지스터들(floating point registers) 및/또는 벡터 레지스터들을 예들로서 포함할 수 있다. 다른 종류의 레지스터들은 다른 종류의 명령들에서 사용될 수 있다. 예를 들어, 부동 소수점 레지스터들은 부동 소수점 명령들에 의해서 사용될 부동 소수점 수를 저장하고; 그리고 벡터 레지스터들은, 벡터 명령들을 포함하는, 싱글 명령, 멀티플 데이터(SIMD) 명령들에 의해서 수행되는 벡터 처리를 위한 데이터를 저장한다.

### 발명의 내용

## 과제의 해결 수단

- [0004] 기계어 명령을 실행하기 위한 컴퓨터 프로그램 제품의 제공을 통해 선행 기술의 단점들을 극복하고 장점들을 제공한다. 상기 컴퓨터 프로그램 제품은, 처리 회로에 의해 판독 가능한 그리고 어떤 방법을 수행하기 위해 상기 처리 회로에 의해 실행할 명령들을 저장하는, 컴퓨터 판독 가능 스토리지 매체를 포함한다. 상기 방법은, 예를 들어, 실행을 위한 기계어 명령을, 프로세서에 의해서, 획득하는 단계 및 상기 기계어 명령을 실행하는 단계를 포함하고, 상기 기계어 명령은 컴퓨터 아키텍처에 따라 컴퓨터 실행을 위해 정의되며, 상기 기계어 명령은: 오퍼코드를 제공하기 위한 적어도 하나의 오퍼코드 필드 — 상기 오퍼코드는 벡터 체크섬 연산(a Vector Checksum operation)을 식별함 —; 제1 레지스터를 지정하기 위해 사용될 제1 레지스터 필드 — 상기 제1 레지스터는 제1 오퍼랜드들을 포함함 —; 제2 레지스터를 지정하기 위해 사용될 제2 레지스터 필드 — 상기 제2 레지스터는 제2 오퍼랜드들을 포함함 — 를 포함하고, 그리고 상기 실행하는 단계는: 제1 결과를 획득하기 위해 상기 제2 오퍼랜드의 복수의 엘리먼트들을 서로 더하는 단계(adding together) — 상기 더하는 단계는 하나 또는 그 이상의 엔드 어라운드 캐리 애드 연산들(one or more end around carry add operations)을 수행하는 단계를 포함함 —; 엔드 어라운드 캐리 애드 연산을 수행하는 것과 합을 생산하는 단계(producing a sum)에 기초하여, 상기 합의 선정된(chosen) 위치로부터의 캐리가 있다면, 이를 상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내의 선택된 위치에 더하는 단계; 및 상기 제1 오퍼랜드의 선택된(selected) 엘리먼트 내에 상기 제1 결과를 배치하는 단계(placing)를 포함한다.
- [0005] 본 발명의 하나 또는 그 이상의 특징들과 관련된 방법들과 시스템들이 또한 여기에서 기술되고 청구된다. 추가로, 본 발명의 하나 또는 그 이상의 특징들과 관련된 서비스들 또한 여기에서 기술되고 청구될 수 있다.
- [0006] 본 발명의 하나 또는 그 이상의 특징들의 기술들을 통해 추가 특징들과 장점들이 실현된다. 다른 실시 예들과 특징들이 여기에서 상세하게 기술되며 청구범위의 일부로 간주된다.

## 도면의 간단한 설명

- [0007] 하나 또는 그 이상의 특징들이 구체적으로 언급되고 본 명세서의 끝 부분의 청구 범위에서 예시로서 분명하게 청구된다. 전술한 것과 다른 대상들, 특징들, 및 장점들은 다음과 같은 내용으로 첨부되는 도면들과 그 다음에 오는 발명을 실시하기 위한 구체적인 내용을 참조하면 분명해진다.
- 도 1은 본 발명의 하나 또는 그 이상의 특징들을 포함하고 사용하기 위한 컴퓨팅 환경의 한 예를 도시한다.
- 도 2a는 본 발명의 하나 또는 그 이상의 특징들을 포함하고 사용하기 위한 컴퓨팅 환경의 다른 예를 도시한다.
- 도 2b는 도 2a의 메모리에 관하여 더 상세하게 도시한다.
- 도 3은 레지스터 파일의 한 예를 도시한다.
- 도 4a는 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령(a Vector Floating Point Test Data Class Immediate instruction)의 포맷의 한 예를 도시한다.
- 도 4b는 도 4a의 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령의 제3 오퍼랜드의 비트 값들의 한 예를 도시한다.
- 도 4c는 도 4a의 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령과 관련된 로직의 한 실시 예를 도시한다.
- 도 4d는 도 4a의 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령의 실행 블록도의 한 예를 도시한다.
- 도 4e는 2진 부동 소수점 데이터의 다양한 클래스들의 정의의 한 예를 도시한다. 도 5a는 벡터 체크섬 명령(a Vector Checksum instruction)의 포맷의 한 예를 도시한다.
- 도 5b는 도 5a의 벡터 체크섬 명령과 관련된 로직의 한 실시 예를 도시한다.
- 도 5c는 도 5a의 벡터 체크섬 명령의 실행 블록도의 한 예를 도시한다.
- 도 6a는 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트 명령(a Vector Galois Field Multiply Sum and Accumulate instruction)의 포맷의 한 예를 도시한다.
- 도 6b는 도 6a의 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트 명령과 관련된 로직의 한 실시 예를 도시한다.

도 6c는 도 6a의 벡터 갈로이스 필드 멀티플라이 섬 및 어큐뮬레이트 명령의 실행 블록도의 한 예를 도시한다.

도 7a는 벡터 제너레이트 마스크 명령(a Vector Generate Mask instruction)의 포맷의 한 예를 도시한다.

도 7b는 도 7a의 벡터 제너레이트 마스크 명령과 관련된 로직의 한 실시 예를 도시한다.

도 7c는 도 7a의 벡터 제너레이트 마스크 명령의 실행 블록도의 한 예를 도시한다.

도 8a는 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령(a Vector Element Rotate and Insert Under Mask instruction)의 포맷의 한 예를 도시한다.

도 8b는 도 8a의 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령과 관련된 로직의 한 실시 예를 도시한다.

도 8c는 도 8a의 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령의 실행 블록도의 한 예를 도시한다.

도 9a는 벡터 예외 코드(a Vector Exception Code)의 포맷의 한 예를 도시한다.

도 9b는 도 9a의 벡터 예외 코드를 세트하기 위한 로직의 한 실시 예를 도시한다.

도 10은 본 발명의 하나 또는 그 이상의 특징들을 포함하는 컴퓨터 프로그램 제품의 한 실시 예를 도시한다.

도 11은 호스트 컴퓨터 시스템의 한 실시 예를 도시한다.

도 12는 컴퓨터 시스템의 추가 예를 도시한다.

도 13은 컴퓨터 네트워크를 포함하는 컴퓨터 시스템의 또 다른 예를 도시한다.

도 14는 컴퓨터 시스템의 여러 엘리먼트들의 한 실시 예를 도시한다.

도 15a는 도 14의 컴퓨터 시스템의 실행 유닛의 한 실시 예를 도시한다.

도 15b는 도 14의 컴퓨터 시스템의 분기 유닛의 한 실시 예를 도시한다.

도 15c는 도 14의 컴퓨터 시스템의 로드/저장 유닛의 한 실시 예를 도시한다.

도 16은 에뮬레이트된 호스트 컴퓨터 시스템의 한 실시 예를 도시한다.

### 발명을 실시하기 위한 구체적인 내용

- [0008] 본 발명의 하나 또는 그 이상의 특징들에 따라, 벡터 예외 처리(vector exception processing)뿐만 아니라, 다양한 벡터 명령들을 포함하는 벡터 퍼실리티(a vector facility)가 제공된다. 여기서 기술되는 명령들 각각은 하나 또는 그 이상의 벡터 레지스터들(여기에서는 벡터들이라고도 불림)을 사용하는 싱글 인스터럭션, 멀티플 데이터(SIMD) 명령(a Single Instruction, Multiple Data (SIMD) instruction)이다. 벡터 레지스터는, 예를 들어, 프로세서 레지스터(또한 하드웨어 레지스터라고도 한다)이며, 이는 CPU 또는 기타 프로세서의 일부로서 이용 가능한 작은 용량의 스토리지(예를 들어, 주 메모리는 아님)이다. 각각의 벡터 레지스터는 하나 또는 그 이상의 엘리먼트들을 갖는 벡터 오퍼랜드(a vector operand)를 보유하며, 엘리먼트(an element)는 길이가, 예를 들어, 1, 2, 4 또는 8 바이트이다. 다른 실시 예들에서, 엘리먼트들은 다른 크기를 가질 수 있고; 그리고 벡터 명령이 SIMD 명령이 아닐 수 있다.
- [0009] 도 1을 참조하여 본 발명의 하나 또는 그 이상의 특징들을 포함 및 사용하기 위한 컴퓨팅 환경의 한 실시 예를 기술한다. 컴퓨팅 환경(100)은 예를 들어 프로세서(102)(예를 들어, 중앙 처리 장치), 메모리(104)(예를 들어, 메인 메모리), 및 예를 들어 하나 또는 그 이상의 버스들(108) 및/또는 기타 연결 수단들을 통해 서로 결합된 하나 또는 그 이상의 입력/출력(I/O) 디바이스들 및/또는 인터페이스들(106)을 포함한다.
- [0010] 한 예에서, 프로세서(102)는 인터내셔널 비지네스 머신즈 코퍼레이션(International Business Machines Corporation)에서 공급하는 z/Architecture에 기초하며, System z 서버 같은 서버의 일부이며, 이 서버는 인터내셔널 비지네스 머신즈 코퍼레이션에서 또한 공급하며 z/Architecture를 구현한다. z/Architecture의 한 실시 예가 "z/Architecture Principles of Operation,"(IBM® 간행물 번호 SA22-7832-09, 10판, 2012년 9월)라는 제목의 IBM® 간행물에 기술되어 있으며, 이것은 여기에서 그 전체가 참조로써 포함된다. 한 예에서, 상기 프로세서는 인터내셔널 비지네스 머신즈 코퍼레이션에서 또한 공급하는 z/OS 같은 운영체제를 실행한다. IBM®, Z/ARCHITECTURE® 및 Z/OS®는 미국 뉴욕주 아몬크 소재 인터내셔널 비지네스 머신즈 코퍼레이션의 등록 상표이



다. 여기에서 사용되는 다른 명칭들도 인터내셔널 비지네스 머신즈 코포레이션 또는 다른 회사들의 등록 상표, 상표, 또는 제품 명칭일 수 있다.

[0011] 또 하나의 실시 예에서, 프로세서(102)는 인터내셔널 비지네스 머신즈 코포레이션에서 공급하는 Power Architecture에 기초한다. Power Architecture의 한 실시 예가 "Power ISA™ Version 2.06 Revision B"(인터내셔널 비지네스 머신즈 코포레이션, 2010. 07. 23)에 기술되어 있으며, 이것은 여기에서 그 전체가 참조로써 포함된다. POWER ARCHITECTURE®는 인터내셔널 비지네스 머신즈 코포레이션의 등록 상표이다.

[0012] 또 하나의 실시 예에서, 프로세서(102)는 인텔 코포레이션(Intel Corporation)에서 공급하는 Intel 아키텍처에 기초한다. Intel 아키텍처의 한 실시 예가 "Intel® 64 and IA-32 Architectures Developer's Manual: Vol. 2B, Instructions Set Reference, A-L"(오더 넘버 253666-045US, 2013년 01월) 및 "Intel® 64 and IA-32 Architectures Developer's Manual: Vol. 2B, Instructions Set Reference, M-Z"(오더 넘버 253667-045US, 2013년 01월)에 기술되어 있으며, 이들 각각은 여기에서 그 전체가 참조로써 포함된다. Intel®은 미국 캘리포니아 산타클라라 소재 인텔 코포레이션의 등록 상표이다.

[0013] 도 2a를 참조하여 본 발명의 하나 또는 그 이상의 특징들을 포함 및 사용하기 위한 컴퓨팅 환경의 또 하나의 실시 예를 기술한다. 이 예에서, 컴퓨팅 환경(200)은 예를 들어 네이티브 중앙처리장치(202), 메모리(204), 및 예를 들어 하나 또는 그 이상의 버스들(208) 및/또는 기타 연결 수단들을 통해 서로 결합된 하나 또는 그 이상의 입력/출력(I/O) 디바이스들 및/또는 인터페이스들(206)을 포함한다. 예시로서, 컴퓨팅 환경(200)에는 미국 뉴욕 아몬크 소재 인터내셔널 비지네스 머신즈 코포레이션에서 공급하는 PowerPC 프로세서, pSeries 서버 또는 xSeries 서버; 미국 캘리포니아 팔로 알토 소재 휴렛 팩커드(Hewlett Packard Co.)에서 공급하는 Intel Itanium II 프로세서들을 구비한 HP Superdome; 및/또는 인터내셔널 비지네스 머신즈 코포레이션, 휴렛 팩커드, 인텔, 오라클 또는 기타 회사들에서 공급하는 아키텍처들에 기초하는 기타 머신들이 포함될 수 있다.

[0014] 네이티브 중앙처리장치(202)는 상기 환경 내에서 처리하는 동안에 사용되는 하나 또는 그 이상의 범용 레지스터들 및/또는 하나 또는 그 이상의 특수용 레지스터들 같은 하나 또는 그 이상의 네이티브 레지스터들(210)을 포함한다. 이 레지스터들은 상기 환경의 특정 시점의 상태를 표시하는 정보를 포함한다.

[0015] 또한, 네이티브 중앙처리장치(202)는 메모리(204)에 저장된 명령들 및 코드를 실행한다. 한 구체적인 예에서, 상기 중앙처리장치는 메모리(204)에 저장된 에뮬레이터 코드(212)를 실행한다. 이 코드는 한 아키텍처로 구성된 처리 환경이 다른 아키텍처를 에뮬레이트할 수 있게 해준다. 예를 들어, 에뮬레이터 코드(212)는 z/Architecture 이외의 아키텍처들에 기초한 머신들, 즉 PowerPC 프로세서들, pSeries 서버들, xSeries 서버들, HP Superdome 서버들 또는 기타 등등의 머신들이 z/Architecture를 에뮬레이트하여 그 z/Architecture에 기초하여 개발된 소프트웨어와 명령들을 실행할 수 있게 해준다.

[0016] 도 2b를 참조하여 에뮬레이터 코드(212)에 관련된 더 세부적인 사항들을 기술한다. 메모리(204)에 저장된 게스트 명령들(250)은 네이티브 CPU(202)의 아키텍처 이외의 아키텍처에서 실행될 수 있도록 개발된 소프트웨어 명령들(예를 들어, 기계어 명령들과 관련되어 있음)을 포함한다. 예를 들면, 게스트 명령들(250)은 z/Architecture 프로세서(102) 상에서 실행되도록 설계될 수도 있지만, 그 대신에 네이티브 CPU(202) 상에 에뮬레이트되며, 네이티브 CPU는 예를 들어 Intel Itanium II 프로세서일 수 있다. 한 예에서, 에뮬레이터 코드(212)는 메모리(204)로부터 하나 또는 그 이상의 게스트 명령들(250)을 획득하고 그 획득된 명령들에 로컬 버퍼링을 선택적으로 제공하기 위한 명령 패칭 루틴(252)을 포함한다. 그것(212)은 획득된 게스트 명령의 유형을 판정한 후 그 게스트 명령을 하나 또는 그 이상의 대응하는 네이티브 명령들(256)로 변환하기 위한 명령 변환 루틴(254)을 또한 포함한다. 이 변환은 예를 들어 상기 게스트 명령에 의해 수행될 기능(function)을 식별하는 단계와 그 기능을 수행할 네이티브 명령(들)을 선택하는 단계를 포함한다.

[0017] 추가로, 에뮬레이터(212)는 상기 네이티브 명령들이 실행되도록 하게 할 에뮬레이션 제어 루틴(260)을 포함한다. 에뮬레이션 제어 루틴(260)은 네이티브 CPU(202)로 하여금 하나 또는 그 이상의 앞에서 획득된 게스트 명령들을 에뮬레이트하는 네이티브 명령들의 루틴을 실행하게 하고 그러한 실행 마지막에 다음 게스트 명령 또는 게스트 명령들의 그룹을 획득하는 것을 에뮬레이트하도록 상기 명령 패치 루틴에 제어를 반환할 수 있다. 네이티브 명령들(256)의 실행은 메모리(204)로부터 레지스터 내에 데이터를 로딩하는 것; 레지스터로부터 메모리로 다시 데이터를 저장하는 것; 또는 상기 변환 루틴에 의해 결정된 바와 같이, 산술 또는 논리 연산의 몇몇 유형을 수행하는 것을 포함할 수 있다.

- [0018] 각 루틴은 예를 들어 소프트웨어로 구현되며, 이 소프트웨어는 메모리에 저장되고 네이티브 중앙처리장치(202)에 의해 실행된다. 다른 예들에서, 하나 또는 그 이상의 상기 루틴들 또는 연산들은 펌웨어, 하드웨어, 소프트웨어 또는 이들의 일부 조합으로 구현된다. 상기 에뮬레이트된 프로세서의 레지스터들은 네이티브 CPU의 레지스터들(210)을 사용하여 또는 메모리(204) 내 위치들(locations)을 사용하여 에뮬레이트될 수 있다. 실시 예들에서, 게스트 명령들(250), 네이티브 명령들(256) 및 에뮬레이터 코드(212)는 동일한 메모리에 상주할 수도 있고 또는 서로 다른 메모리 디바이스들 중에서 분배될 수 있다.
- [0019] 여기에서 사용할 때, 펌웨어(firmware)는 예를 들어 프로세서의 마이크로코드(microcode), 밀리코드(millicode) 및/또는 매크로코드(macrocode)를 포함한다. 예를 들어, 펌웨어는 상위 레벨(higher level) 머신 코드의 구현에 사용되는 하드웨어-레벨 명령들 및/또는 데이터 구조들을 포함한다. 한 실시 예에서, 펌웨어는 예를 들어 통상적으로 마이크로코드로 전달되는 사유권 있는 코드(proprietary code)를 포함하며 이 마이크로코드는 신뢰 소프트웨어(trusted software) 또는 기본 하드웨어에 특화된 마이크로코드를 포함하고 운영체제가 시스템 하드웨어에 액세스하는 것을 제어한다.
- [0020] 한 예에서, 획득되어 변환되고 실행되는 게스트 명령(250)은 여기에서 기술되는 명령이다. 한 아키텍처(예를 들어, z/Architecture)로 이루어진 상기 명령이 메모리로부터 페치되고 변환되고 다른 아키텍처(예를 들어, PowerPC, pSeries, xSeries, Intel 등등의 아키텍처)로 이루어진 일련의 네이티브 명령들(256)로서 표현된다. 그 다음에 이 네이티브 명령들이 실행된다.
- [0021] 한 실시 예에서, 여기에서 기술하는 여러 명령들은 벡터 명령들이며, 이들은 벡터 퍼실리티의 일부이다. 벡터 퍼실리티는 예를 들어 1 내지 16개 엘리먼트 범위의 고정 사이즈 벡터들을 제공한다. 각 벡터는 상기 퍼실리티에서 정의된 벡터 명령들에 의해 연산되는 데이터를 포함한다. 한 실시 예에서, 만일 벡터가 다수 엘리먼트들로 구성되면, 각 엘리먼트는 다른 엘리먼트들과 병렬로 처리된다. 모든 엘리먼트의 처리가 완료되기 전에는 명령 완료는 이루어지지 않는다. 다른 실시 예들에서, 엘리먼트들은 부분적으로 병렬 및/또는 순차로 처리된다.
- [0022] 벡터 명령들은 z/Architecture, Power, x86, IA-32, IA-64 등을 포함한(그러나 이에 한정되지 않음) 여러 아키텍처들의 일부로서 구현될 수 있다. 여기에 기술된 실시 예들이 z/Architecture에 대한 것일지라도, 본 발명의 벡터 명령들과 하나 또는 그 이상의 특징들은 다수의 다른 아키텍처들에 기초할 수 있다. z/Architecture는 단지 하나의 예시일뿐이다.
- [0023] 벡터 퍼실리티가 z/Architecture의 일부로 구현되는 한 실시 예에서, 상기 벡터 레지스터들과 명령들을 사용하기 위해, 벡터 인에이블먼트 컨트롤과 명시된 컨트롤 레지스터 내 레지스터 컨트롤(예를 들어, 컨트롤 레지스터 0)이 예를 들어 일(one)로 세트된다. 만일 상기 벡터 퍼실리티가 설치되어 있고 벡터 명령이 상기 인에이블먼트 컨트롤들이 세트되지 않은 채 실행되면, 데이터 예외가 인지된다. 만일 상기 벡터 퍼실리티가 설치되어 있지 않고 벡터 명령이 실행되면, 연산 예외가 인지된다.
- [0024] 한 실시 예에서, 32개의 벡터 레지스터가 있으며 다른 유형의 레지스터들이 상기 벡터 레지스터들의 4분면(quadrant)에 매핑될 수 있다. 예를 들어, 도 3에서 도시한 바와 같이, 레지스터 파일(300)은 32개의 벡터 레지스터들(302)을 포함하고 각각의 레지스터는 길이가 128 비트이다. 길이가 64 비트인 16개 부동 소수점 레지스터들(302)은 상기 벡터 레지스터들을 오버레이(overlay) 할 수 있다. 그렇게 하여, 예로서, 부동 소수점 레지스터 2가 수정되면, 벡터 레지스터 2도 수정된다. 다른 유형의 레지스터들에는 다른 매핑들이 가능할 수 있다.
- [0025] 벡터 데이터는 스토리지에서 예를 들어 다른 데이터 포맷들과 마찬가지로 좌측-에서-우측 순으로 나타난다. 0~7로 번호가 붙은 데이터 포맷의 비트들이 스토리지에서 최좌측(가장 낮은 번호가 붙은) 바이트 위치에 있는 바이트를 구성하고, 비트들 8~15가 다음 순차 위치에 있는 바이트를 구성하는 등의 방식이다. 다른 예에서, 벡터 데이터는 스토리지에서 우측-에서-좌측 순 같이 다른 순서로 나타날 수도 있다.
- [0026] 여기서 기술된 벡터 명령들의 각각은 복수의 필드들을 가지며 이 필드들의 하나 또는 그 이상은 그 자신과 관련된 아래 첨자 번호(subscript number)를 갖는다. 이 명령의 필드에 관련된 아래 첨자 번호는 그 필드가 적용되는 오퍼랜드를 나타낸다. 예를 들어, 벡터 레지스터  $V_1$ 에 관련된 아래 첨자 번호 1은  $V_1$ 에 있는 레지스터가 제1 오퍼랜드를 포함한다는 것을 나타내는 등의 방식이다. 레지스터 오퍼랜드는 길이에 있어서, 예를 들어 128 비트인, 하나의 레지스터이다.
- [0027] 또한, 벡터 퍼실리티가 제공되는 다수의 벡터 명령들은 명시된 비트들의 필드를 갖는다. 이 필드는, 레지스터 확장 비트(register extension bit) 또는 RXB라 불리는데, 각각의 벡터 레지스터 지정 오퍼랜드들(vector register designated operands)을 위한 최상위 비트를 포함한다. 명령에 의해 명시되지 않은 레지스터 지정

(register designations)을 위한 비트들은 유보되고 제로로 세트된다. 상기 최상위 비트는, 예를 들어, 4-비트 레지스터 지정의 좌측에 연결되어(concatenated) 5-비트 벡터 레지스터 지정(a five-bit vector register designation)을 생성한다.

- [0028] 한 예에서, RXB 필드는 4개 비트(예를 들어 비트들 0~3)를 포함하고, 이 비트들은 다음과 같이 정의된다:
- [0029] 0 - 명령의 (예를 들어, 비트들 8~11 내의) 제1 벡터 레지스터 지정을 위한 최상위 비트.
- [0030] 1 - 있을 경우, 명령의 (예를 들어, 비트들 12~15 내의) 제2 벡터 레지스터 지정을 위한 최상위 비트.
- [0031] 2 - 있을 경우, 명령의 (예를 들어, 비트들 16~19 내의) 제3 벡터 레지스터 지정을 위한 최상위 비트.
- [0032] 3 - 있을 경우, 명령의 (예를 들어, 비트들 32~35 내의) 제4 벡터 레지스터 지정을 위한 최상위 비트.
- [0033] 각 비트는 예를 들어 어셈블러에 의해 레지스터 번호에 따라서 제로 또는 일로 세트된다. 예를 들어, 레지스터들 0~15에 대해서, 비트는 0으로 세트되고; 레지스터들 16~31에 대해서, 비트는 1로 세트되는 식으로 된다.
- [0034] 한 실시 예에서, 각 RXB 비트는 하나 또는 그 이상의 벡터 레지스터들을 포함하는 명령 내의 특정한 위치에 대한 확장 비트이다. 예를 들어, 하나 또는 그 이상의 벡터 명령들에서, RXB의 비트 0은 예를 들어  $V_1$ 로 할당되는 위치 8~11에 대한 확장 비트이고; RXB의 비트 1은 예를 들어  $V_2$ 로 할당되는 위치 12~15에 대한 확장 비트인 등의 방식이다. 또 다른 실시 예에서, RXB 필드는 추가 비트들을 포함하고, 둘 이상의 비트가 각 벡터 또는 위치에 대한 확장자(extension)로 사용된다.
- [0035] RXB 필드를 포함하는 본 발명의 한 특징에 따라 제공된 한 명령이 벡터 부동 소수점 테스트 데이터 클래스 즉시(VFTCI) 명령(a Vector Floating Point Test Data Class Immediate (VFTCI) instruction)이고, 이 명령의 예가 도 4a에 도시되어 있다. 한 예에서, 벡터 부동 소수점 테스트 데이터 클래스 즉시(VFTCI) 명령(400)은 벡터 부동 소수점 테스트 데이터 클래스 즉시 연산을 나타내는 오퍼코드 필드들(402a, 예를 들어 비트들 0~7; 402b, 예를 들어 비트들 40~47); 제1 벡터 레지스터( $V_1$ )를 지정하기 위해 사용되는 제1 벡터 레지스터 필드(404, 예를 들어 비트들 8~11); 제2 벡터 레지스터( $V_2$ )를 지정하기 위해 사용되는 제2 벡터 레지스터 필드(406, 예를 들어 비트들 12~15); 비트마스크(a bitmask)를 포함하기 위한 즉시 필드(an immediate field)( $I_3$ )(408, 예를 들어 비트들 16~27); 제1 마스크 필드(a first mask field)( $M_5$ )(410, 예를 들어 비트들 28~31); 제2 마스크 필드(a second mask field)( $M_4$ )(412, 예를 들어 비트들 32~35); 및 RXB 필드(414, 예를 들어 비트들 36~39)를 포함한다. 필드들(404~414)의 각각은, 한 예에서, 별개이며 상기 오퍼코드(들)로부터 독립적이다. 또한, 한 실시 예에서, 그들은 별개이고 서로로부터 독립적이지만, 다른 실시 예들에서, 둘 이상의 필드가 결합될 수도 있다. 이 필드들의 사용에 대한 추가 정보를 아래에 기술한다.
- [0036] 한 예에서, 오퍼코드 필드(402a)에 의해 지정된 오퍼코드의 선택된 비트들(예를 들어, 처음 두 비트들)은 이 명령의 길이를 명시한다. 이 특정 예에서, 선택된 비트들은 상기 길이가 3개 하프워드들(three halfwords)임을 나타낸다. 또한 상기 명령의 포맷은 확장된 오퍼코드 필드를 갖는 벡터 레지스터-및-즉시 연산(a vector register-and-immediate operation with an extended opcode field)이다. 상기 벡터(V) 필드들의 각각은 RXB에 의해 명시되는 자신의 대응 확장 비트와 함께 벡터 레지스터를 지정한다. 구체적으로, 벡터 레지스터들에 있어서, 오퍼랜드를 보유하는 레지스터는 예를 들어 상기 레지스터 필드의 4-비트 필드에 자신의 대응 레지스터 확장 비트(RXB)를 최상위 비트로서 더한 것을 사용하여 명시된다. 예를 들어, 만일 상기 4-비트 필드가 0110이고 상기 확장 비트가 0이면, 5 비트 필드 00110은 6번 레지스터를 표시한다.
- [0037] 또한, VFTCI 명령의 한 실시 예에서,  $V_1$ (404) 및  $V_2$ (406)는, 상기 명령을 위해서, 각각, 제1 오퍼랜드 및 제2 오퍼랜드를 포함하는 벡터 레지스터들을 명시한다. 또한,  $I_3$ (408)은 복수의 비트들을 갖는 비트 마스크를 포함하고, 각각의 비트는 2진 부동 소수점 엘리먼트 클래스(a binary floating point element class) 및 부호(양(positive) 또는 음(negative))를 표시하기 위해 사용되며, 이에 관해서는 이하에서 더 상세하게 기술된다.
- [0038] 다른 실시 예에서, 비트 마스크는, 예를 들어, 범용 레지스터로, 메모리로, 벡터 레지스터의 한 엘리먼트로(엘리먼트마다 다름) 또는 주소 계산(an address computation)으로부터 제공될 수 있다. 그것은 명령의 명시적 오퍼랜드(an explicit operand)로서 또는 암시적 오퍼랜드(an implied operand) 또는 입력으로서 포함될 수 있다.

- [0039]  $M_5$  필드(410)는 예를 들어 4개 비트들 0~3을 갖고, 예를 들어 비트 0에서 싱글 엘리먼트 컨트롤(S)을 명시한다. 만일 비트 0이 1로 세트되면, 연산은 벡터 내의 0-인덱스된 엘리먼트(the zero-indexed element)에 관해서만 일어난다. 제1 오퍼랜드 벡터 내의 다른 모든 엘리먼트들의 비트 위치들은 예측 불가능하다. 만일 비트 0이 0으로 세트되면, 연산은 벡터 내의 모든 엘리먼트들에 관해서 일어난다.
- [0040]  $M_4$  필드(412)는, 예를 들어, 명령의 제2 오퍼랜드 내 부동 소수점 수들의 크기를 명시하기 위해 사용된다. 한 예에서, 이 필드는 3으로 세트되는 데, 이는 2배 정밀도 2진 부동 소수점 수(a double precision binary floating point number)를 표시한다. 다른 예들도 또한 가능하다.
- [0041] 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령의 한 실시 예의 실행에서, 제3 오퍼랜드로부터 하나 또는 그 이상의 비트들을 선택하기 위해 제2 오퍼랜드의 부동 소수점 엘리먼트 또는 엘리먼트들의 클래스와 부호가 검사된다. 만일 선택된 비트가 세트되면, 제1 오퍼랜드 내 대응 엘리먼트의 모든 비트 위치들은 1로 세트된다; 그렇지 않으면, 그들은 0으로 세트된다. 다시 말하면, 만일 제2 오퍼랜드의 엘리먼트 내에 보유했던 부동 소수점 수의 클래스/부호가 제3 오퍼랜드 내 세트 비트(a set bit)(즉, 예를 들어, 1로 세트된 비트)와 일치하면(match), 제2 오퍼랜드의 엘리먼트에 대응하는 제1 오퍼랜드의 엘리먼트는 1로 세트된다. 한 예에서, 모든 오퍼랜드 엘리먼트들은 긴 포맷 BFP(2진 부동 소수점) 수들을 보유한다.
- [0042] 여기서 표시한 바와 같이, 제3 오퍼랜드의 12개 비트들(명령 텍스트의 비트들 16~27)은 BFP 데이터 클래스 및 부호의 12개 조합들을 명시하기 위해 사용된다. 한 예에서, 도 4b에서 도시한 바와 같이, BFP 오퍼랜드 엘리먼트들은 여섯 개의 클래스들(430)로 나누어지는데, 즉: 영(zero), 정규수(normal number), 비정규수(subnormal number), 무한대(infinity), QNaN(quiet Not-a-Number), 및 SNaN(signaling NaN)로 나누어지고, 각 클래스는 이들과 연관된 부호(432)(양 또는 음의)를 갖는다. 따라서, 예를 들어,  $I_3$ 의 비트 0은 양의 부호를 갖는 영(zero) 클래스를 명시하고, 비트 1은 음의 부호를 갖는 영 클래스를 명시하는 등의 방식이다.
- [0043] 제3 오퍼랜드 비트들 중 하나 또는 그 이상은 1로 세트될 수 있다. 또한, 한 실시 예에서, 명령은 하나 또는 그 이상의 엘리먼트들 상에서 동시에 연산할 수 있다.
- [0044] SNaN(Signaling NaN)들 및 QNaN(Quiet NaN)들을 포함하는 오퍼랜드 엘리먼트들은 IEEE 예외를 일으킴이 없이 검사된다.
- [0045] 모든 엘리먼트들에 대한 결과 요약 조건 코드(Resulting Summary Condition Code):
- [0046] 0 모든 엘리먼트들에 대해서 선택된 비트는 1이다(일치)
- [0047] 1 모든 엘리먼트들에 대해서는 아니지만, 적어도 하나에 대해서 선택된 비트는 1이다(S-비트가 0일 때)
- [0048] 2 --
- [0049] 3 모든 엘리먼트들에 대해서 선택된 비트는 0이다(불일치)
- [0050] IEEE 예외들: 없음
- [0051] 프로그램 예외들:
- [0052] • 데이터 예외 코드(DXC) FE를 갖는 데이터, 벡터 명령, 벡터 퍼실리티가 인에이블되지 않음을 나타냄
- [0053] • 연산(만일 z/Architecture를 위한 벡터 퍼실리티가 설치되어 있지 않은 경우)
- [0054] • 명세
- [0055] • 트랜잭션 제한
- [0056] 프로그래밍 노트:
- [0057] 1. 이 명령은 예외 또는 IEEE 플래그들을 세트하는 것의 위험 없이 오퍼랜드 엘리먼트들을 테스트하기 위한 방법을 제공한다.



2. S 비트가 세트되어 있을 때, 1의 조건 코드는 사용되지 않는다.

- [0058]
- [0059] 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령(Vector Floating Point Test Data Class Immediate instruction)의 한 실시 예에 관한 더 상세한 설명이 도 4c 및 4d를 참조하여 기술된다. 특히, 도 4c는 프로세서(예를 들어, CPU)에 의해서 수행되는 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령과 관련된 로직의 한 실시 예를 도시하고, 도 4d는 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령의 실행을 보여주는 블록도의 한 예를 도시한다.
- [0060] 도 4c를 참조하면, 초기에, 엘리먼트 인덱스(Ei)라 하는 변수가 0으로 초기화된다(단계 450). 그 다음, 엘리먼트 Ei(이 경우에 엘리먼트 0임)의 값이 명령의 제2 오퍼랜드로부터(예를 들어, V<sub>2</sub>에 의해서 지정된 레지스터 내에 저장된 오퍼랜드로부터) 추출된다(단계452). 이 값은, 긴 포맷 2진 부동 소수점 값이며, 아래에서 기술하는 바와 같이, 제2 오퍼랜드의 부동 소수점 엘리먼트를 위한 클래스 및 부호를 획득하기 위해 타입 넘버(a type number)로 변환된다(단계 454). 한 예에서, 부동 소수점 수의 크기(453)는 상기 변환 로직에 대한 입력이다. 획득된 클래스 및 부호는, 도 4b를 참조하여 기술한 바와 같이, 특정 클래스/부호 비트에 관련된다. 예를 들어서, 만일 상기 변환이 상기 부동 소수점 수가 양의 정규수(a positive, normal number)를 나타낸다면, 비트 2가 상기 부동 소수점 수와 관련된다.
- [0061] 변환 후에, 상기 변환에 기초하여 결정된 특정 비트에 대응하는 제3 오퍼랜드 내 비트(선택된 비트라 함)가 검사된다(단계 456). 만일 선택된 비트가 세트되어 있으면(질의 단계 458), 엘리먼트(Ei)에 대응하는 제1 오퍼랜드 내의 엘리먼트는 모두 1로 세트되고(단계 460); 그렇지 않으면, 제1 오퍼랜드 내의 그 엘리먼트는 0으로 세트된다(단계 462). 예를 들어, 만일 엘리먼트 0 내의 부동 소수점 수의 변환이 양의 비정규수(a positive, normal number)를 나타내면, 비트 2가 그 수와 관련된다. 따라서, 제3 오퍼랜드의 비트 2는 검사되고, 만일 그것이 1로 세트되면, 제1 오퍼랜드의 엘리먼트 0은 모두 1로 세트된다.
- [0062] 그 후, Ei가 제2 오퍼랜드의 엘리먼트들의 최대 수와 같은지에 관한 결정이 내려진다(질의 단계 464). 만일 같지 않으면, Ei는 증가되는데, 예를 들어, 1만큼 증가되고(단계 466), 그리고 처리는 단계 452에서 계속된다. 이와 다르게, 만일 Ei가 엘리먼트들의 최대 수와 같다면, 요약 조건 코드(summary condition code)가 생성된다(단계 468). 요약 조건 코드는 제2 오퍼랜드의 모든 엘리먼트들에 대한 처리를 요약한다. 예를 들어, 만일 선택된 비트가 모든 엘리먼트들에 대해서 1이면(일치), 최종 조건 코드는 0이다. 한편, 만일 선택된 비트가, 엘리먼트들 모두는 아니지만, 적어도 하나에 대해서 1이면(S-비트가 0이 아닐 때), 조건 코드는 1이고, 만일 선택된 비트가 엘리먼트들 모두에 대해서 0이면(불일치), 조건 코드는 3이다.
- [0063] 상기 처리는 도 4d의 블록도에서 그림으로 도시되어 있다. 도시된 바와 같이, 벡터 레지스터(480)는 복수의 엘리먼트들(482a-482n)을 포함하고, 각각은 부동 소수점 수(FP number)를 포함한다. 각각의 부동 소수점 수와 부동 소수점 수(483a-483n)의 크기(size)가 타입 넘버로의 변환 로직(convert-to-type number logic)(484a-484n)에 대한 입력이고, 출력은 상기 부동 소수점 수에 대한 클래스/부호를 나타내는 특정 비트이다. 그 다음, 각각의 특정 비트에 대응하는 각각의 마스크(486a-486n) 내 선택된 비트가 검사된다. 상기 선택된 비트가 세트되어 있는지에 따라서, 벡터 레지스터(488) 내 제1 오퍼랜드가 세트된다. 예를 들어, 만일 제2 오퍼랜드의 엘리먼트 0에 대해서, 선택된 비트가 세트되어 있으면, 제1 오퍼랜드의 엘리먼트(490a)는 모두 1로 세트된다. 이런 식으로, 만일 제2 오퍼랜드의 엘리먼트 1에 대한 선택된 비트가 세트되지 않으면(예를 들어, 0으로 세트되었다면), 제1 오퍼랜드의 엘리먼트(490b)는 0으로 세트된다.
- [0064] 타입 넘버로의 변환 로직(the convert-to-type number logic)의 한 실시 예에 관한 더 상세한 설명이 지금부터 기술된다. 초기에, 부동 소수점 수는, 표준 IEEE 2진 부동 소수점 수이며, 알려진 바와 같이, 부호(a sign), 지수(exponent)(8 비트)+127, 및 프랙션(a fraction)(23비트)의 세 부분으로 변환된다. 그 다음, 이들 세 부분 값들 모두는, 도 4e에서 도시한 바와 같은, 클래스 및 부호를 결정하기 위해 검사된다. 예를 들어, 부호는 부호 파트의 값이고, 클래스(즉, 도 4e에서의 실체)는 지수 및 프랙션(도 4e에서 유닛 비트는 프랙션의 암시된 비트임)의 값들에 기초한다. 예를 들어, 만일 지수와 프랙션(유닛 비트를 포함하는)의 값들이 0이라고 하면, 클래스는 0이고, 만일 부호 파트가 양(positive)이라면, 부호는 양이다. 따라서, 비트 0(도 4b)은 이 부동 소수점 수의 클래스/부호를 나타낸다.
- [0065] 위에서 벡터 내 엘리먼트들의 부동 소수점 클래스를 테스트하여 결과 비트마스크(a resulting bitmask)를 세팅하는 명령의 한 실시 예를 기술하였다. 벡터 부동 소수점 테스트 데이터 클래스 즉시 명령(The Vector Floating Point Test Data Class Immediate instruction)은 즉시 필드(an immediate field)를 가지고 있고, 이 필드에서

각 비트는 검출할 부동 소수점 수들의 클래스를 표시한다. 입력 벡터의 각 부동 소수점 엘리먼트는 명령에 의해서 명시된 클래스들 중 어느 하나에 그 값이 해당하는지를 알기 위해 테스트된다. 만일 부동 소수점 엘리먼트가 상기 클래스들 중 하나에 해당한다면, 출력 벡터의 대응 엘리먼트의 비트 위치들은 1로 세트된다. 이것은 어떠한 예외들 또는 인터럽션들을 일으키지 없이 2진 부동 소수점 수에 관한 어떤 속성들(예를 들어, 클래스 및 부호)을 결정하기 위한 기법을 제공한다.

[0066] 다른 실시 예에서, 테스트는 제3 오퍼랜드의 어느 비트들이 세트(예를 들어 1로) 되었는지를 검사하여, 제2 오퍼랜드의 하나 또는 그 이상의 엘리먼트들의 클래스/부호가 상기 세트 비트들 중 하나와 동일한지를 결정함으로써 수행될 수도 있다. 그 다음 제1 오퍼랜드는 상기 비교에 기초하여 세트된다.

[0067] 다른 특징으로, 벡터 체크섬 명령(a Vector Checksum instruction)이 제공되는데, 한 예가 도 5a에 도시되어 있다. 한 예에서, 벡터 체크섬 명령(500)은 벡터 체크섬 연산을 나타내는 오퍼코드 필드들(502a, 예를 들어, 비트 0~7; 502b, 예를 들어, 비트 40~47); 제1 벡터 레지스터( $V_1$ )를 지정하기 위해 사용되는 제1 벡터 레지스터 필드(504, 예를 들어, 비트 8~11); 제2 벡터 레지스터( $V_2$ )를 지정하기 위해 사용되는 제2 벡터 레지스터 필드(506, 예를 들어, 비트 12~15); 제3 벡터 레지스터( $V_3$ )를 지정하기 위해 사용되는 제3 벡터 레지스터 필드(508, 예를 들어, 비트 16~19); 및 RXB 필드(510, 예를 들어, 비트 36~39)를 포함한다. 필드들(504~510)의 각각은, 한 예에서, 별개이며 상기 오퍼코드(들)로부터 독립적이다. 또한, 한 실시 예에서, 그들은 별개이고 서로로부터 독립적이지만, 다른 실시 예들에서, 둘 이상의 필드가 결합될 수도 있다.

[0068] 다른 실시 예에서, 제3 벡터 레지스터 필드는 명령의 명시적 오퍼랜드(an explicit operand)로서 포함되지 않지만, 대신에 그것은 암시적 오퍼랜드(an implied operand) 또는 입력이다. 또한, 상기 오퍼랜드 내에 제공된 값은 다른 방법들로, 예를 들어, 범용 레지스터로, 메모리로, 주소 계산 등으로서 제공될 수 있다.

[0069] 또 다른 실시 예에서, 제3 오퍼랜드는, 그것이 명시적이든지 또는 암시적이든지, 전혀 제공되지 않는다.

[0070] 한 예에서, 오퍼코드 필드(502a)에 의해 지정된 오퍼코드의 선택된 비트들(예를 들어, 처음 두 비트들)은 이 명령의 길이를 명시한다. 이 특정 예에서, 선택된 비트들은 상기 길이가 3개 하프워드들(three halfwords)임을 나타낸다. 또한 상기 명령의 포맷은 확장된 오퍼코드 필드를 갖는 벡터 레지스터-및-레지스터 연산(a vector register-and-register operation with an extended opcode field)이다. 상기 벡터( $V$ ) 필드들의 각각은 RXB에 의해 명시되는 이 대응 확장 비트와 함께 벡터 레지스터를 지정한다. 구체적으로, 벡터 레지스터들에 있어서, 오퍼랜드를 보유하는 레지스터는 예를 들어 상기 레지스터 필드의 4 비트 필드에 자신의 대응 레지스터 확장 비트(RXB)를 최상위 비트로서 더한 것을 사용하여 명시된다.

[0071] 벡터 체크섬 명령의 한 실시 예의 실행에서, 제2 오퍼랜드로부터의 엘리먼트들은, 예를 들어, 워드-크기(word-sized)이고, 제3 오퍼랜드의 선택된 엘리먼트, 예를 들어, 제3 오퍼랜드의 워드 1 내의 엘리먼트와 서로 하나씩(one by one) 더해진다. (다른 실시 예에서, 상기 제3 오퍼랜드의 선택된 엘리먼트의 더하기는 선택적이다.) 상기 합(sum)은 제1 오퍼랜드의 선택된 위치, 예를 들어, 워드 1에 배치된다(placed). 0들이 제1 오퍼랜드의 기타 워드 엘리먼트들, 예를 들어, 워드 엘리먼트들 0 및 2~3에 배치된다. 상기 워드-크기의 엘리먼트들은 모두 32-비트 부호없는 2진 정수들로서 취급된다. 엘리먼트의 각각의 더하기 후에, 캐리(carry, 자리올림), 예를 들어, 상기 합의 비트 위치 0으로부터의 캐리는, 예를 들어, 제1 오퍼랜드의 워드 엘리먼트 1 내의 결과의 비트 위치 31에 더해진다.

[0072] 조건 코드(Condition Code): 상기 코드는 불변인 채로 있다.

[0073] 프로그램 예외들:

[0074] • 데이터 예외 코드(DXC) FE를 갖는 데이터, 벡터 명령, 벡터 퍼실리티가 인에이블되지 않음을 나타냄

[0075] • 연산(만일 z/Architecture를 위한 벡터 퍼실리티가 설치되어 있지 않은 경우)

[0076] • 트랜잭션 제한

[0077] 프로그래밍 노트:

[0078] 1. 제3 오퍼랜드의 콘텐츠는 체크섬 계산 알고리즘의 시작에서 영을 보유한다.

- [0079] 2. 16 비트 체크섬이 사용되는데, 예를 들어, TCP/IP 어플리케이션에서 사용된다. 아래의 프로그램은 32-비트 체크섬이 계산된 후에 실행될 수 있다:
- [0080] VERLLF V2,V1,16(0) (VERLLF - Vector Element Rotate Left Logical - 4-byte value)
- [0081] VAF V2,V1,V2 (VAF - Vector Add - 4 byte value)
- [0082] 엘리먼트 2 내의 하프워드는 16-비트 체크섬을 보유한다.
- [0083] 벡터 체크섬 명령에 관한 더 상세한 설명이 도 5b 및 5c를 참조하여 기술된다. 한 예에서, 도 5b는 벡터 체크섬 명령의 실행에서 프로세서에 의해서 수행되는 로직의 한 실시 예를 도시하고, 도 5c는 벡터 체크섬 명령의 실행의 한 예의 블록도를 도시한다.
- [0084] 도 5b를 참조하면, 초기에, 제1 오퍼랜드(OP1)에 대한 엘리먼트 인덱스(Ey)가 세트되는데, 예를 들어 1로 세트되며, 이는 제1 오퍼랜드의 엘리먼트 1을 나타낸다(단계 530). 비슷하게, 제3 오퍼랜드(OP3)에 대한 엘리먼트 인덱스(Ex)가 세트되는데, 예를 들어 1로 세트되며, 이는 제3 오퍼랜드의 엘리먼트 1을 나타낸다(단계 532). 그 다음, 엘리먼트 인덱스(Ei)는 0으로 세트되고, 엘리먼트 인덱스(Ey)에서의 엘리먼트, 즉 이 예에서 엘리먼트 1은 0으로 초기화된다(단계 534). 다른 실시 예에서, Ex 및 Ey는 임의의 유효 엘리먼트 인덱스로 세트될 수 있다.
- [0085] 엔드 어라운드 캐리(EAC) 애드(An end around carry (EAC) add)가 수행되어  $OP1(Ey) = OP1(Ey) + OP2(Ei) + OP2(Ei+1)$ 가 된다(단계 536). 따라서 출력 벡터(OP1)의 엘리먼트 1은 그 엘리먼트의 콘텐츠에 제2 오퍼랜드(OP2)의 엘리먼트 0 내의 값과 제2 오퍼랜드의 엘리먼트 1 내의 값을 더한 것과 동일하게 세트된다. 엔드 어라운드 캐리 애드로, 더하기 연산이 수행되고 그 더하기로부터의 모든 캐리는 상기 합에 다시 더해져서 새로운 합을 생산한다.
- [0086] 다른 실시 예에서, 전술한 더하기 대신에, 다음과 같이 수행된다: 임시 어큐뮬레이터 값(a temporary accumulator value)이 정의되고 0으로 초기화되며, 그런 다음 한 번에 한 엘리먼트가 더해진다(added at a time). 다른 실시 예에서, 모든 워드들은 병렬로 더해지고 임시 어큐뮬레이터는 없다. 다른 변형 예들도 또한 가능하다.
- [0087] 그 후, 제2 오퍼랜드 내에서 더해질 추가의 엘리먼트들이 있는지에 관한 결정이 내려진다(질의 단계 538). 예를 들어, 제2 오퍼랜드의 엘리먼트들의  $\# > Ei - 2$  인가? 만일 더해질 제2 오퍼랜드 엘리먼트들이 더 있다면, Ei는 증분되는데, 예를 들어 2만큼 증분되고(단계 540), 처리는 단계 536에서 계속된다.
- [0088] 제2 오퍼랜드에서 엘리먼트들의 더하기를 마친 후에, 그 결과가 제3 오퍼랜드 내의 값에 더해진다. 예를 들어, (모든 제2 오퍼랜드 엘리먼트들에 걸친 EAC 애드(add)의 합인) 제1 오퍼랜드의 엘리먼트(Ey)와 제3 오퍼랜드(OP3)의 엘리먼트(Ex) 내의 값의 엔드 어라운드 캐리 애드가 수행된다(즉,  $EAC\ ADD\ OP1(Ey) + OP3(Ex)$ )(단계 542). 이것은 도 5c에서 그림으로 도시되어 있다.
- [0089] 도 5c에 도시된 바와 같이, 제2 오퍼랜드(550)는 복수의 엘리먼트들(552a-552n)을 포함하고, 이들 엘리먼트들은 제3 오퍼랜드(560)의 워드 1(562) 내의 엘리먼트와 서로 하나씩 더해진다. 그 결과는 제1 오퍼랜드(570)의 엘리먼트 1(572) 내에 배치된다. 이것은 수학적으로 방정식 " $Ey = Ex + Ei$ 의 합"으로 도시되고, 여기서  $i=0$ 부터  $n$ 까지이고, 상기 더하기는 엔드 어라운드 캐리 더하기(an end around carry addition)이다.
- [0090] 전술한 바는 단순 산술(lane arithmetic)을 수행하는 대신에 벡터 레지스터의 엘리먼트들에 걸친 체크섬(a checksum)을 수행하는 벡터 체크섬 명령의 한 실시 예이다. 한 실시 예에서, 벡터 체크섬 명령은 체크섬들을 수행하는데, 엔드 어라운드 캐리 더하기들과 교차-합(sum-across)을 수행함으로써 한다. 한 예에서, 벡터 체크섬 명령은 벡터 레지스터로부터 네 개의 4-바이트 정수 엘리먼트들을 취하여 그들을 서로 더한다. 상기 더하기들로부터의 모든 캐리들은 다시 더해진다. 상기 4-바이트 합은 다른 오퍼랜드 내의 4-바이트 엘리먼트에 더해지고, 그 다음 또 다른 벡터 레지스터(yet a further vector register)(예를 들어, 벡터 레지스터의 상위 엘리먼트들(the higher order elements) 내에 영들이 저장된 벡터 레지스터의 하위 4-바이트 엘리먼트(the low order 4-byte element)) 내에 세이브된다(saved).
- [0091] 다른 실시 예에서, 추가의 벡터 레지스터 또는 다른 레지스터가 상기 값을 세이브하기 위해 사용되지 않지만, 대신에 다른 레지스터들(즉, 오퍼랜드들) 중 하나는 어큐뮬레이터(an accumulator)로서 사용된다.
- [0092] 제공되는 상기 체크섬은 데이터 무결성(data integrity)을 보존하기 위해 사용될 수 있다. 체크섬은 수신된 데

이터가 정확한지를 검증하기 위해(verify) 종종 데이터에 적용되어(applied) 잡음이 많은 채널을 통해 전송된다. 여기서 기술된 바와 같이, 이 예에서, 체크섬은 순차 4-바이트 정수들을 서로 더함으로써 계산된다. 만일 정수 산술 연산으로부터 캐리가 있다면, 그 캐리, 그리고 추가의 1이 런닝 합(the running sum)에 더해진다.

[0093] 여기서는 체크섬들이 기술되었지만, 다른 엔드 어라운드 더하기들을 위해 유사한 기법이 사용될 수 있다.

[0094] 본 발명의 특징에 따라 제공되는 다른 명령은 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트(VGFMA) 명령(a Vector Galois Field Multiply Sum and Accumulate (VGFMA) instruction)이며, 이것의 한 예가 도 6a에 도시된다. 한 예에서, 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트(VGFMA) 명령(600)은 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트(VGFMA) 명령을 나타내는 오퍼코드 필드들(602a, 예를 들어, 비트 0~7; 602b, 예를 들어, 비트 40~47); 제1 벡터 레지스터( $V_1$ )를 지정하기 위해 사용되는 제1 벡터 레지스터 필드(604, 예를 들어, 비트8~11); 제2 벡터 레지스터( $V_2$ )를 지정하기 위해 사용되는 제2 벡터 레지스터 필드(606, 예를 들어, 비트12~15); 제3 벡터 레지스터( $V_3$ )를 지정하기 위해 사용되는 제3 벡터 레지스터 필드(608, 예를 들어, 비트 16~19); 마스크 필드( $M_5$ )(610, 예를 들어, 비트 20~23); 제4 벡터 레지스터( $V_4$ )를 지정하기 위해 사용되는 제4 벡터 레지스터 필드(612, 예를 들어, 비트 32~35); 및 RXB 필드(614, 예를 들어, 비트 36~39)를 포함한다. 필드들(604~614)의 각각은, 한 예에서, 별개이며 상기 오퍼코드(들)로부터 독립적이다. 또한, 한 실시 예에서, 그들은 별개이고 서로로부터 독립적이지만, 다른 실시 예들에서, 둘 이상의 필드가 결합될 수도 있다.

[0095] 한 예에서, 오퍼코드 필드(602a)에 의해 지정된 오퍼코드의 선택된 비트들(예를 들어, 처음 두 비트들)은 이 명령의 길이를 명시한다. 이 특정 예에서, 선택된 비트들은 상기 길이가 3개 하프워드들(three halfwords)임을 나타낸다. 또한 상기 명령의 포맷은 확장된 오퍼코드 필드를 갖는 벡터 레지스터-및-레지스터 연산(a vector register-and-register operation with an extended opcode field)이다. 상기 벡터(V) 필드들의 각각은 RXB에 의해 명시되는 자신의 대응 확장 비트와 함께 벡터 레지스터를 지정한다. 구체적으로, 벡터 레지스터들에 있어서, 오퍼랜드를 보유하는 레지스터는 예를 들어 상기 레지스터 필드의 4-비트 필드에 자신의 대응 레지스터 확장 비트(RXB)를 최상위 비트로서 더한 것을 사용하여 명시된다.

[0096]  $M_5$  필드(610)는, 예를 들어, 4 비트, 즉 0~3을 가지며, 엘리먼트 크기(ES) 컨트롤을 명시한다. 상기 엘리먼트 크기 컨트롤(the element size control)은 벡터 레지스터 오퍼랜드들 2(two) 및 3(three) 내의 엘리먼트들의 크기를 명시하며; 제1 및 제4 오퍼랜드 내의 엘리먼트들은 상기 ES 컨트롤에 의해서 명시된 이들의 크기의 두 배이다(twice). 예를 들어,  $M_5$  내의 0의 값은 바이트-크기의 엘리먼트들을 나타내고; 1은 하프워드(halfword)를 나타내며; 2는 워드(word)를 나타내고; 그리고 3은 더블워드(doubleword)를 나타낸다.

[0097] 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트(VGFMA) 명령의 한 실시 예의 실행에서, 제2 오퍼랜드의 각 엘리먼트는 제3 오퍼랜드의 대응 엘리먼트와 갈로이스 필드(즉, 유한수의 엘리먼트들을 갖는 유한 필드(a finite field))에서 곱해진다. 다시 말하면, 제2 오퍼랜드의 각 엘리먼트는 캐리없는 곱셈(carryless multiplication)을 사용하여 제3 오퍼랜드의 대응 엘리먼트와 곱해진다. 한 예에서, 갈로이스 필드는 2의 차수(次數)(an order of two)를 갖는다. 이 곱셈은 표준 2진 곱셈과 비슷하지만, 시프트된 피승수(multiplicand)를 더하는 대신에, 그것은 배타적 논리합(XOR) 연산이 된다. 예를 들어, 더블 엘리먼트-크기의 곱들(double element-sized products)의 결과인 짝수-홀수 쌍들(the resulting even-odd pairs)은 서로 배타적 논리합(XOR) 연산이되고 그리고 제4 오퍼랜드의 대응 엘리먼트, 예를 들어, 이중-폭 엘리먼트(double-wide element)와 배타적 논리합(XOR) 연산이 된다. 그 결과들은, 예를 들어, 제1 오퍼랜드의 더블-와이드 엘리먼트들 내에 배치된다.

[0098] 조건 코드(Condition Code): 상기 코드는 불변인 채로 있다.

[0099] 프로그램 예외들:

[0100] • 데이터 예외 코드(DXC) FE를 갖는 데이터, 벡터 명령, 벡터 퍼실리티가 인에이블되지 않음을 나타냄

[0101] • 연산(만일 z/Architecture를 위한 벡터 퍼실리티가 설치되어 있지 않은 경우)

[0102] • 명세



- [0103] • 트랜잭션 제한
- [0104] 다른 실시 예에서, 상기 명령은 하나 또는 그 이상의 더 적은 오퍼랜드들(fewer operands)을 포함할 수 있다. 예를 들어, 제4 오퍼랜드 대신에, 배타적 논리합 연산이 될 값을 제1 오퍼랜드 내에 있게 하며, 이것은 그 결과들도 또한 포함한다. 다른 변형 예들도 또한 가능하다.
- [0105] 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트 명령의 실행의 한 실시 예에 관한 더 상세한 설명이 도 6b 및 6c를 참조하여 기술된다. 한 예에서, 도 6b는 벡터 갈로이스 필드 멀티플라이 섬 및 어큐물레이트 명령을 실행하기 위하여 프로세서에 의해서 수행되는 로직의 한 실시 예를 도시하고, 도 6c는 상기 로직의 실행을 도시하는 블록도의 한 예를 도시한다.
- [0106] 도 6b를 참조하면, 초기에, 짝수/홀수 쌍들(even/odd pairs)이 제2 오퍼랜드(OP2), 제3 오퍼랜드(OP3), 및 제4 오퍼랜드(OP4)로부터 추출되어(단계 630), 캐리없는 멀티플라이 섬 어큐물레이트 기능(a carryless multiply sum accumulate function)이 수행된다(단계 632). 예를 들어, 2의 거듭제곱(a power of 2)에 관하여 갈로이스 필드 내 연산할 때, 캐리없는 곱셈은 시프트와 XOR(배타적 OR)이며, 이는 모든 캐리를 효과적으로 무시한다. 결과는 제1 오퍼랜드(OP1) 내에 배치되고(단계 634), 추출되어야 할 쌍들이 더 있는지에 관한 결정이 질의 단계 636에서 내려진다. 만일 쌍들이 더 있다면, 처리는 단계 630에서 계속되고; 그렇지 않으면, 처리는 완료된다(단계 638). 한 예에서, 엘리먼트 크기(631)는 단계들 630~634에 대한 입력이다.
- [0107] 단계 632의 캐리없는 멀티플라이 섬 어큐물레이트 기능에 관한 더 상세한 설명이 도 6c를 참조하여 기술된다. 도시한 바와 같이, 오퍼랜드들 OP2H(652a), OP2L(652b)의 쌍이 제2 오퍼랜드(650)로부터 추출된다. 또한, 오퍼랜드 쌍 OP3H(662a), OP3L(662b)가 제3 오퍼랜드(660)로부터 추출되고, 오퍼랜드 쌍 OP4H(672a), OP4L(672b)가 제4 오퍼랜드(670)로부터 추출된다. 오퍼랜드 OP2H(652a)는 오퍼랜드 OP3H(662a)에 캐리없는 곱셈으로 곱해지고, 결과 H(680a)가 제공된다. 비슷하게, 오퍼랜드 OP2L(652b)는 오퍼랜드 OP3L(662b)에 캐리없는 곱셈을 사용하여 곱해지고, 결과 L(680b)이 제공된다. 그 다음에 결과 H(680a)는 결과 L(680b)과 배타적 논리합(XOR) 연산이 되고, 그 결과는 오퍼랜드 OP4H(672a) 및 OP4L(672b)와 배타적 논리합 연산이 되며, 그 결과는 OP1H(690a), OP1L(690b) 내에 배치된다.
- [0108] 캐리없는 멀티플라이 연산을 수행하고 그 다음에 최종 배타적 논리합(XOR)을 수행하여 어큐물레이트된 합을 생성하는 벡터 명령에 관하여 지금까지 기술하였다. 이 기법은 2의 차수(次數)를 갖는 유한 필드 내에서 연산을 수행하는 에러 검출 코드들 및 암호화 방법(cryptography)의 다양한 실시 예들에서 사용될 수 있다.
- [0109] 한 예에서, 상기 명령은 벡터 레지스터의 복수의 엘리먼트들 상에서 캐리없는 멀티플라이 연산을 수행하여 합을 획득한다. 또한, 상기 명령은 상기 합에 대해서 최종 배타적 OR을 수행하여 어큐물레이트된 합을 생성한다. 상기 명령은, 실행될 때, 갈로이스 필드 내에서 제2 벡터 및 제3 벡터의 대응 엘리먼트들을 곱하고, 시프트된 피승수(the shifted multiplicand)는 배타적 논리합(XOR) 연산된다. 각각의 이중-폭 곱(double-wide product)도 서로 XOR 연산되고, 그 결과는 제1 벡터의 이중-폭 대응 엘리먼트와 XOR 연산된다. 그리고 그 결과는 제1 벡터 레지스터 내에 저장된다. 더블-워드 엘리먼트들이 위에서 기술되었지만, 다른 엘리먼트 크기들의 워드-크기의 엘리먼트들이 사용될 수도 있다. 상기 명령은 다수의 다른 엘리먼트 크기들에서도 연산할 수 있다.
- [0110] 본 발명의 한 특징에 따라 제공된 또 다른 명령이 벡터 제너레이트 마스크(VGM) 명령(a Vector Generate Mask (VGM) instruction)이고, 이 명령의 예가 도 7a에 도시된다. 한 예에서, 벡터 제너레이트 마스크(VGM) 명령(700)은 벡터 제너레이트 마스크 연산을 나타내는 오퍼코드 필드들(702a, 예를 들어, 비트 0~7; 702b, 예를 들어, 비트 40~47); 제1 벡터 레지스터( $V_1$ )를 지정하기 위해 사용되는 제1 벡터 레지스터 필드(704, 예를 들어, 비트 8~11); 제1 값을 명시하기 위해 사용되는 제1 즉시 필드(a first immediate field)( $I_2$ )(706, 예를 들어, 비트 16~24); 제2 값을 명시하기 위해 사용되는 제2 즉시 필드(a second immediate field)( $I_3$ )(708, 예를 들어, 비트 24~32); 마스크 필드(a mask field)( $M_4$ )(710, 예를 들어, 비트 32~35); 및 RXB 필드(712, 예를 들어, 비트 36~39)를 포함한다. 필드들(704~712)의 각각은, 한 예에서, 별개이며 상기 오퍼코드(들)로부터 독립적이다. 또한, 한 실시 예에서, 그들은 별개이고 서로로부터 독립적이지만, 다른 실시 예들에서, 둘 이상의 필드가 결합될 수도 있다.
- [0111] 다른 실시 예에서, 상기 제1 값 및/또는 제2 값은, 예를 들어, 범용 레지스터로, 메모리로, 벡터 레지스터의 한 엘리먼트로(엘리먼트마다 다름) 또는 주소 계산(an address computation)으로부터 제공될 수 있다. 그것은 명령의 명시적 오퍼랜드(an explicit operand)로서 또는 암시적 오퍼랜드(an implied operand) 또는 입력으로서

포함될 수 있다.

[0112] 한 예에서, 오퍼코드 필드(702a)에 의해 지정된 오퍼코드의 선택된 비트들(예를 들어, 처음 두 비트들)은 이 명령의 길이를 명시한다. 이 특정 예에서, 선택된 비트들은 상기 길이가 3개 하프워드들(three halfwords)임을 나타낸다. 또한 상기 명령의 포맷은 확장된 오퍼코드 필드를 갖는 벡터 레지스터-및-즉시 연산(a vector register-and-immediate operation with an extended opcode field)이다. 상기 벡터(V) 필드들의 각각은 RXB에 의해 명시되는 자신의 대응 확장 비트와 함께 벡터 레지스터를 지정한다. 구체적으로, 벡터 레지스터들에 있어서, 오퍼랜드를 보유하는 레지스터는 예를 들어 상기 레지스터 필드의 4-비트 필드에 자신의 대응 레지스터 확장 비트(RXB)를 최상위 비트로서 더한 것을 사용하여 명시된다.

[0113]  $M_4$  필드(710)는, 예를 들어, 엘리먼트 크기 (ES) 컨트롤을 명시한다. 상기 엘리먼트 사이즈 컨트롤은 벡터 레지스터 오퍼랜드들 내 엘리먼트들의 사이즈를 명시한다. 한 예에서,  $M_4$  필드의 비트 0은 한 바이트(a byte)를 명시하고; 비트 1은 하프워드(halfword)(예를 들어, 2바이트)를 명시하며; 비트 2는 워드(word)(예를 들어, 4 바이트; 즉, 풀워드)를 명시하고; 그리고 비트 3은 더블워드(doubleword)를 명시한다.

[0114] 벡터 제너레이트 마스크 명령의 한 실시 예의 실행에서, 제1 오퍼랜드 내 각 엘리먼트에 대해, 비트 마스크가 생성된다. 상기 마스크는 1로 세트된 비트들을 포함하는데, 이들은, 예를 들어,  $I_2$  내의 부호 없는 정수 값에 의해서 명시된 비트 위치에서 시작하여, 예를 들어,  $I_3$  내의 부호 없는 정수 값에 의해서 명시된 비트 위치로 끝난다. 모든 다른 비트 위치들은 0으로 세트된다. 한 예에서, 명시된 엘리먼트 크기를 위해 비트 위치들 모두를 표시하기 위해 필요한 비트들의 수만  $I_2$  및  $I_3$  필드들로부터 사용되고; 다른 비트들은 무시된다. 만일  $I_2$  필드 내의 비트 위치가  $I_3$  필드 내의 비트 위치보다 크다면, 비트들의 범위는 명시된 엘리먼트 크기에 대해 최대 비트 위치에서 랩한다(wrap). 예를 들어, 바이트-크기의 엘리먼트들을 가정할 때, 만일  $I_2=1$ 이고  $I_3=6$ 이라면, 결과 마스크(the resulting mask)는  $X'7E'$  또는  $B'01111110'$ 이다. 그러나, 만일  $I_2=6$ 이고  $I_3=1$ 이라면, 결과 마스크(the resulting mask)는  $X'81'$  또는  $b'10000001'$ 이다.

[0115] 조건 코드(Condition Code): 상기 코드는 불변인 채로 있다.

[0116] 프로그램 예외들:

- [0117] • 데이터 예외 코드(DXC) FE를 갖는 데이터, 벡터 명령, 벡터 퍼실리티가 인에이블되지 않음을 나타냄
- [0118] • 연산(만일 z/Architecture를 위한 벡터 퍼실리티가 설치되어 있지 않은 경우)
- [0119] • 명세
- [0120] • 트랜잭션 제한

[0121] 벡터 제너레이트 마스크 명령의 한 실시 예에 관한 더 상세한 설명이 도 7b 및 7c를 참조하여 기술된다. 구체적으로, 도 7b는 프로세서에 의해서 수행되는 벡터 제너레이트 마스크 명령과 관련된 로직의 한 실시 예를 도시하고, 도 7c는 벡터 제너레이트 마스크 명령의 실행의 한 실시 예를 예시하는 블록도의 한 예를 도시한다.

[0122] 도 7b를 참조하면, 초기에, 한 마스크가 제1 오퍼랜드 내 각 엘리먼트를 위해 생성된다(단계720). 이 단계는 다양한 입력들을 사용하는데, 이 입력들에는 출발 위치(the starting position)로서 제2 오퍼랜드 필드 내에 명시된 값(722), 그리고 종료 위치(the ending position)로서 제3 오퍼랜드 필드 내에 명시된 값(724), 그리고  $M_4$  필드 내에 명시된 엘리먼트들의 크기(726)가 포함된다. 이들 입력들은 상기 마스크를 생성하고 제1 오퍼랜드(OP1)의 선택된 엘리먼트, 예를 들어, 엘리먼트 0의 위치들을 채우기 위해 사용된다(단계730). 예를 들어, 제1 오퍼랜드(OP1)의 엘리먼트 0은 복수의 위치들(예를 들어, 비트 위치들)을 포함하는데,  $I_2$  내의 부호없는 정수 값에 의해서 명시된 위치에서 시작하고,  $I_3$  내의 부호없는 정수 값에 의해서 명시된 위치에서 종료하며, 제1 오퍼랜드의 엘리먼트 0의 위치들(예를 들어, 비트들)은 1로 세트된다. 다른 비트 위치들은 0으로 세트된다. 그 후, 제1 오퍼랜드 내의 엘리먼트들이 더 있는지에 관한 결정이 내려진다(질의 단계 734). 만일 엘리먼트들이 더 있다면, 처리는 단계 720에서 계속된다. 그렇지 않으면, 처리는 완료된다(단계736).

- [0123] 상기 마스크의 생성과 제1 오퍼랜드의 채움(the filling)이 도 7c에 그림으로 도시되어 있다. 도시한 바와 같이, 제1 오퍼랜드의 각 엘리먼트를 위한 마스크들이 생성되는데(720) 입력들(예를 들어, 722~726)을 사용하여 생성되고, 상기 마스크들을 생성하는 것의 결과들은 제1 오퍼랜드의 엘리먼트들(740) 내에 저장된다.
- [0124] 벡터의 각 엘리먼트를 위한 비트 마스크들을 생성하는 명령에 관해 위에서 상세하게 기술하였다. 한 실시 예에서, 상기 명령은 시작 비트 위치 및 종료 비트 위치를 취하고 각 엘리먼트에 대해 복제되는 비트 마스크를 생성한다. 상기 명령은 비트 범위를 명시하고, 상기 범위 내에서 각 비트는 벡터 레지스터의 각 엘리먼트를 위해 1로 세트되며, 한편 다른 비트들은 0으로 세트된다.
- [0125] 한 실시 예에서, 비트 마스크들을 생성하는 명령을 사용하는 것은 유익함을 제공하는데, 예를 들어, 명령 스트림의 캐시 풋프린트(a cache footprint of an instruction stream)을 증가시키며 얼마나 많은 마스크들이 필요한가에 따라서는 크리티컬 루프에서(in a critical loop) 대기시간(latency)을 증가시킬 수도 있는, 메모리로부터 비트 마스크들을 로드하는 것보다 유익함을 제공한다.
- [0126] 본 발명의 한 특징에 따라 제공된 또 다른 명령은 벡터 엘리먼트 로테이트 및 인서트 언더 마스크(VERIM) 명령(a Vector Element Rotate and Insert Under Mask (VERIM) instruction)이고, 이 명령의 예가 도 8a에 도시되어 있다. 한 예에서, 벡터 엘리먼트 로테이트 및 인서트 언더 마스크(VERIM) 명령(800)은 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 연산을 나타내는 오퍼코드 필드들(802a, 예를 들어, 비트 0~7; 802b, 예를 들어, 비트 40~47); 제1 벡터 레지스터( $V_1$ )를 지정하기 위해 사용되는 제1 벡터 레지스터 필드(804, 예를 들어, 비트 8~11); 제2 벡터 레지스터( $V_2$ )를 지정하기 위해 사용되는 제2 벡터 레지스터 필드(806, 예를 들어, 비트 12~15); 제3 벡터 레지스터( $V_3$ )를 지정하기 위해 사용되는 제3 벡터 레지스터 필드(808, 예를 들어, 비트 16~19); 예를 들어, 각 엘리먼트를 회전하기 위한 비트들의 수를 명시하는 부호없는 정수를 포함하는 즉시 필드(an immediate field)( $I_4$ )(812, 예를 들어, 비트 24~31); 마스크 필드(a mask field)( $M_5$ )(814, 예를 들어, 비트 32~35); 및 RXB 필드 (816, 예를 들어, 비트 36~39)를 포함한다. 필드들(804~816)의 각각은, 한 예에서, 별개이며 상기 오퍼코드(들)로부터 독립적이다. 또한, 한 실시 예에서, 그들은 별개이고 서로로부터 독립적이지만, 다른 실시 예들에서, 둘 이상의 필드가 결합될 수도 있다.
- [0127] 한 예에서, 오퍼코드 필드(802a)에 의해 지정된 오퍼코드의 선택된 비트들(예를 들어, 처음 두 비트들)은 이 명령의 길이를 명시한다. 이 특정 예에서, 선택된 비트들은 상기 길이가 3개 하프워드들(three halfwords)임을 나타낸다. 또한 상기 명령의 포맷은 확장된 오퍼코드 필드를 갖는 벡터 레지스터-및-즉시 연산(a vector register-and-immediate operation with an extended opcode field)이다. 상기 벡터( $V$ ) 필드들의 각각은 RXB에 의해 명시되는 자신의 대응 확장 비트와 함께 벡터 레지스터를 지정한다. 구체적으로, 벡터 레지스터들에 있어서, 오퍼랜드를 보유하는 레지스터는 예를 들어 상기 레지스터 필드의 4-비트 필드에 자신의 대응 레지스터 확장 비트(RXB)를 최상위 비트로서 더한 것을 사용하여 명시된다.
- [0128]  $M_5$  필드는 엘리먼트 사이즈(ES) 컨트롤을 명시한다. 상기 엘리먼트 사이즈 컨트롤은 벡터 레지스터 오퍼랜드들 내 엘리먼트들의 사이즈를 명시한다. 한 예에서,  $M_5$  필드의 비트 0은 한 바이트(a byte)를 명시하고; 비트 1은 하프워드(halfword)(예를 들어, 2바이트)를 명시하며; 비트 2는 워드(word)(예를 들어, 4 바이트; 즉, 풀워드)를 명시하고; 그리고 비트 3은 더블워드(doubleword)를 명시한다.
- [0129] 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령의 한 실시 예의 실행에서, 제2 오퍼랜드의 각 엘리먼트는 제4 오퍼랜드에 의해서 명시된 비트들의 수만큼 왼쪽으로 회전된다(rotated left by the number of bits). 상기 엘리먼트의 최좌측 비트 위치로부터 시프트된 각 비트는 상기 엘리먼트의 최우측 비트 위치 내에 재입력된다(reenter). 제3 오퍼랜드는 각 엘리먼트 내 마스크를 포함한다. 1(one)인 제3 오퍼랜드 내의 각 비트를 위해, 제2 오퍼랜드 내의 회전된 엘리먼트들의 대응 비트는 제1 오퍼랜드 내의 대응 비트를 대체한다(replace). 다시 말하면, 상기 회전된 엘리먼트들의 대응 비트의 값은 제1 오퍼랜드 내의 대응 비트의 값을 대체한다. 0(zero)인 제3 오퍼랜드 내의 각 비트를 위해, 제1 오퍼랜드의 대응 비트는 변하지 않는다. 제1 오퍼랜드가 제2 또는 제3 오퍼랜드와 동일할 때를 제외하고, 제2 및 제3 오퍼랜드들은 변하지 않는다.
- [0130] 제4 오퍼랜드는, 예를 들어, 부호없는 2진 정수이고, 이는 제2 오퍼랜드 내의 각 엘리먼트를 회전하기 위한 비트들의 수를 명시한다. 만일 이 값이 명시된 엘리먼트 크기에서의 비트들의 수보다 크다면, 이 값은 상기 엘리먼트 내의 비트들의 수의 모듈로(modulo) 연산으로 감소된다.

- [0131] 한 예에서, 제3 오퍼랜드 내에 포함된 상기 마스크는 여기서 기술한 VGM 명령을 사용하여 생성된다.
- [0132] 조건 코드(Condition Code): 상기 코드는 불변인 채로 있다.
- [0133] 프로그램 예외들:
- [0134] • 데이터 예외 코드(DXC) FE를 갖는 데이터, 벡터 명령, 벡터 퍼실리티가 인에이블되지 않음을 나타냄
- [0135] • 연산(만일 z/Architecture를 위한 벡터 퍼실리티가 설치되어 있지 않은 경우)
- [0136] • 명세
- [0137] • 트랜잭션 제한
- [0138] 프로그래밍 노트:
- [0139] 1. VERIM 및 VGM의 조합이 로테이트 및 인서트 선택 비트들 명령(a Rotate and Insert Selected Bits instruction)의 완전 기능(the full functionality)을 달성하기 위해 사용될 수 있다.
- [0140] 2.  $I_4$  필드의 비트들은 각 엘리먼트를 왼쪽으로 회전하기 위한 비트들의 수를 명시하는 부호없는 2진 정수를 보유하도록 정의되지만, 오른쪽-회전 량(a rotate-right amount)을 효과적으로 명시하는 음의 값(a negative value)이 코드될 수 있다.
- [0141] 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령의 실행에 관한 더 상세한 설명이 도 8b 및 8c를 참조하여 기술된다. 특히, 도 8b는 프로세서에 의해서 수행되는 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령과 관련된 로직의 한 실시 예를 도시하고, 도 8c는 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령의 실행의 한 예를 그림으로 도시한다.
- [0142] 도 8b를 참조하면, 제2 오퍼랜드의 선택된 엘리먼트가 제4 오퍼랜드에서 명시된 양(amount)(820) 만큼 회전된다(단계 830). 만일 제4 오퍼랜드 내에 명시된 값이 엘리먼트 크기(822) 내에 명시된 비트들의 수보다 크다면, 그 값은 상기 엘리먼트 내 비트들의 수의 모듈로(modulo) 연산으로 감소된다.
- [0143] 상기 엘리먼트의 비트들을 회전시킨 후, 마스크 하의 머지(a merge under mask)가 수행된다(단계 832). 예를 들어, 1인 제3 오퍼랜드 내의 각 비트(824)를 위해, 제2 오퍼랜드 내의 회전된 엘리먼트의 대응 비트는 제1 오퍼랜드 내의 대응 비트를 대체한다.
- [0144] 그 후, 회전될 엘리먼트들이 더 있는지에 관한 결정이 내려진다(질의 단계 834). 만일 회전될 엘리먼트들이 더 있다면, 처리는 단계 830에서 계속된다. 그렇지 않으면, 처리는 완료된다(단계 836).
- [0145] 도 8c를 참조하면, 도시한 바와 같이, 제2 오퍼랜드의 엘리먼트들이 입력들(820 및 822)에 기초하여 회전된다(830). 또한, 마스크 하의 머지가 입력(824)을 이용하여 수행된다(832). 출력은 제1 오퍼랜드(850) 내에 제공된다.
- [0146] 위에서 벡터 엘리먼트 로테이트 및 인서트 언더 마스크 명령의 한 예를 기술하였다. 이 명령은 선택된 오퍼랜드 내의 엘리먼트들을 비트들의 정의된 수만큼 회전하기 위해 사용된다. 비트들이 명시되었더라도, 다른 실시 예에서, 엘리먼트들은 위치들의 수만큼 회전될 수 있고, 위치들은 비트들이 아닐 수도 있다. 또한, 상기 명령은 다른 엘리먼트 크기들에도 사용될 수 있다.
- [0147] 한 예로서, 그러한 명령은 테이블 룩업들을 위한 수들(numbers for table lookups)로부터 특정 비트 범위들을 선택하기 위해 사용된다.
- [0148] 특정 벡터 명령들 또는 기타 SIMD 연산들의 실행 동안, 예외(an exception)가 일어날 수 있다. SIMD 연산 상에서 예외가 일어났을 때, 보통으로, 벡터 레지스터의 어느 엘리먼트가 그 예외를 일으켰는지 알려지지 않는다. 소프트웨어 인터럽트 핸들러(a software interrupt handler)는 어느 엘리먼트 또는 엘리먼트들이 예외를 일으켰는지를 결정하기 위해 각 엘리먼트를 추출하여 스칼라 모드에서 계산을 다시 해야한다. 그러나, 본 발명의 한 특징에 따라서, 기계(예를 들어, 프로세서)가 벡터 연산 때문에 프로그램 인터럽트를 처리할 때, 엘리먼트 인덱스가 보고되는데, 이는, 예를 들어, 예외를 일으켰던 벡터 내의 가장 낮게 인덱스된 엘리먼트(the lowest indexed element in the vector)를 나타낸다. 그 다음에, 소프트웨어 인터럽트 핸들러는 문제의 엘리먼트로 즉



시 건너가서 요구되거나 원하는 조치들을 수행할 수 있다.

[0149] 예를 들어, 한 실시 예에서, 벡터 데이터 예외가 프로그램 인터럽션을 일으킬 때, 벡터 예외 코드(VXC)가 예를 들어, 실제 메모리 위치(예를 들어, 위치 (147)(X<sup>93</sup>)에 저장되고, 0들이 예를 들어, 실제 메모리 위치들 (144~146)(X<sup>90</sup> ~ X<sup>92</sup>)에 저장된다. 다른 실시 예에서, 만일 지정된 제어 레지스터(예를 들어, CR0)의 명시된 비트(예를 들어, 비트45)가 1이면, VXC는 또한 부동 소수점 제어 레지스터의 데이터 예외 코드(DXC) 내에도 배치된다. 제어 레지스터 0의 비트 45가 0이고 제어 레지스터 0의 비트 46이 1일 때, FPC 레지스터의 DXC와 위치(147)(X<sup>93</sup>)에서 스토리지의 콘텐츠는 예측불가능하다.

[0150] 한 실시 예에서, VXC는 벡터 부동 소수점 예외들의 다양한 종류 사이를 구별하여 어느 엘리먼트가 예외를 일으키는지를 표시한다. 한 예에서, 도 9a에 도시된 바와 같이, 벡터 예외 코드(900)는 벡터 인덱스(VIX)(902), 및 벡터 인터럽트 코드(VIC)(904)를 포함한다. 한 예에서, 상기 벡터 인덱스는 벡터 예외 코드의 비트 0~3을 포함하고, 그 값은 예외를 인지한 선택된 벡터 레지스터의 최좌측 엘리먼트의 인덱스이다. 또한, 상기 벡터 인터럽트 코드는 상기 벡터 예외 코드의 비트 4~7 내에 포함되고, 예로서, 다음의 값들을 갖는다:

|        |      |                                |
|--------|------|--------------------------------|
| [0151] | 0001 | IEEE 무효 연산                     |
| [0152] | 0010 | IEEE 0으로 나누기(Division by zero) |
| [0153] | 0011 | IEEE 오버플로(Overflow)            |
| [0154] | 0100 | IEEE 언더플로(Underflow)           |
| [0155] | 0101 | IEEE 부정확(Inexact)              |

[0156] 다른 실시 예에서, VXC는 예외를 일으키는 엘리먼트의 벡터 인덱스 또는 다른 위치 표시자(position indicator)만을 포함한다.

[0157] 한 실시 예에서, VXC는 다수의 명령들에 의해서 세트될 수 있는데, 이들은, 예를 들어, 다음 명령들을 포함한다: 기타 종류의 벡터 부동 소수점 명령들 및/또는 기타 명령들뿐만 아니라, 예들로서, 벡터 부동 소수점(FP) 애드(Vector Floating Point (FP) Add), 벡터 FP 컴페어 스칼라(Vector FP Compare Scalar), 벡터 FP 컴페어 이퀄(Vector FP Compare Equal), 벡터 FP 컴페어 하이 또는 이퀄(Vector FP Compare High or Equal), 고정 64-비트로부터 벡터 FP 변환(Vector FP Convert From Fixed 64-Bit), 로지컬 64-비트로부터 벡터 FP 변환(Vector FP Convert From Logical 64-Bit), 고정 64-비트의 벡터 FP 변환(Vector FP Convert to Fixed 64-Bit), 로지컬 64-비트의 벡터 FP 변환(Vector FP Convert to Logical 64-Bit), 벡터 FP 디바이드(Vector FP Divide), 벡터 로드 FP 정수(Vector Load FP Integer), 벡터 FP 로드 력션드(Vector FP Load Lengthened), 벡터 FP 로드 라운디드(Vector FP Load Rounded), 벡터 FP 멀티플라이(Vector FP Multiply), 벡터 FP 멀티플라이 및 애드(Vector FP Multiply and Add), 벡터 FP 멀티플 및 서브트랙트(Vector FP Multiple and Subtract), 벡터 FP 스퀘어 루트(Vector FP Square Root), 그리고 벡터 FP 서브트랙트( Vector FP Subtract).

[0158] 벡터 예외 코드를 세트하는 것에 관한 더 상세한 설명이 도 9b를 참조하여 기술된다. 한 실시 예에서, 컴퓨팅 환경의 프로세서는 이 로직을 수행한다.

[0159] 도 9b를 참조하면, 초기에, 벡터 레지스터 상에서 연산하는 명령이 실행되는데, 예를 들면, 위에서 열거한 명령들 중 하나 또는 그와 다른 명령이 실행된다(단계920). 상기 명령의 실행 동안, 예외 조건을 마주치게 된다(encountered)(단계 922). 한 예에서, 이 예외 조건은 인터럽트를 일으킨다. 벡터의 어느 엘리먼트가 예외를 일으켰는지에 관한 결정이 내려진다(단계 924). 예를 들어, 벡터 레지스터의 하나 또는 그 이상의 엘리먼트들에 관하여 계산을 수행하는 프로세서의 하나 또는 그 이상의 하드웨어 유닛들이 예외를 결정하고 신호를 제공한다. 예를 들어, 만일 복수의 하드웨어들이 벡터 레지스터의 복수의 엘리먼트들에 관하여 계산을 병렬로 수행하고 있고, 상기 엘리먼트들의 하나 또는 그 이상의 처리 동안 예외를 마주치게 되면, 예외를 마주쳤던 처리를 수행하는 하드웨어 유닛(들)은, 그것이 처리 중이었던 엘리먼트의 표시(indication)뿐만 아니라, 예외 조건을 신호로 내어 보낸다(signal). 다른 실시 예에서, 만일 벡터의 엘리먼트들이 순차로 실행되고, 예외를 엘리먼트의 처리 동안 마주친다면, 하드웨어는 그 예외가 일어났을 때 그것이 작업중이었던 시퀀스 내의 어떤 엘리먼트를 표시할 것이다.

[0160] 신호로 보내지는 예외에 기초하여, 벡터 예외 코드가 세트된다(단계 926). 이것은, 인터럽트 코드뿐만 아니라,

예를 들어, 예외를 일으킨 벡터 레지스터 내의 엘리먼트 위치를 표시하는 것을 포함한다.

- [0161] 효율적인 벡터 예외 처리(efficient vector exception handling)를 제공하는 벡터 예외 코드가 위에서 상세히 기술되었다. 한 예에서, 기계가 벡터 연산 때문에 프로그램 인터럽트를 처리할 때, 엘리먼트 인덱스가 보고되는데, 이는 예외를 일으킨 벡터 레지스터 내의 가장 낮게 인덱스된 엘리먼트(a lowest indexed element)를 나타낸다. 특정 예로서, 만일 벡터 애드(a vector add)가 수행되고 벡터 레지스터 당 두 개의 엘리먼트들이 있다면, 즉 A0+B0 및 A1+B1을 제공하는데, A0+B0에 대해서는 부정확한 결과가 수신되지만, A1+B1에 대해서는 그렇지 않다면, VIX는 0으로 세트되고 VIC는 0101로 세트된다. 다른 예에서, 만일 A0+B0가 예외를 수신하지 않지만, A1+B1은 예외를 수신하는 일이 일어나면, VIX는 1로 세트된다(VIC=0101). 만일 둘 모두 예외를 취한다면, VIX는 0으로 세트되는데 그 이유는 그것이 최좌측 인덱스된 위치(the leftmost indexed position)이고 VIC=0101이기 때문이다.
- [0162] 벡터 레지스터 내 예외의 위치를 표시하는 벡터 예외 코드뿐만 아니라, 다양한 벡터 명령들에 관해서 위에서 상세하게 기술하였다. 제공된 흐름도들에서, 일부 처리는 순차적인 것으로 보여질 수 있지만; 그러나, 하나 또는 그 이상의 실시 예들에서, 엘리먼트들은 병렬로 처리되고, 따라서, 예를 들어, 처리될 엘리먼트들이 더 있는지를 체크할 필요가 없을 수 있다. 다수의 다른 변형 예들도 또한 가능하다.
- [0163] 또한, 다른 실시 예들에서, 명령의 하나 또는 그 이상의 필드들의 콘텐츠는 범용 레지스터, 메모리로, 벡터 레지스터의 한 엘리먼트(엘리먼트마다 다름)로 또는 주소 계산으로부터 제공될 수 있다. 그들은 명령의 명시적 오퍼랜드(an explicit operand)로서 또는 암시적 오퍼랜드(an implied operand) 또는 입력으로서 포함될 수 있다. 또한, 하나 또는 그 이상의 명령들은 더 적은(less) 오퍼랜드들 또는 입력들을 사용할 수 있고, 대신에, 하나 또는 그 이상의 오퍼랜드들은 다수의 연산들 또는 단계들을 위해 사용될 수 있다.
- [0164] 더 나아가, 명령 필드 내에 엘리먼트 크기 컨트롤을 포함하는 대신에, 엘리먼트 크기 컨트롤은, 여기서 기술한 바와 같이, 다른 방법들로 제공될 수 있다. 또한, 엘리먼트 크기는 오퍼코드에 의해서 지정될 수도 있다. 예를 들어, 명령의 특정 오퍼코드는 엘리먼트의 크기 등 뿐만 아니라 연산을 명시한다.
- [0165] 여기에서, 메모리, 메인 메모리, 스토리지 및 메인 스토리지는 명시적으로 또는 맥락적으로 다르게 언급되지 않는 한 교환하여 사용할 수 있다.
- [0166] 이 기술분야에서 통상의 지식을 가진 자는 인식할 수 있는 바와 같이, 본 발명의 특징들은 시스템, 방법 또는 컴퓨터 프로그램 제품으로 구현될 수 있다. 따라서, 본 발명의 특징들은 전적으로 하드웨어 실시 예, 전적으로 소프트웨어 실시 예(펌웨어, 상주 소프트웨어, 마이크로-코드 등 포함) 또는 소프트웨어와 하드웨어 특징들을 조합한 실시 예(여기에서는 모두 "회로", "모듈", "시스템"으로 불릴 수 있음)의 형태를 취할 수 있다. 또한, 본 발명의 특징들은 컴퓨터 판독 가능 프로그램 코드가 그 위에 구현된 하나 또는 그 이상의 컴퓨터 판독 가능 매체(들)에 구현된 컴퓨터 프로그램 제품의 형태를 취할 수 있다.
- [0167] 하나 또는 그 이상의 컴퓨터 판독 가능 매체(들)의 임의의 조합이 사용될 수 있다. 컴퓨터 판독 가능 매체는 컴퓨터 판독 가능 스토리지 매체일 수 있다. 컴퓨터 판독 가능 스토리지 매체는, 예를 들어, 전자, 자기, 광학, 전자자기, 적외선 또는 반도체 시스템, 장치, 또는 디바이스이거나 전술한 것들의 모든 적절한 조합으로 될 수 있으나 그에 한정되지는 않는다. 컴퓨터 판독 가능 스토리지 매체의 더 구체적인 예들(비포괄적인 목록)에는 휴대용 컴퓨터 디스켓, 하드 디스크, 랜덤 액세스 메모리(RAM), 판독-전용 메모리(ROM), 소거 및 프로그램가능 판독-전용 메모리(EPROM 또는 플래시 메모리), 휴대용 콤팩트 디스크 판독-전용 메모리(CD-ROM), 광 스토리지 디바이스, 자기 스토리지 디바이스, 또는 전술한 것들의 모든 적절한 조합이 포함된다. 이 문서의 컨텍스트에서, 컴퓨터 판독 가능 스토리지 매체는 명령 실행을 위한 시스템, 장치, 또는 디바이스에 의해 또는 그와 연결하여 사용할 프로그램을 포함 또는 저장할 수 있는 모든 유형의(tangible) 매체일 수 있다.
- [0168] 이제 도 10을 참조하면, 한 예에서, 컴퓨터 프로그램 제품(1000)은 예를 들어 하나 또는 그 이상의 비-일시적인(non-transitory) 컴퓨터 판독 가능 스토리지 매체(1002)를 포함하며 이 매체상에 컴퓨터 판독 가능 프로그램 코드 수단 또는 로직(1004)을 저장하여 본 발명의 하나 또는 그 이상의 특징들을 제공 및 가능하게 만든다.
- [0169] 컴퓨터 판독 가능 매체상에 구현된 프로그램 코드는 무선, 유선, 광섬유 케이블, RF 등 또는 전술한 것들의 적절한 조합으로 된 것을 포함한(그러나 이에 한정되지는 않는) 적절한 매체를 사용하여 전송될 수 있다.
- [0170] 본 발명의 특징들에 대한 동작들을 실행하기 위한 컴퓨터 프로그램 코드는 JAVA, Smalltalk, C++ 또는 그와 유사 언어 등의 객체 지향 프로그래밍 언어와 "C" 프로그래밍 언어, 어셈블리 언어 또는 그와 유사한 언어 등의 종래의 절차적 프로그래밍 언어들을 포함하여, 하나 또는 그 이상의 프로그래밍 언어들을 조합하여 작성될 수



있다. 상기 프로그램 코드는 전적으로 사용자의 컴퓨터상에서, 부분적으로 사용자의 컴퓨터상에서, 독립형(stand-alone) 소프트웨어 패키지로써, 부분적으로 사용자의 컴퓨터상에서 그리고 부분적으로 원격 컴퓨터상에서 또는 전적으로 원격 컴퓨터나 서버상에서 실행될 수 있다. 위에서 마지막의 경우에, 원격 컴퓨터는 근거리 통신망(LAN) 또는 광역 통신망(WAN)을 포함한 모든 종류의 네트워크를 통해서 사용자의 컴퓨터에 접속될 수 있고, 또는 이 접속은 (예를 들어, 인터넷 서비스 제공자를 이용한 인터넷을 통해서) 외부 컴퓨터에 이루어질 수도 있다.

[0171] 여기에서는 방법들, 장치들(시스템들) 및 컴퓨터 프로그램 제품들의 순서 예시도들 및/또는 블록도들을 참조하여 본 발명의 특징들을 기술한다. 순서 예시도들 및/또는 블록도들의 각 블록과 순서 예시도들 및/또는 블록도들 내 블록들의 조합들은 컴퓨터 프로그램 명령들에 의해 구현될 수 있다는 것을 이해할 수 있을 것이다. 이 컴퓨터 프로그램 명령들은 범용 컴퓨터, 특수목적용 컴퓨터, 또는 기타 프로그램가능 데이터 처리 장치의 프로세서에 제공되어 머신(machine)을 생성하고, 그렇게 하여 그 명령들이 상기 컴퓨터 또는 기타 프로그램가능 데이터 처리 장치의 프로세서를 통해서 실행되어, 상기 순서도 및/또는 블록도의 블록 또는 블록들에 명시된 기능들/동작들을 구현하기 위한 수단을 생성할 수 있다.

[0172] 상기 컴퓨터 프로그램 명령들은 또한 컴퓨터 판독 가능 매체에 저장될 수 있으며, 컴퓨터, 기타 프로그램가능 데이터 처리 장치 또는 다른 디바이스들에 지시하여 상기 컴퓨터 판독 가능 매체에 저장된 명령들이 상기 순서도 및/또는 블록도의 블록 또는 블록들에 명시된 기능/동작들을 구현하는 명령들을 포함하는 제조품(an article of manufacture)을 생성하도록 특정한 방식으로 기능하게 할 수 있다.

[0173] 상기 컴퓨터 프로그램 명령들은 또한 컴퓨터, 기타 프로그램가능 데이터 처리 장치, 또는 다른 디바이스들에 로드되어, 컴퓨터, 기타 프로그램가능 장치 또는 다른 디바이스들에서 일련의 동작 단계들이 수행되게 하여 컴퓨터 구현 프로세스를 생성하며, 그렇게 하여 상기 컴퓨터 또는 기타 프로그램가능 장치상에서 실행되는 명령들이 순서도 및/또는 블록도의 블록 또는 블록들에 명시된 기능들/동작들을 구현하기 위한 프로세스들을 제공할 수 있다.

[0174] 도면들 내 순서도 및 블록도들은 여러 실시 예들에 따른 시스템들, 방법들 및 컴퓨터 프로그램 제품들의 가능한 구현들의 아키텍처, 기능(functionality), 및 연산(operation)을 예시한다. 이와 관련하여, 상기 순서도 또는 블록도들 내 각 블록은 상기 명시된 논리적 기능(들)을 구현하기 위한 하나 또는 그 이상의 실행 가능한 명령들을 포함한 모듈, 세그먼트 또는 코드의 일부분을 나타낼 수 있다. 일부 다른 구현들에서, 상기 블록에 언급되는 기능들은 도면들에 언급된 순서와 다르게 일어날 수도 있다는 것에 또한 유의해야 한다. 예를 들면, 연속으로 도시된 두 개의 블록들은 실제로는 사실상 동시에 실행될 수도 있고, 또는 이 두 블록들은 때때로 관련된 기능에 따라서는 역순으로 실행될 수도 있다. 블록도들 및/또는 순서 예시도의 각 블록, 및 블록도들 및/또는 순서 예시도 내 블록들의 조합들은 특수목적용 하드웨어 및 컴퓨터 명령들의 명시된 기능들 또는 동작들, 또는 이들의 조합들을 수행하는 특수목적용 하드웨어-기반 시스템들에 의해 구현될 수 있다는 것에 또한 유의한다.

[0175] 전술한 것에 추가하여, 하나 또는 그 이상의 특징들은 컴퓨터 환경의 관리를 서비스하는 서비스 제공자에 의해 제공, 공급, 배치, 관리, 서비스 등이 될 수 있다. 예를 들면, 서비스 제공자는 하나 또는 그 이상의 고객들을 위해 하나 또는 그 이상의 특징들을 수행하는 컴퓨터 코드 및/또는 컴퓨터 인프라스트럭처의 제작, 유지, 지원 등을 할 수 있다. 그 대가로, 서비스 제공자는 가입제(subscription) 및/또는 수수료 약정에 따라 고객으로부터 대금을 수령할 수 있으며, 이는 예이다. 또한, 서비스 제공자는 하나 또는 그 이상의 제3자들에게 광고 콘텐츠를 판매하고 대금을 수령할 수 있다.

[0176] 한 특징으로, 하나 또는 그 이상의 특징들 수행하기 위한 애플리케이션이 배치될 수 있다. 한 예로서, 애플리케이션의 배치는 하나 또는 그 이상의 특징들을 수행하는 데 실시 가능한 컴퓨터 인프라스트럭처를 제공하는 것을 포함한다.

[0177] 추가 특징으로서, 컴퓨터 판독 가능 코드를 컴퓨팅 시스템으로 통합하는 것을 포함하는 컴퓨팅 인프라스트럭처가 배치될 수 있으며, 그 컴퓨팅 시스템에서 상기 코드는 상기 컴퓨팅 시스템과 결합하여 하나 또는 그 이상의 특징들을 수행하는 것이 가능하다.

[0178] 추가 특징으로서, 컴퓨터 판독 가능 코드를 컴퓨터 시스템으로 통합시키는 것을 포함하는 컴퓨팅 인프라스트럭처 통합을 위한 프로세스가 제공될 수 있다. 상기 컴퓨터 시스템은 컴퓨터 판독 가능 매체를 포함하고, 상기 컴퓨터 시스템에서 상기 컴퓨터 매체는 하나 또는 그 이상의 특징들을 포함한다. 상기 코드는 상기 컴퓨터 시스템과 결합하여 하나 또는 그 이상의 특징들을 수행하는 것이 가능하다.

- [0179] 위에서 여러 실시 예들이 기술되었지만, 이들은 단지 예시일 뿐이다. 예를 들면, 다른 아키텍처들로 된 컴퓨팅 환경들이 하나 또는 그 이상의 특징들을 포함하고 사용할 수 있다. 또한, 다른 사이즈의 벡터들이 사용될 수도 있으며, 본 발명의 하나 또는 그 이상의 특징들에서 벗어나지 않고 상기 명령에 대한 변경들이 이루어질 수 있다. 또한, 벡터 레지스터 이외의 레지스터들도 사용될 수 있다. 또한, 다른 실시 예들에서, 벡터 오퍼랜드는 벡터 레지스터 대신에 메모리 위치일 수 있다. 다른 변형 예들도 또한 가능하다.
- [0180] 또한, 다른 종류의 컴퓨팅 환경들도 하나 또는 그 이상의 특징들로부터 이득을 얻을 수 있다. 예로서, 프로그램 코드를 저장 및/또는 실행하기에 적합한 데이터 처리 시스템이 사용될 수 있으며, 이 시스템은 시스템 버스를 통해서 메모리 엘리먼트들에 직접적으로 또는 간접적으로 결합된 적어도 두 개의 프로세서를 포함한다. 상기 메모리 엘리먼트들은, 예를 들어 프로그램 코드의 실제 실행 동안 사용되는 로컬 메모리, 대용량 스토리지(bulk storage), 및 코드가 실행 동안에 대용량 스토리지로부터 검색되어야 하는 횟수를 줄이기 위해 적어도 일부 프로그램 코드의 임시 저장(temporary storage)을 제공하는 캐시 메모리를 포함한다.
- [0181] 입력/출력 또는 I/O 디바이스들(키보드, 디스플레이, 포인팅 디바이스, DASD, 테이프, CD, DVD, 썸 드라이브 및 기타 메모리 매체 등을 포함하나 이에 한정되지는 않음)은 직접 또는 중개(intervening) I/O 컨트롤러들을 통해서 상기 시스템에 결합될 수 있다. 네트워크 어댑터 또한 상기 시스템에 결합되어 상기 데이터 처리 시스템이 중개하는 사실 또는 공공 네트워크를 통해서 기타 데이터 처리 시스템 또는 원격 포인터 또는 스토리지 디바이스에 결합되는 것을 가능하게 한다. 모뎀, 케이블 모뎀, 및 이더넷 카드는 이용 가능한 네트워크 어댑터의 단지 일부 예이다.
- [0182] 도 11을 참조하면, 하나 또는 그 이상의 특징들을 구현하기 위한 호스트 컴퓨터 시스템(5000)의 대표적인 컴포넌트들이 도시된다. 대표적인 호스트 컴퓨터(5000)는 컴퓨터 메모리(즉, 중앙 스토리지)(5002)와 통신하는 하나 또는 그 이상의 CPU들(5001)을 포함하고, 또한 스토리지 매체 디바이스들(5011)로 그리고 다른 컴퓨터들 또는 SAN들 등과 통신하기 위한 네트워크들(5010)로 가는 I/O 인터페이스들을 포함한다. CPU(5001)는 아키텍처화된 명령 세트((architected instruction set)와 아키텍처화된 기능(architected functionality)을 갖는 아키텍처에 부합한다. CPU(5001)는 프로그램 주소들(가상 주소들)을 메모리의 실제 주소들로 변환하기 위한 동적 주소 변환(DAT)(5003)을 가질 수 있다. DAT는 통상적으로 컴퓨터 메모리(5002)의 블록에 나중에 액세스할 때 주소 변환의 지연이 필요 없도록 변환들을 캐시하기 위한 변환 색인 버퍼(TLB, translation lookaside buffer)(5007)를 포함한다. 통상적으로, 캐시(5009)는 컴퓨터 메모리(5002)와 프로세서(5001) 사이에서 사용된다. 캐시(5009)는 하나 이상의 CPU가 이용 가능한 큰 캐시(large cache)와 그 큰 캐시와 각 CPU 사이에 있는 더 작고 더 빠른 (더 하위 레벨) 캐시들을 갖는 계층형(hierarchical)일 수 있다. 어떤 구현들에서는, 더 하위 레벨(lower level) 캐시들은 명령 페치와 데이터 액세스를 위한 별개의(separate) 하위 레벨 캐시들을 제공하기 위해 분할된다. 한 실시 예에서, 한 명령이 명령 페치 유닛(5004)에 의해 캐시(5009)를 통해서 메모리(5002)로부터 페치된다. 명령은 명령 디코드 유닛(instruction decode unit)(5006)에서 디코드되고 (어떤 실시 예들에서는 다른 명령들과 함께) 명령 실행 유닛 또는 유닛들(5008)로 디스패치된다(dispatched). 통상적으로 몇 가지의 실행 유닛들 (5008)이 채용되며, 예를 들면 산술 실행 유닛(arithmetic execution unit), 부동 소수점 실행 유닛(floating point execution unit) 및 분기 명령 실행 유닛(branch instruction execution unit)이 있다. 명령은 실행 유닛에 의해 실행되고, 명령이 명시한 레지스터들 또는 메모리로부터 필요한 만큼 오퍼랜드들에 액세스한다. 만일 오퍼랜드가 메모리(5002)로부터 액세스(로드 또는 저장)되면, 로드/저장 유닛(load/store unit)(5005)이 통상적으로 실행되는 명령의 제어에 따라 액세스를 처리한다. 명령들은 하드웨어 회로들에서 또는 내부 마이크로코드(펌웨어)에서 또는 이 둘의 조합에 의해서 실행될 수 있다.
- [0183] 전술한 바와 같이, 컴퓨터 시스템은 로컬 (또는 메인) 스토리지에 정보를 포함하고, 또한 주소지정(addressing), 보호(protection), 그리고 참조 및 변경 기록(reference and change recording)을 포함한다. 주소지정의 몇 가지 예로는 주소의 형식(format of addresses), 주소 공간의 개념(concept of address spaces), 주소의 여러 유형(various types of addresses), 및 한 유형의 주소가 또 다른 유형의 주소로 변환되는 방식(manner)이 있다. 메인 스토리지의 일부는 영구적으로 할당된 스토리지 위치들을 포함한다. 메인 스토리지는 시스템에 데이터의 직접 주소지정 가능한 고속 액세스 스토리지(fast-access storage)를 제공한다. 데이터와 프로그램들은 모두 (입력 디바이스들로부터) 메인 스토리지로 로드된 후에 처리될 수 있다.
- [0184] 메인 스토리지는 때때로 캐시라고 불리는 하나 또는 그 이상의 더 작고 더 고속의 액세스 버퍼 스토리지들을 포함한다. 캐시는 통상적으로 CPU 또는 I/O 프로세서와 물리적으로 연관된다. 구별되는(distinct) 스토리지 매체의 물리적 구축과 사용의 영향들은, 수행을 제외하고는, 일반적으로 프로그램에 의해 관찰되지 않는다.

- [0185] 명령들 용과 데이터 오퍼랜드들 용으로 별개 캐시들이 유지될 수 있다. 캐시 내의 정보는 캐시 블록(cache block) 또는 캐시 라인(또는 줄여서 라인)이라 불리는 인테그럴 경계(integral boundary)상의 인접 바이트들에 보존된다. 어떤 모델은 캐시 라인의 사이즈를 바이트로 회신하는 EXTRACT CACHE ATTRIBUTE 명령을 제공할 수 있다. 어떤 모델은 또한 스토리지를 데이터 또는 명령 캐시로의 프리페치(prefetch) 또는 캐시로부터 데이터의 해제를 실현하는 PREFETCH DATA 명령과 PREFETCH DATA RELATIVE LONG 명령을 제공할 수 있다.
- [0186] 스토리지는 비트들의 긴 수평의 열(a long horizontal string of bits)로 보인다. 대부분의 연산들에 있어서, 스토리지에 대한 액세스는 좌측-에서-우측(left-to-right) 순으로 진행된다. 비트들의 문자열(string)은 8비트의 유닛들로 세분된다. 8-비트 단위를 바이트(byte)라 부르고, 이것은 모든 정보 포맷들의 기본적인 빌딩 블록(building block)이다. 스토리지에서 각 바이트 위치는 음이 아닌 고유한 정수로 식별되고, 이것은 그 바이트 위치의 주소, 또는, 간단히 말해서 바이트 주소(byte address)이다. 인접 바이트 위치들은 좌측의 0부터 시작해서 좌측-에서-우측 순으로 진행되는 연속되는 주소들이다. 주소들은 무부호 2진 정수들이며 24, 31, 또는 64비트이다.
- [0187] 정보는 스토리지와 CPU 또는 채널 서브시스템 사이에서, 1 바이트 또는 바이트들의 그룹으로, 한 번에 전송된다. 다르게 명시되지 않으면, 예를 들어, z/Architecture에서 스토리지 내 바이트들의 그룹은 그 그룹의 제일 좌측 바이트에 의해 주소지정된다. 그룹 내 바이트의 수는 수행될 연산에 의해 암시되거나 분명하게 명시된다. CPU 연산에서 사용될 때, 바이트들의 그룹은 필드(field)라 불린다. 각 바이트들의 그룹 내에서, 예를 들어, z/Architecture에서, 비트들은 좌측-에서-우측 순으로 번호가 붙는다. z/Architecture에서, 제일 좌측 비트들은 때때로 "상위(high-order)" 비트들로 불리고 제일 우측 비트들은 "하위(low-order)" 비트들로 불린다. 그러나 비트 번호는 스토리지 주소가 아니다. 바이트만 주소지정될 수 있다. 스토리지 내 한 바이트의 개별 비트들에서 연산하기 위해서는, 전체 바이트가 액세스된다. 한 바이트 내 비트들은 (예를 들어, z/Architecture에서) 0에서 7까지, 좌측에서 우측으로 번호가 붙는다. 한 주소 내 비트들은 24-비트 주소에서는 8~31 또는 40~63으로, 또는 31-비트 주소에서는 1~31 또는 33~63으로 번호가 붙을 수 있고; 64-비트 주소에서는 0~63으로 번호가 붙는다. 다른 고정-길이 포맷의 다수 바이트들 내에서, 그 포맷을 이루는 비트들은 0부터 시작해서 연속적으로 번호가 붙는다. 예러 검출의 목적을 위해서, 그리고 바람직하게는 교정을 위해서, 하나 또는 그 이상의 검사용 비트들이 각 바이트와 또는 바이트들의 그룹과 함께 전송된다. 이러한 검사용 비트들은 머신에 의해 자동적으로 생성되며 프로그램에 의해 직접적으로 제어될 수 없다. 스토리지 용량은 바이트 수로 표시된다. 스토리지-오퍼랜드 필드의 길이가 명령의 연산 코드에 의해 암시될 때, 그 필드는 고정 길이(fixed length)를 가졌다고 말하며, 그 길이는 1, 2, 4, 8, 또는 16 바이트일 수 있다. 어떤 명령들에는 더 큰 필드들이 암시될 수 있다. 스토리지-오퍼랜드 필드의 길이가 암시되지 않고 분명하게 언급될 때, 그 필드는 가변 길이(variable length)를 가졌다고 말한다. 가변-길이 오퍼랜드는 길이가 1 바이트의 증분들 만큼씩 (또는 어떤 명령들에서는, 2 바이트의 배수로 또는 다른 배수들로) 변할 수 있다. 정보가 스토리지에 배치될 때, 비록 스토리지에 대한 물리적 경로의 폭이 저장되는 필드의 길이보다 더 클 수 있을지라도, 단지 그 지정된 필드에 포함된 그 바이트 위치들의 내용들만 대체된다.
- [0188] 정보의 일정 유닛들(units)은 스토리지에서 인테그럴 경계(integral boundary) 상에 있어야 한다. 경계(boundary)는 그 스토리지 주소가 그 유닛의 길이의 바이트 배수일 때 정보의 유닛에 대해서 인테그럴(integral)하다고 불린다. 인테그럴 경계 상의 2, 4, 8, 및 16 바이트의 필드들에는 특별한 명칭들이 주어진다. 하프워드(halfword)는 2-바이트 경계 상의 2개의 연속 바이트들의 그룹이고 명령들의 기본 빌딩 블록이다. 워드(word)는 4-바이트 경계 상의 4개의 연속 바이트들의 그룹이다. 더블워드(doubleword)는 8-바이트 경계 상의 8개의 연속 바이트들의 그룹이다. 쿼드워드(quadword)는 16-바이트 경계 상의 16개의 연속 바이트들의 그룹이다. 스토리지 주소들이 하프워드, 워드, 더블워드, 및 쿼드워드를 지정할 때, 그 주소의 2진 표시는 1개, 2개, 3개, 또는 4개의 제일 우측 제로(zero) 비트들을 각각 포함한다. 명령들은 2-바이트 인테그럴 경계들 상에 있어야 한다. 대부분의 명령들의 스토리지 오퍼랜드들은 경계-정렬(boundary-alignment) 요건들을 갖지 않는다.
- [0189] 명령들과 데이터 오퍼랜드들에 대한 별개의 캐시들을 구현하는 디바이스들상에서, 만일 프로그램이 어떤 캐시 라인에 저장되고 그 캐시 라인으로부터 명령들이 후속적으로 폐치되면, 그 저장이 후속적으로 폐치되는 명령들을 변경하는지 여부와 상관 없이, 상당한 지연을 겪게 될 것이다.
- [0190] 한 실시 예에서, 본 발명은 소프트웨어로 실시될 수 있다(이 소프트웨어는 때때로 라이선스된 내부 코드, 펌웨어, 마이크로-코드, 밀리-코드, 피코-코드 등으로 불리며, 이들 중 어떤 것이든 본 발명의 하나 또는 그 이상의 특징들에 부합할 것이다). 도 11을 참조하면, 하나 또는 그 이상의 특징들을 구현하는 소프트웨어 프로그램 코드는 CD-ROM 드라이브, 테이프 드라이브 또는 하드 드라이브와 같은 장기 스토리지(long-term storage) 매체 디

바이스들(5011)로부터 호스트 시스템(5000)의 프로세서(5001)에 의해 액세스된다. 소프트웨어 프로그램 코드는 디스켓, 하드 드라이브, 또는 CD-ROM과 같은 데이터 처리 시스템에 사용할 용도로 알려진 여러 가지 매체들 중 어느 하나에 구현될 수 있다. 코드는 그러한 매체상에 배포되거나, 또는 한 컴퓨터 시스템의 컴퓨터 메모리(5002) 또는 스토리지의 사용자들로부터 네트워크(5010)를 통해서 다른 컴퓨터 시스템들에, 그러한 다른 시스템들의 사용자에게 의해 사용될 용도로 배포될 수 있다.

[0191] 소프트웨어 프로그램 코드는 여러 가지 컴퓨터 컴포넌트들의 기능과 상호작용(interaction) 및 하나 또는 그 이상의 애플리케이션 프로그램들을 제어하는 운영체제를 포함한다. 프로그램 코드는 보통으로 스토리지 매체 디바이스(5011)로부터 상대적으로 더 고속의 컴퓨터 스토리지(5002)—이것은 프로세서(5001)에 의한 처리에 이용 가능함—로 페이지된다. 메모리 내 소프트웨어 프로그램 코드를 물리적 매체상에 구현하는 기술과 방법, 및/또는 네트워크들을 통해서 소프트웨어 코드를 배포하는 기술과 방법은 잘 알려져 있으며 여기에서는 더 논의하지 않을 것이다. 프로그램 코드는, 유형의 매체(전자 메모리 모듈들(RAM), 플래시 메모리, 콤팩트 디스크(CDs), DVDs, 자기 테이프 등을 포함하나, 이러한 것들로 한정되지 않음)상에 생성되고 저장될 때, 흔히 "컴퓨터 프로그램 제품"으로 불린다. 컴퓨터 프로그램 제품 매체는 통상적으로 처리 회로에 의해 판독 가능하며, 컴퓨터 시스템에서 처리 회로에 의해 실행하기 위해 판독 가능한 것이 바람직하다.

[0192] 도 12는 하나 또는 그 이상의 특징들이 실시될 수 있는 대표적인 워크스테이션 또는 서버 하드웨어 시스템을 예시한다. 도 12의 시스템(5020)은 선택적인 주변 디바이스들을 포함하여, 개인용 컴퓨터, 워크스테이션 또는 서버 같은 대표적인 베이스 컴퓨터 시스템(5021)을 포함한다. 베이스 컴퓨터 시스템(5021)은 하나 또는 그 이상의 프로세서들(5026)과 버스를 포함하며, 버스는 알려진 기술들에 따라 프로세서(들)(5026)와 시스템(5021)의 다른 컴포넌트들 사이를 연결하여 통신을 가능하게 하기 위해 채용되는 것이다. 버스는 프로세서(5026)를 메모리(5025)와 장기 스토리지(5027)에 연결하며 장기 스토리지는, 예를 들어, 하드 드라이브(예를 들어, 자기 매체, CD, DVD 및 플래시 메모리를 포함함) 또는 테이프 드라이브를 포함할 수 있다. 시스템(5021)은 또한 사용자 인터페이스 어댑터를 포함할 수 있으며, 이 사용자 인터페이스 어댑터는 마이크로프로세서(5026)를 버스를 통해서 키보드(5024), 마우스(5023), 프린터/스캐너(5030) 및/또는 기타 인터페이스 디바이스들과 같은 하나 또는 그 이상의 인터페이스 디바이스들에 연결하며, 상기 기타 인터페이스 디바이스들은 터치 감응식 스크린(touch sensitive screen), 디지털 입력 패드(digitized entry pad) 등과 같은 사용자 인터페이스 디바이스일 수 있다. 버스는 또한 LCD 스크린 또는 모니터와 같은 디스플레이 디바이스(5022)를 디스플레이 어댑터를 통해서 마이크로프로세서(5026)에 연결한다.

[0193] 시스템(5021)은 네트워크(5029)와 통신(5028)이 가능한 네트워크 어댑터를 경유하여 다른 컴퓨터들 또는 컴퓨터들의 네트워크들과 통신할 수 있다. 네트워크 어댑터들의 예로는 통신 채널(communications channels), 토큰 링(token ring), 이더넷(Ethernet) 또는 모뎀(modems)이 있다. 이와는 달리, 시스템(5021)은 CDPD(cellular digital packet data) 카드 같은 무선 인터페이스를 사용하여 통신할 수 있다. 시스템(5021)은 근거리 통신망(LAN) 또는 광역 통신망(WAN)에서 다른 컴퓨터들과 연관될 수 있고, 또는 시스템(5021)은 또 다른 컴퓨터와 클라이언트/서버 배열방식(arrangement)에서 클라이언트가 될 수 있다. 이들 모든 구성들과 적절한 통신 하드웨어 및 소프트웨어는 이 기술분야에서 알려져 있다.

[0194] 도 13은 하나 또는 그 이상의 특징들이 실시될 수 있는 데이터 처리 네트워크(5040)를 예시한다. 데이터 처리 네트워크(5040)는 무선 네트워크와 유선 네트워크 같은 복수의 개별 네트워크들을 포함할 수 있으며, 이들의 각각은 복수의 개별 워크스테이션들(5041, 5042, 5043, 5044)을 포함할 수 있다. 또한, 이 기술분야에서 통상의 지식을 가진 자들은 인식할 수 있는 바와 같이, 하나 또는 그 이상의 LAN들이 포함될 수 있으며, 여기에서 LAN은 호스트 프로세서에 결합된 복수의 지능형(intelligent) 워크스테이션들을 포함할 수 있다.

[0195] 계속해서 도 13을 참조하면, 네트워크들은 또한 게이트웨이 컴퓨터 (클라이언트 서버 5046) 또는 애플리케이션 서버(데이터 저장소를 액세스할 수 있고 또한 워크스테이션 5045로부터 직접 액세스될 수 있는 원격 서버 5048)와 같은 메인프레임 컴퓨터들 또는 서버들을 포함할 수 있다. 게이트웨이 컴퓨터(5046)는 각 개별 네트워크로의 진입점(a point of entry) 역할을 한다. 게이트웨이는 하나의 네트워킹 프로토콜을 또 하나의 네트워킹 프로토콜에 연결할 때 필요하다. 게이트웨이(5046)는 바람직하게는 통신 링크를 통해 또 하나의 네트워크(예를 들면 인터넷 5047)에 결합될 수 있다. 게이트웨이(5046)는 또한 통신 링크를 사용하여 하나 또는 그 이상의 워크스테이션들(5041, 5042, 5043, 5044)에 직접 결합될 수 있다. 게이트웨이 컴퓨터는 인터넷내셔널 비즈니스 머신즈 코포레이션에서 입수 가능한 IBM eServer System z 서버를 활용하여 구현될 수 있다.

[0196] 도 12와 도 13을 동시에 참조하면, 본 발명의 하나 또는 그 이상의 특징들을 구현할 수 있는 소프트웨어 프로그램



래밍 코드가 시스템(5020)의 프로세서(5026)에 의해 CD-ROM 드라이브 또는 하드 드라이브와 같은 장기 스토리지 매체(5027)로부터 액세스될 수 있다. 소프트웨어 프로그래밍 코드는 디스켓, 하드 드라이브, 또는 CD-ROM과 같은 데이터 처리 시스템과 함께 사용할 용도로 알려진 여러 가지 매체들 중 어느 하나에 구현될 수 있다. 코드는 그러한 매체상에 배포되거나, 또는 한 컴퓨터 시스템의 메모리 또는 스토리지의 사용자들(5050, 5051)로부터 네트워크를 통해서 다른 컴퓨터 시스템들에, 그러한 다른 시스템들의 사용자에게 의해 사용될 용도로 배포될 수 있다.

[0197] 이와는 달리, 프로그래밍 코드는 메모리(5025)에 구현되고, 프로세서 버스를 사용하여 프로세서(5026)에 의해 액세스될 수 있다. 이러한 프로그래밍 코드는 여러 가지 컴퓨터 컴포넌트들의 기능과 상호작용 및 하나 또는 그 이상의 애플리케이션 프로그램들(5032)을 제어하는 운영체제를 포함한다. 프로그램 코드는 보통으로 스토리지 매체(5027)로부터 고속의 메모리(5025)-이것은 프로세서(5026)에 의한 처리에 이용 가능함-로 페이지된다. 메모리 내 소프트웨어 프로그래밍 코드를 물리적 매체상에 구현하는 기술과 방법, 및/또는 네트워크들을 통해서 소프트웨어 코드를 배포하는 기술과 방법은 잘 알려져 있으며 여기에서는 더 논의하지 않을 것이다. 프로그램 코드는, 유형의 매체(전자 메모리 모듈들(RAM), 플래시 메모리, 콤팩트 디스크(CDs), DVDs, 자기 테이프 등을 포함하나, 이러한 것들로 한정되지 않음)상에 생성되고 저장될 때, 흔히 "컴퓨터 프로그램 제품"으로 불린다. 컴퓨터 프로그램 제품 매체는 통상적으로 처리 회로에 의해 판독 가능하며, 컴퓨터 시스템에서 처리 회로에 의해 실행하기 위해 판독 가능한 것이 바람직하다.

[0198] 프로세서가 가장 쉽게 이용 가능한 캐시(보통으로 프로세서의 다른 캐시들보다 더 빠르고 더 작음)는 가장 낮은(L1 또는 레벨 1) 캐시이고 메인 저장소(메인 메모리)는 가장 높은 레벨의 캐시(만일 3개의 레벨이 있다면 L3)이다. 가장 낮은 레벨의 캐시는 흔히 실행될 기계어 명령들을 보유하는 명령 캐시(I-캐시)와 데이터 오퍼랜드들을 보유하는 데이터 캐시(D-캐시)로 나뉜다.

[0199] 도 14를 참조하면, 예시적인 프로세서 실시 예가 프로세서(5026)에 대해 도시된다. 프로세서 성능을 향상시키기 위해서 메모리 블록들을 버퍼하기 위해 통상적으로 하나 또는 그 이상의 캐시(5053) 레벨들이 채용된다. 캐시(5053)는 사용될 가능성이 있는 메모리 데이터의 캐시 라인들을 보유하는 고속 버퍼이다. 통상적인 캐시 라인들은 64, 128 또는 256 바이트의 메모리 데이터이다. 별개의 캐시들은 흔히 데이터를 캐시하기 위해서보다는 명령들을 캐시하기 위해 채용된다. 이 기술분야에서 잘 알려진 "스누프(snoop)" 알고리즘들에 의해 캐시 일관성(cache coherence)(메모리 내 라인들의 사본들과 캐시들의 동기화(synchronization))이 종종 제공된다. 프로세서 시스템의 메인 메모리 스토리지(5025)는 종종 캐시로 불린다. 4개 레벨의 캐시(5053)를 가진 프로세서 시스템에서, 메인 스토리지(5025)는 때로 레벨 5(L5) 캐시로 불리는데, 왜냐하면 그것은 통상적으로 더 빠르며 컴퓨터 시스템이 이용 가능한 비휘발성 스토리지(DASD, 테이프 등)의 일부분만을 보유하기 때문이다. 메인 스토리지(5025)는 운영체제에 의해 메인 스토리지(5025)의 안팎으로(in and out of) 페이지되는 데이터의 페이지들을 "캐시"한다.

[0200] 프로그램 카운터(명령 카운터)(5061)는 실행될 현재 명령의 주소를 추적한다. z/Architecture 프로세서 내 프로그램 카운터는 64비트이고 이전의 주소지정 한계(addressing limits)를 지원하기 위해 31비트 또는 24비트로 잘려질 수 있다. 프로그램 카운터는 통상적으로 컴퓨터의 PSW(프로그램 상태 워드)에 구현되어, 그것이 컨텍스트 전환(context switching) 동안 지속되도록 한다. 그리하여, 프로그램 카운터 값을 갖는 진행중인 프로그램은, 예를 들어, 운영체제에 의해 인터럽트될 수 있다(프로그램 환경에서 운영체제 환경으로 컨텍스트 전환). 프로그램이 활성이 아닐 때, 프로그램의 PSW는 프로그램 카운터 값을 유지하고, 운영체제가 실행 중일 때 운영체제의(PSW 내) 프로그램 카운터가 사용된다. 통상적으로, 프로그램 카운터는 현재 명령의 바이트 수와 동일한 양으로 증분된다. 감소된 명령 세트 컴퓨팅(Reduced Instruction Set Computing, RISC) 명령들은 통상적으로 고정 길이이고, 한편 콤플렉스 명령 세트 컴퓨팅(Complex Instruction Set Computing, CISC) 명령들은 통상적으로 가변 길이이다. IBM z/Architecture의 명령들은 2, 4 또는 6 바이트의 길이를 갖는 CISC 명령들이다. 프로그램 카운터(5061)는, 예를 들어, 분기 명령의 분기 채택 연산(branch taken operation) 또는 컨텍스트 전환 연산에 의해 변경된다. 컨텍스트 전환 연산에서, 현재의 프로그램 카운터 값은 실행되고 있는 프로그램에 관한 상태 정보(예를 들어, 조건 코드들과 같은 것)와 함께 프로그램 상태 워드에 세이브되고(saved), 실행될 새로운 프로그램 모듈의 명령을 가리키는 새로운 프로그램 카운터 값이 로드된다. 프로그램 카운터(5061) 내에 분기 명령의 결과를 로딩함으로써 프로그램이 결정을 내리거나 그 프로그램 내에서 루프를 돌도록 허용하기 위해, 분기 채택 연산(branch taken operation)이 수행된다.

[0201] 통상적으로 프로세서(5026)를 대신하여 명령들을 폐치하기 위해 명령 폐치 유닛(5055)이 채용된다. 폐치 유닛은 "다음 순차의 명령들"이나, 분기 채택 명령들의 타겟 명령들, 또는 컨텍스트 전환에 뒤이은 프로그램의 첫 번째



명령들을 폐치한다. 현대 명령(Modern Instruction) 폐치 유닛은 프리페치된(prefetched) 명령들이 사용될 수 있는 가능성에 기초하여 추론적으로 명령들을 프리페치하는 프리페치 기술들을 종종 채용한다. 예를 들어, 폐치 유닛은 16 바이트의 명령—이는 그 다음 순차 명령 및 그 이후 순차 명령들의 추가 바이트들을 포함함—을 폐치할 수 있다.

[0202] 그런 다음, 폐치된 명령들이 프로세서(5026)에 의해 실행된다. 한 실시 예에서, 폐치된 명령(들)은 폐치 유닛의 디스패치 유닛(5056)으로 보내진다. 디스패치 유닛이 그 명령(들)을 디코드하고, 디코드된 명령(들)에 관한 정보를 적절한 유닛들(5057, 5058, 5060)로 전달한다. 실행 유닛(5057)이 통상적으로 명령 폐치 유닛(5055)으로부터 디코드된 산술 명령들(arithmetic instructions)에 관한 정보를 수신할 것이고, 그 명령의 오퍼코드(opcode)에 따라 오퍼랜드들에 대한 산술 연산들(arithmetic operations)을 수행할 것이다. 오퍼랜드들이 바람직하게는, 메모리(5025), 아키텍처화된 레지스터들(5059)로부터 또는 실행되고 있는 명령의 즉시 필드(immediate field)로부터 실행 유닛(5057)에 제공된다. 저장될 때, 실행의 결과들이 메모리(5025)나, 레지스터들(5059)에 또는 다른 머신 하드웨어(예를 들어, 제어 레지스터들, PSW 레지스터들 및 그와 유사한 것)에 저장된다.

[0203] 통상적으로 프로세서(5026)는 명령의 기능을 실행하기 위한 하나 또는 그 이상의 유닛들(5057, 5058, 5060)을 갖는다. 도 15a를 참조하면, 실행 유닛(5057)은 인터페이싱 로직(5071)을 거쳐서 아키텍처화된 범용 레지스터들(5059), 디코드/디스패치 유닛(5056), 로드 저장 유닛(5060), 및 기타(5065) 프로세서 유닛들과 통신할 수 있다. 실행 유닛(5057)은, 산술 논리 유닛(arithmetic logic unit, ALU)(5066)이 연산할 정보를 보유하기 위해 몇몇의 레지스터 회로들(5067, 5068, 5069)을 채용할 수 있다. ALU는 논리곱(AND), 논리합(OR) 및 배타논리합(XOR), 로테이트(rotate) 및 시프트(shift)와 같은 논리 함수뿐만 아니라 더하기, 빼기, 곱하기 및 나누기와 같은 산술 연산들도 수행한다. 바람직하게는, ALU는 설계에 종속적인 특수 연산들을 지원한다. 다른 회로들은, 예를 들어, 조건 코드들 및 복구 지원 로직을 포함하는 다른 아키텍처화된 퍼실리티들(5072)을 제공할 수 있다. 통상적으로, ALU 동작의 결과는 출력 레지스터 회로(5070)에 보유(hold)되고, 이 출력 레지스터 회로(5070)는 여러 가지 다른 처리 기능들에 그 결과를 전달할 수 있다. 프로세서 유닛들의 배열방식(arrangements)은 다양하며, 본 설명은 본 발명의 한 실시 예에 관한 대표적인 이해를 제공하려는 의도일 뿐이다.

[0204] 예를 들어, ADD 명령은 산술 및 논리 기능을 갖는 실행 유닛(5057)에서 실행될 것이고, 한편 예를 들어 부동 소수점 명령은 특수한 부동 소수점 능력을 갖는 부동 소수점 실행에서 실행될 것이다. 바람직하게는, 실행 유닛은 오퍼랜드들에 관한 오퍼코드 정의 기능(opcode defined function)을 수행함으로써 명령에 의해 식별된 오퍼랜드들에 관해 연산한다. 예를 들어, ADD 명령은 그 명령의 레지스터 필드들에 의해 식별되는 두 개의 레지스터들(5059)에서 발견되는 오퍼랜드들에 관해 실행 유닛(5057)에 의해 실행될 수 있다.

[0205] 실행 유닛(5057)은 두 개의 오퍼랜드들에 관해 산술 덧셈(arithmetic addition)을 수행하고 그 결과를 제3 오퍼랜드에 저장하며, 여기서, 제3 오퍼랜드는 제3 레지스터 또는 두 개의 소스 레지스터들 중 하나일 수 있다. 바람직하게는, 실행 유닛은 산술 논리 유닛(ALU)(5066)을 이용하며 이 ALU(5066)는 더하기, 빼기, 곱하기, 나누기 중 어느 것이든지 포함하는 여러 가지 대수 함수들(algebraic functions) 뿐만 아니라 시프트(Shift), 로테이트(Rotate), 논리곱(And), 논리합(Or) 및 배타논리합(XOR)과 같은 여러 가지 논리 함수들을 수행할 수 있다. 일부 ALU들(5066)은 스칼라 연산들을 위해 설계되며 일부는 부동 소수점을 위해 설계된다. 데이터는 아키텍처에 따라 빅 엔디언(Big Endian)(여기서 최하위 바이트(least significant byte)는 가장 높은 바이트 주소에 있음) 또는 리틀 엔디언(Little Endian)(여기서 최하위 바이트는 가장 낮은 바이트 주소에 있음)일 수 있다. IBM z/Architecture는 빅 엔디언이다. 부호화된 필드들(signed fields)은 아키텍처에 따라, 부호(sign) 및 크기(magnitude), 1의 보수 또는 2의 보수일 수 있다. 2의 보수에서 음의 값 또는 양의 값은 단지 ALU 내에서 덧셈만을 필요로 하므로, ALU가 뺄셈 능력을 설계할 필요가 없다는 점에서 2의 보수가 유리하다. 숫자들은 일반적으로 속기(shorthand)로 기술되는데, 12비트 필드는 예를 들어, 4,096바이트 블록의 주소를 정의하고 일반적으로 4 Kbyte(Kilobyte) 블록으로 기술된다.

[0206] 도 15b를 참조하면, 분기 명령을 실행하기 위한 분기 명령 정보는 통상적으로 분기 유닛(5058)으로 보내지는데, 이 분기 유닛(5058)은 다른 조건부 연산들(conditional operations)이 완료되기 전에 그 분기의 결과를 예측하도록 분기 이력 테이블(5082)과 같은 분기 예측 알고리즘을 흔히 채용한다. 현재 분기 명령의 타겟은, 그 조건부 연산들이 완료되기 전에 폐치되고 추론적으로 실행될 것이다. 조건부 연산들이 완료될 때, 추론적으로 실행된 분기 명령들은 조건부 연산 및 추론된 결과의 조건들에 기초하여 완료되거나 폐기된다. 통상적인 분기 명령은, 만일 그 조건 코드들이 분기 명령의 분기 요건을 충족한다면, 조건 코드들을 테스트하고 타겟 주소로 분기할 수 있고, 타겟 주소는, 예를 들어, 레지스터 필드들 또는 그 명령의 즉시 필드에서 발견되는 수들을 포함하는 몇 개의 수들에 기초하여 계산될 수 있다. 분기 유닛(5058)은 복수의 입력 레지스터 회로들(5075, 5075,

5077) 및 출력 레지스터 회로(5080)를 갖는 ALU(5074)를 채용할 수 있다. 분기 유닛(5058)은, 예를 들어, 범용 레지스터들(5059), 디코드 디스패치 유닛(5056) 또는 기타 회로들(5073)과 통신할 수 있다.

[0207] 명령들의 그룹의 실행은 여러 가지 이유들로 인터럽트될 수 있는데, 이러한 이유들에는, 예를 들어, 운영체제에 의해 개시되는 컨텍스트 전환, 컨텍스트 전환을 초래하는 프로그램 예외 또는 에러, 컨텍스트 전환 또는 (멀티-스레드 환경에서) 복수의 프로그램들의 멀티-스레딩 활동을 초래하는 I/O 인터럽션 신호가 포함된다. 바람직하게는 컨텍스트 전환 액션은 현재 실행중인 프로그램에 관한 상태 정보(state information)를 세이브하고, 그런 다음 호출되는 또 다른 프로그램에 관한 상태 정보를 로드한다. 상태 정보는, 예를 들어, 하드웨어 레지스터들 또는 메모리에 저장될 수 있다. 바람직하게는, 상태 정보는 실행될 다음 명령을 가리키는 프로그램 카운터 값, 조건 코드들, 메모리 변환 정보 및 아키텍처화된 레지스터 콘텐츠를 포함한다. 컨텍스트 전환 활동은, 하드웨어 회로들, 애플리케이션 프로그램들, 운영체제 프로그램들 또는 펌웨어 코드(마이크로코드, 피코-코드 또는 라이선스된 내부 코드(LIC)) 단독으로 또는 이것들의 조합으로 실행될 수 있다.

[0208] 프로세서는 명령 정의 방법들(instruction defined methods)에 따라 오퍼랜드들에 액세스한다. 명령은 명령의 일부분의 값을 사용하는 즉시 오퍼랜드(immediate operand)를 제공할 수 있고, 범용 레지스터들 또는 특수 목적용 레지스터들(예를 들어, 부동 소수점 레지스터들)을 분명하게 가리키는 하나 또는 그 이상의 레지스터 필드들을 제공할 수 있다. 명령은 오퍼코드 필드에 의해 오퍼랜드들로서 식별되는 암시 레지스터들(implied registers)을 이용할 수 있다. 명령은 오퍼랜드들에 대한 메모리 위치들을 이용할 수 있다. 오퍼랜드의 메모리 위치는 레지스터, 즉시 필드(immediate field), 또는 레지스터들과 즉시 필드의 조합에 의해 제공될 수 있고, 이는 z/Architecture 장 변위(long displacement) 퍼실리티가 전형적인 예이며, 여기서 명령은 기준 레지스터, 인덱스 레지스터 및 즉시 필드(변위 필드)-이것들은 예를 들어 메모리에서 오퍼랜드의 주소를 제공하기 위해 함께 더해짐-를 정의한다. 만일 다르게 표시되지 않는다면, 여기서의 위치는 통상적으로 메인 메모리(메인 스토리지) 내 위치를 암시한다.

[0209] 도 15c를 참조하면, 프로세서는 로드/저장 유닛(5060)을 사용하여 스토리지에 액세스한다. 로드/저장 유닛(5060)은 메모리(5053)에서 타겟 오퍼랜드의 주소를 획득하고 레지스터(5059) 또는 또 다른 메모리(5053) 위치에 오퍼랜드를 로딩함으로써 로드 연산을 수행할 수 있고, 또는 메모리(5053)에서 타겟 오퍼랜드의 주소를 획득하고 레지스터(5059) 또는 또 다른 메모리(5053) 위치로부터 획득된 데이터를 메모리(5053) 내 타겟 오퍼랜드 위치에 저장함으로써 저장 연산을 수행할 수 있다. 로드/저장 유닛(5060)은 추론적(speculative)일 수 있고, 명령 순서에 비해 순서가 다른(out-of-order) 순서로 메모리에 액세스할 수 있지만, 로드/저장 유닛(5060)은 명령들이 순서대로 실행된 것으로 프로그램들에 대한 외관(appearance)을 유지할 것이다. 로드/저장 유닛(5060)은 범용 레지스터들(5059), 디코드/디스패치 유닛(5056), 캐시/메모리 인터페이스(5053) 또는 기타 엘리먼트들(5083)과 통신할 수 있고, 스토리지 주소들을 계산하기 위해 그리고 순서대로 연산들을 유지하기 위한 파이프라인 시퀀싱을 제공하기 위해 여러 가지 레지스터 회로들, ALU들(5085) 및 제어 로직(5090)을 포함한다. 일부 연산들은 순서가 바뀔 수 있으나, 이 기술분야에서 잘 알려진 바와 같이, 로드/저장 유닛은, 순서가 바뀐 연산들이 그 프로그램에 순서대로 수행된 것처럼 나타나도록 하는 기능을 제공한다.

[0210] 바람직하게는, 애플리케이션 프로그램이 "보는(sees)" 주소들은 흔히 가상 주소들로 불린다. 가상 주소들은 때로는 "논리적 주소들(logical addresses)" 및 "유효 주소들(effective addresses)"로 불린다. 이들 가상 주소들은 여러 가지 동적 주소 변환(DAT) 기술들 중 하나에 의해 물리적 메모리 위치로 다시 보내진다는 점에서 가상이고, 상기 여러 가지 동적 주소 변환(DAT) 기술들에는, 단순히 오프셋 값으로 가상 주소를 프리픽싱(prefixing)하는 것, 하나 또는 그 이상의 변환 테이블들을 통해 가상 주소를 변환하는 것이 포함될 수 있으나, 이러한 것들로 한정되는 것은 아니며, 바람직하게는, 변환 테이블들은 적어도 세그먼트 테이블 및 페이지 테이블만을 또는 이것들의 조합을 포함하며, 바람직하게는, 세그먼트 테이블은 페이지 테이블을 가리키는 엔트리를 갖는다. z/Architecture에서는, 변환의 계층(hierarchy of translation)이 제공되는데, 이 변환의 계층에는 영역 제1 테이블, 영역 제2 테이블, 영역 제3 테이블, 세그먼트 테이블 및 선택적인 페이지 테이블이 포함된다. 주소 변환의 수행은 흔히 변환 색인 버퍼(TLB)를 이용하여 향상되는데, 이 변환 색인 버퍼는 연관된 물리적 메모리 위치에 가상 주소를 매핑하는 엔트리들을 포함한다. DAT가 변환 테이블들을 사용하여 가상 주소를 변환할 때, 엔트리들이 생성된다. 그런 다음, 후속적으로 가상 주소를 사용할 때 느린 연속적인 변환 테이블 액세스들보다 오히려 빠른 TLB의 엔트리를 이용할 수 있다. TLB 콘텐츠는 LRU(Least Recently used)를 포함하는 여러 가지 대체 알고리즘들에 의해 관리될 수 있다.

[0211] 프로세서가 멀티-프로세서 시스템의 프로세서인 경우, 각각의 프로세서는 I/O, 캐시들, TLB들 및 메모리와 같은 공유 리소스들(shared resources)을 일관성(coherency)을 위해 인터로크(interlock)를 유지하는 역할을 한다.

통상적으로, "스누프(snoop)" 기술들이 캐시 일관성을 유지하는 데 이용될 것이다. 스누프 환경에서, 각각의 캐시 라인은 공유를 용이하게 하기 위해, 공유 상태(shared state), 독점 상태(exclusive state), 변경된 상태(changed state), 무효 상태(invalid state) 중 어느 하나에 있는 것으로 표시될 수 있다.

[0212] I/O 유닛들(5054, 도 14)은 프로세서에 주변기기에 연결하기 위한 수단을 제공하는데, 예를 들어, 그 주변기에는 테이프, 디스크, 프린터, 디스플레이, 및 네트워크가 포함된다. I/O 유닛들은 흔히 소프트웨어 드라이버들에 의해 컴퓨터 프로그램에 제공된다. IBM®의 System z 같은 메인프레임들에서, 채널 어댑터들 및 오픈 시스템 어댑터들은 운영체제와 주변 디바이스들 사이의 통신을 가능하게 하는, 메인프레임의 I/O 유닛들이다.

[0213] 또한, 다른 종류의 컴퓨팅 환경들도 하나 또는 그 이상의 특징들로부터 이득을 얻을 수 있다. 한 예로, 환경(environment)은 에뮬레이터(예, 소프트웨어 또는 다른 에뮬레이션 메커니즘들)를 포함할 수 있으며, 이 에뮬레이터에서 특정 아키텍처(예를 들어, 명령 실행, 주소 변환과 같은 아키텍처화된 함수들, 및 아키텍처화된 레지스터들을 포함함) 또는 그것의 서브세트(subset)가 (예를 들어, 프로세서 및 메모리를 갖는 네이티브 컴퓨터 시스템 상에서) 에뮬레이트된다. 이러한 환경에서, 비록 그 에뮬레이터를 실행하는 컴퓨터가 에뮬레이트되고 있는 능력들과는 다른 아키텍처를 가질 수 있지만, 에뮬레이터의 하나 또는 그 이상의 에뮬레이션 기능들은 본 발명의 하나 또는 그 이상의 실시 예들을 구현할 수 있다. 한 예로서, 에뮬레이션 모드에서, 에뮬레이트되고 있는 특정 명령 또는 연산은 디코드되고, 적절한 에뮬레이션 기능이 개별 명령 또는 연산을 구현하도록 만들어진다.

[0214] 에뮬레이션 환경에서, 호스트 컴퓨터는, 예를 들어, 명령들 및 데이터를 저장하는 메모리, 메모리로부터 명령들을 폐치하고 또한 선택적으로 그 폐치된 명령을 위한 로컬 버퍼링을 제공하는 명령 폐치 유닛, 폐치된 명령들을 수신하고 폐치된 명령들의 유형을 결정하는 명령 디코드 유닛, 및 명령들을 실행하는 명령 실행 유닛을 포함한다. 실행은 메모리로부터 레지스터 내에 데이터를 로딩하는 것; 레지스터로부터 메모리로 다시 데이터를 저장하는 것; 또는 디코드 유닛에 의해 결정된 바와 같이, 산술 또는 논리 연산의 몇몇 유형을 수행하는 것을 포함할 수 있다. 한 예에서, 각각의 유닛은 소프트웨어에서 구현된다. 예를 들어, 그 유닛들에 의해 수행되고 있는 연산들은 에뮬레이터 소프트웨어 내에서 하나 또는 그 이상의 서브루틴들로서 구현된다.

[0215] 더 구체적으로는, 메인프레임에서, 아키텍처화된 기계어 명령들(machine instructions)이 프로그래머들, 대개는 오늘날의 "C" 프로그래머들에 의해, 흔히 컴파일러 애플리케이션(compiler application)을 통해 사용되고 있다. 스토리지 매체에 저장되는 이들 명령들은 원래(natively) z/Architecture IBM® 서버에서 또는 이와는 다르게 다른 아키텍처들을 실행하는 머신들에서 실행될 수 있다. 그것들은 기존의 그리고 장래의 IBM® 메인프레임 서버들에서 그리고 IBM®의 다른 머신들(예, Power Systems 서버들 및 System x® 서버들) 상에서 에뮬레이트될 수 있다. 그것들은 IBM®, Intel®, AMD™ 및 기타 회사에 의해 제조된 하드웨어를 사용하는 광범위한 머신들 상의 리눅스를 실행하는 머신들에서 실행될 수 있다. 또한, z/Architecture 하의 그 하드웨어 상에서의 실행 이외에, Hercules, UMX, 또는 FSI(Fundamental Software, Inc)—여기서 일반적으로 실행은 에뮬레이션 모드에 있음—에 의해 에뮬레이션을 사용하는 머신들 뿐만이 아니라 리눅스도 사용될 수 있다. 에뮬레이션 모드에서, 에뮬레이션 소프트웨어는 네이티브 프로세서에 의해 실행되어 에뮬레이트된 프로세서의 아키텍처를 에뮬레이트한다.

[0216] 네이티브 프로세서(native processor)는 통상적으로 에뮬레이트된 프로세서의 에뮬레이션을 수행하기 위해 펌웨어(firmware) 또는 네이티브 운영체제를 포함하는 에뮬레이션 소프트웨어를 실행한다. 에뮬레이션 소프트웨어는 그 에뮬레이트된 프로세서 아키텍처의 명령들을 폐치 및 실행하는 역할을 한다. 에뮬레이션 소프트웨어는 명령 경계들(instruction boundaries)을 추적하기 위해 에뮬레이트된 프로그램 카운터를 유지한다. 에뮬레이션 소프트웨어는 한 번에 하나 또는 그 이상의 에뮬레이트된 기계어 명령들을 폐치하여, 하나 또는 그 이상의 그 에뮬레이트된 기계어 명령들을 네이티브 프로세서에 의해 실행하기 위한 네이티브 기계어 명령들의 대응 그룹으로 변환시킬 수 있다. 이들 변환된 명령들은 캐시되어 더 빠른 변환이 수행될 수 있도록 할 수 있다. 그럼에도 불구하고, 에뮬레이션 소프트웨어는, 운영체제들 및 에뮬레이트된 프로세서를 위해 작성된 애플리케이션들이 정확하게 연산되도록 보장하기 위해, 그 에뮬레이트된 프로세서 아키텍처의 아키텍처 규칙들을 유지해야 한다. 더 나아가, 에뮬레이션 소프트웨어는 그 에뮬레이트된 프로세서 아키텍처에 의해 식별된 자원들을 제공해야 하며—이 자원들에는 제어 레지스터들, 범용 레지스터들, 부동 소수점 레지스터들, 예를 들어 세그먼트 테이블들 및 페이지 테이블들을 포함하는 동적 주소 변환 함수, 인터럽트 메커니즘들, 컨텍스트 전환 메커니즘들, TOD(Time of Day) 클록들 및 I/O 서브시스템들에 대한 아키텍처화된 인터페이스들이 포함됨—그리하여 운영체제, 또는 에뮬레이트된 프로세서 상에서 실행되도록 지정된 애플리케이션 프로그램이, 에뮬레이션 소프트웨어를 갖는 네이티브 프로세서상에서 실행될 수 있도록 한다.



[0217] 에뮬레이트되고 있는 특정 명령이 디코드되고, 서브루틴이 개별 명령의 기능을 수행하기 위해 호출(call)된다. 에뮬레이트된 프로세서의 기능을 에뮬레이트하는 에뮬레이션 소프트웨어 기능은, 예를 들어, "C" 서브루틴 또는 드라이버, 또는 특정 하드웨어를 위해 드라이버를 제공하는 몇몇 다른 방법들로 구현되며, 이는 하나 또는 그 이상의 실시 예들의 설명을 이해하고 나면 이 기술 분야에서 통상의 지식을 가진 자들이 도출해 낼 수 있을 것이다. 여러 가지 소프트웨어 및 하드웨어 에뮬레이션 특허들—예를 들어, Beausoleil 외 발명의 미국 특허증 (Letters Patent) 제5,551,013호 "하드웨어 에뮬레이션을 위한 멀티프로세서(Multiprocessor for Hardware Emulation)"; Scalzi 외 발명의 미국 특허증 제6,009,261호 "타겟 프로세서 상에서 호환가능하지 않은 명령들을 에뮬레이트하기 위한 저장된 타겟 루틴들의 전처리(Preprocessing of Stored Target Routines for Emulating Incompatible Instructions on a Target Processor)"; Davidian 외 발명의 미국 특허증 제5,574,873호 "게스트 명령들을 에뮬레이트하는 직접 액세스 에뮬레이션 루틴들에 대한 게스트 명령을 디코드하는 것(Decoding Guest Instruction to Directly Access Emulation Routines that Emulate the Guest Instructions)"; Gorishek 외 발명의 미국 특허증 제6,308,255호 "시스템에서 논-네이티브 코드를 실행할 수 있도록 하는 코프로세서 지원에 사용되는 대칭형 다중 처리 버스 및 칩셋(Symmetrical Multiprocessing Bus and Chipset Used for Coprocessor Support Allowing Non-Native Code to Run in a System)"; Lethin 외 발명의 미국 특허증 제6,463,582호 "아키텍처 에뮬레이션을 위한 동적 최적화 객체 코드 변환 및 동적 최적화 객체 코드 변환 방법(Dynamic Optimizing Object Code Translator for Architecture Emulation and Dynamic Optimizing Object Code Translation Method)"; Eric Traut 발명의 미국 특허증 제5,790,825호 "호스트 명령들의 동적 리컴파일레이션을 통해 호스트 컴퓨터 상에서 게스트 명령들을 에뮬레이트하기 위한 방법(Method for Emulating Guest Instructions on a Host Computer Through Dynamic Recompilation of Host Instructions)" 등이 포함되나, 이러한 것들로 한정되는 것은 아님, 이들 각각은 여기에서 그 전체가 참조로써 포함됨—이 기술 분야에서 통상의 지식을 가진 자들이 이용할 수 있는 목표 머신에 대한 다른 머신을 위해 아키텍처화된 명령 포맷의 에뮬레이션을 달성하는 알려진 여러 가지 방법들을 예시하고 있다.

[0218] 도 16에서는, 호스트 아키텍처의 호스트 컴퓨터 시스템(5000')을 에뮬레이트하는 에뮬레이트된 호스트 컴퓨터 시스템(5092)의 예가 제공된다. 에뮬레이트된 호스트 컴퓨터 시스템(5092)에서, 호스트 프로세서(CPU)(5091)는 에뮬레이트된 호스트 프로세서(또는 가상 호스트 프로세서)이고 호스트 컴퓨터(5000')의 프로세서(5091)의 네이티브 명령 세트 아키텍처(native instruction set architecture)와는 다른 네이티브 명령 세트 아키텍처를 갖는 에뮬레이션 프로세서(5093)를 포함한다. 에뮬레이트된 호스트 컴퓨터 시스템(5092)은 에뮬레이션 프로세서(5093)가 액세스 가능한 메모리(5094)를 갖는다. 상기 예시 실시 예에서, 메모리(5094)는 호스트 컴퓨터 메모리(5096) 부분과 에뮬레이션 루틴들(5097) 부분으로 분할된다. 호스트 컴퓨터 메모리(5096)는 호스트 컴퓨터 아키텍처에 따른 에뮬레이트된 호스트 컴퓨터(5092)의 프로그램들이 이용할 수 있다. 에뮬레이션 프로세서(5093)는 에뮬레이트된 프로세서(5091)의 명령 이외의 아키텍처의 아키텍처화된 명령 세트의 네이티브 명령들, 즉 에뮬레이션 루틴들 메모리(5097)로부터 획득된 네이티브 명령들을 실행하며, 시퀀스 & 액세스/디코드 루틴—이는 액세스되는 호스트 명령의 기능을 에뮬레이트하기 위해 네이티브 명령 실행 루틴을 결정하기 위해 액세스되는 호스트 명령(들)을 디코드할 수 있음—에서 획득된 하나 또는 그 이상의 명령(들)을 채용함으로써 호스트 컴퓨터 메모리(5096) 내 프로그램으로부터 실행하기 위한 호스트 명령을 액세스할 수 있다. 호스트 컴퓨터 시스템(5000') 아키텍처에 대하여 정의된 다른 퍼실리티들이 아키텍처화된 퍼실리티들 루틴들(architected facilities routines)에 의해 에뮬레이트될 수 있는데, 이러한 것들에는, 예를 들어, 범용 레지스터들, 제어 레지스터들(control registers), 동적 주소 변환(dynamic address translation) 및 I/O 서브시스템 지원 및 프로세서 캐시 등과 같은 퍼실리티들이 포함된다. 에뮬레이션 루틴들(emulation routines)은 또한 (범용 레지스터들 및 가상 주소들의 동적 변환 같은) 에뮬레이션 프로세서(5093)에서 이용 가능한 기능들을 이용하여 에뮬레이션 루틴들의 성능을 향상시킬 수 있다. 또한 특수 하드웨어(special hardware) 및 오프-로드 엔진들(off-load engines)이 제공되어 호스트 컴퓨터(5000')의 기능을 에뮬레이팅함에 있어서 프로세서(5093)를 보조할 수 있다.

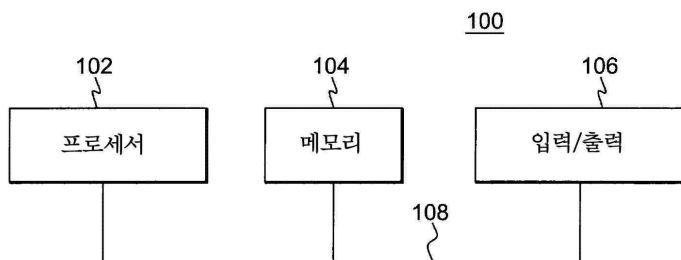
[0219] 본 명세서 내에 사용되는 용어는 단지 특정 실시 예들을 기술할 목적으로 사용된 것이지 본 발명을 한정하려는 의도로 사용된 것은 아니다. 여기에서 사용할 때, 단수 형태인 "한", "일", 및 "하나" 등은 그 컨텍스트에서 그렇지 않은 것으로 명시되어 있지 않으면, 복수 형태도 또한 포함할 의도로 기술된 것이다. 또한, "포함한다" 및/또는 "포함하는"이라는 말들은 본 명세서에서 사용될 때, 언급되는 특징들, 정수들, 단계들, 동작들, 엘리먼트들, 및/또는 컴포넌트들의 존재를 명시하지만, 하나 또는 그 이상의 다른 특징들, 정수들, 단계들, 동작들, 엘리먼트들, 컴포넌트들 및/또는 이들의 그룹들의 존재 또는 추가를 배제하는 것은 아니라는 것을 이해할 수 있을 것이다.

[0220]

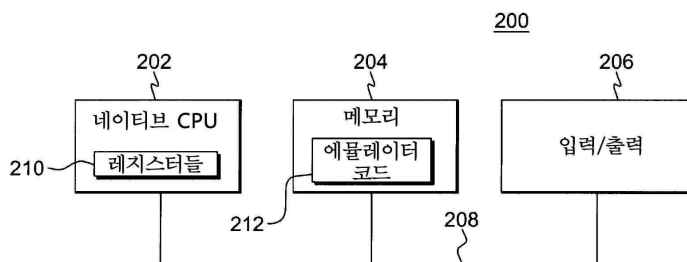
이하의 청구항들에서, 구조들(structures), 재료들(materials), 동작들(acts), 및 모든 수단의 등가물들 또는 단계 플러스 기능 엘리먼트들은 구체적으로 청구되는 다른 청구된 엘리먼트들과 함께 그 기능을 수행하기 위한 구조, 재료, 또는 동작을 포함할 의도가 있다. 하나 또는 그 이상의 특징들에 대한 설명은 예시와 설명의 목적으로 제공되는 것이며, 개시되는 형태로 본 발명의 모든 실시 예들을 빠짐없이 총 망라하거나 본 발명을 한정하려는 의도가 있는 것은 아니다. 이 기술 분야에서 통상의 지식을 가진 자라면 본 발명의 하나 또는 그 이상의 범위와 정신을 벗어나지 않고서 많은 수정 예들 및 변형 예들이 있을 수 있다는 것을 알 수 있다. 실시 예는 하나 또는 그 이상의 특징들 및 실제 응용을 가장 잘 설명하기 위해 그리고 고려되는 구체적인 용도에 적합하게 여러 가지 수정 예들을 갖는 다양한 실시 예들에 대한 하나 또는 그 이상의 특징들을 이 기술 분야에서 통상의 지식을 가진 자들이 이해할 수 있도록 하기 위해 선택되고 기술되었다.

## 도면

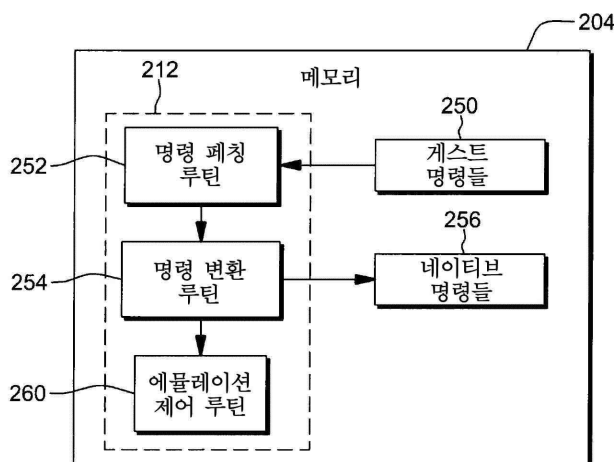
### 도면1



### 도면2a

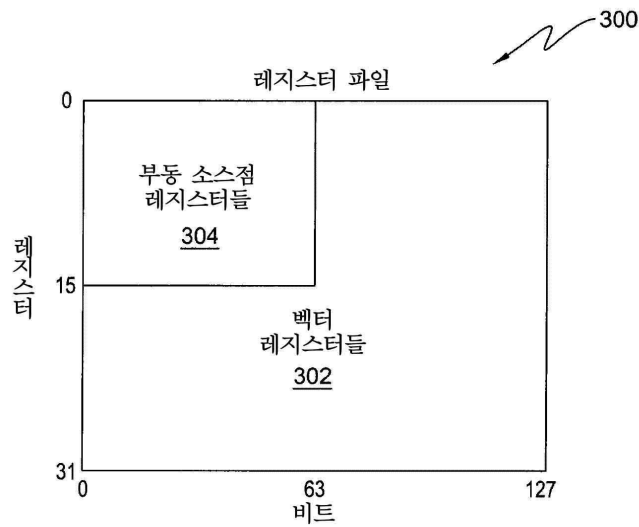


### 도면2b

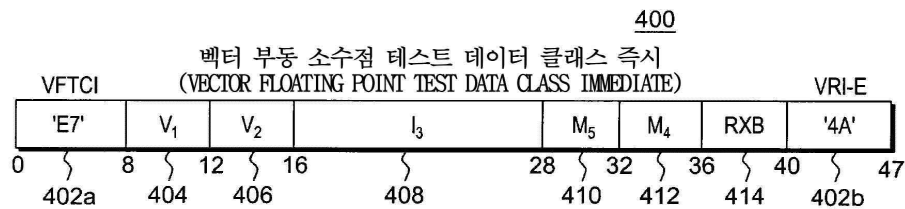




도면3



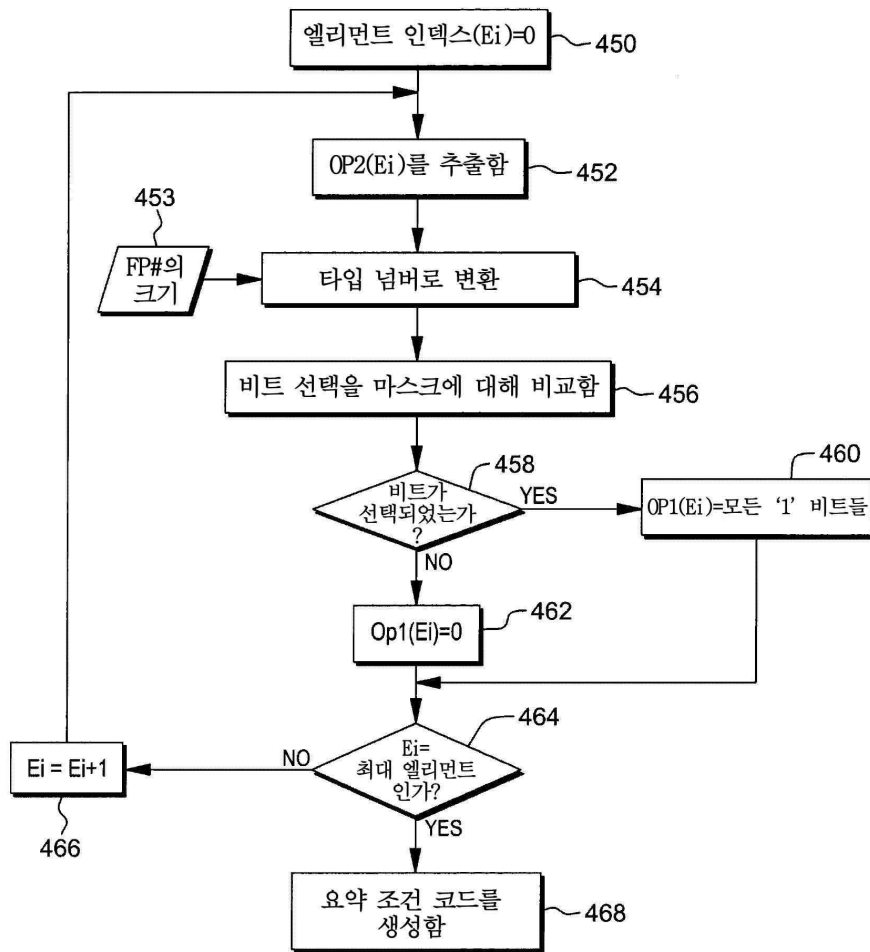
도면4a



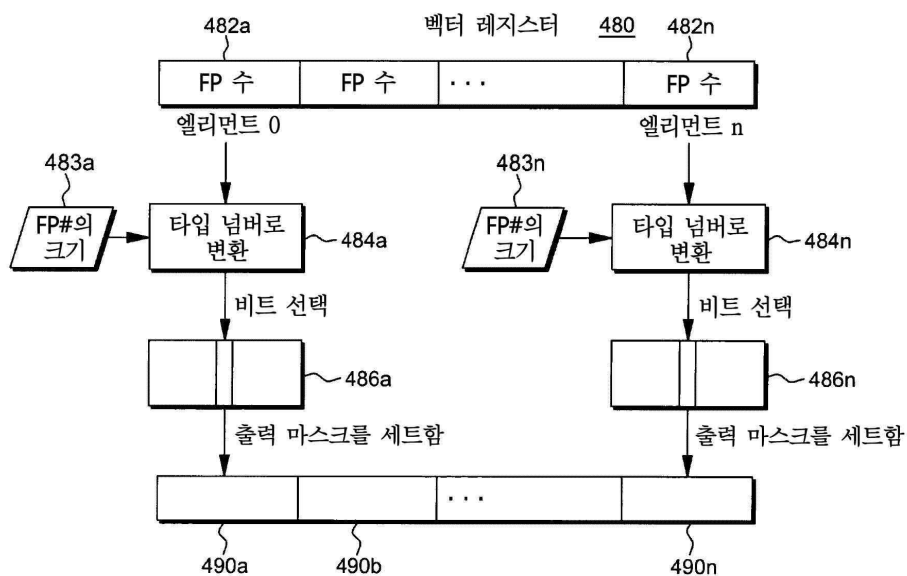
도면4b

| 430 | BFP 엘리먼트 클래스           | 부호가 있을 때 사용되는 비트 |    | 432 |
|-----|------------------------|------------------|----|-----|
|     |                        | +                | -  |     |
|     | 제로                     | 0                | 1  |     |
|     | 정규수(normal number)     | 2                | 3  |     |
|     | 비정규수(subnormal number) | 4                | 5  |     |
|     | 무한대                    | 6                | 7  |     |
|     | QNaN                   | 8                | 9  |     |
|     | SNAN                   | 10               | 11 |     |

도면4c



도면4d



도면4e

| 엔티티                 | 부호 | 편중 지수            | 유닛 비트* | 프랙션          |
|---------------------|----|------------------|--------|--------------|
| 참 제로(true zero)     | ±  | 0                | 0      | 0            |
| 비정규화 수              | ±  | 0**              | 0      | 0이 아님        |
| 정규화 수               | ±  | 0이 아님, 모두 1들이 아님 | 1      | 모두(any)      |
| 무한대                 | ±  | 모두 1들            | -      | 0            |
| QNaN(quiet NaN)     | ±  | 모두 1들            | -      | FO=1, Fr=ANY |
| SNaN(signaling NaN) | ±  | 모두 1들            | -      | FO=0, Fr≠0   |

설명:

\* 유닛 비트는 암시됨

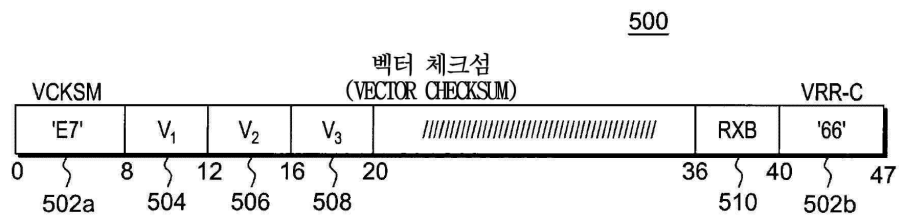
\*\* 편중 지수는 마치 그것이 1의 값을 갖는 것처럼 산술적으로 취급됨

NaN 숫자가 아님

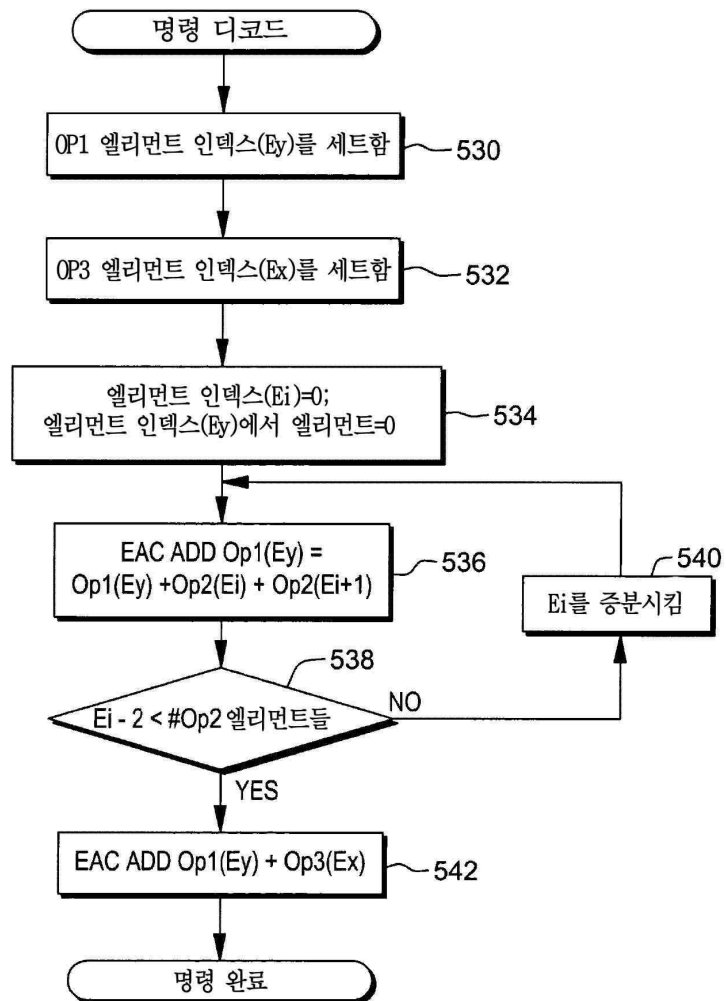
FO 프랙션의 최좌측 비트

Fr 프랙션의 남아있는 비트들

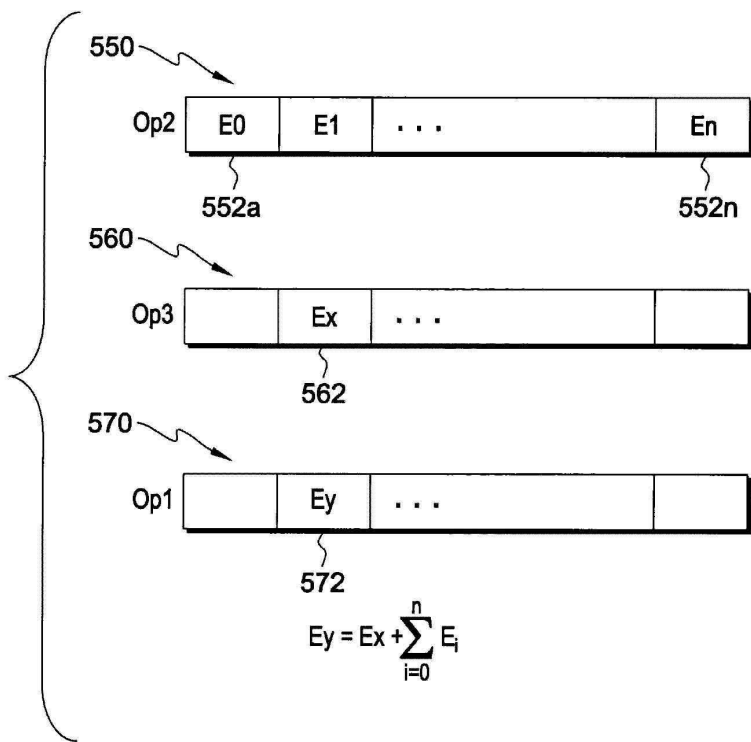
도면5a



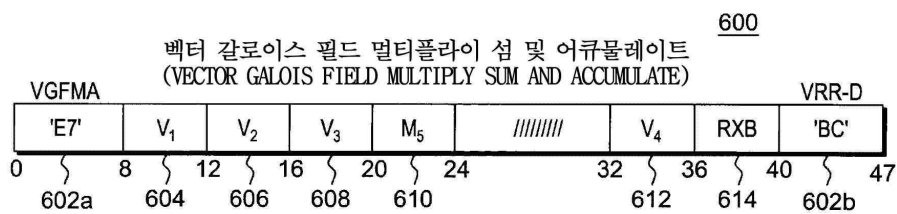
도면5b



도면5c

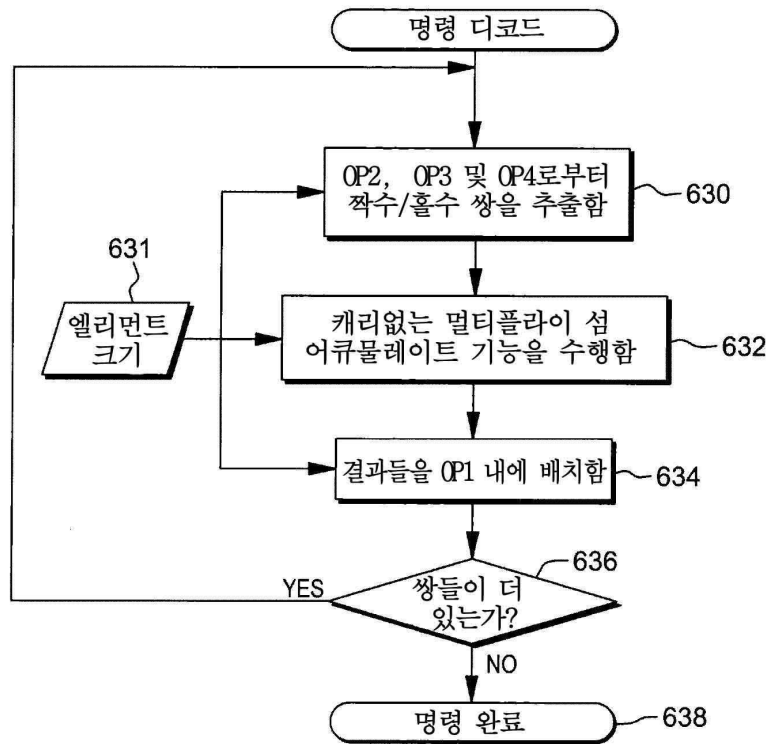


도면6a

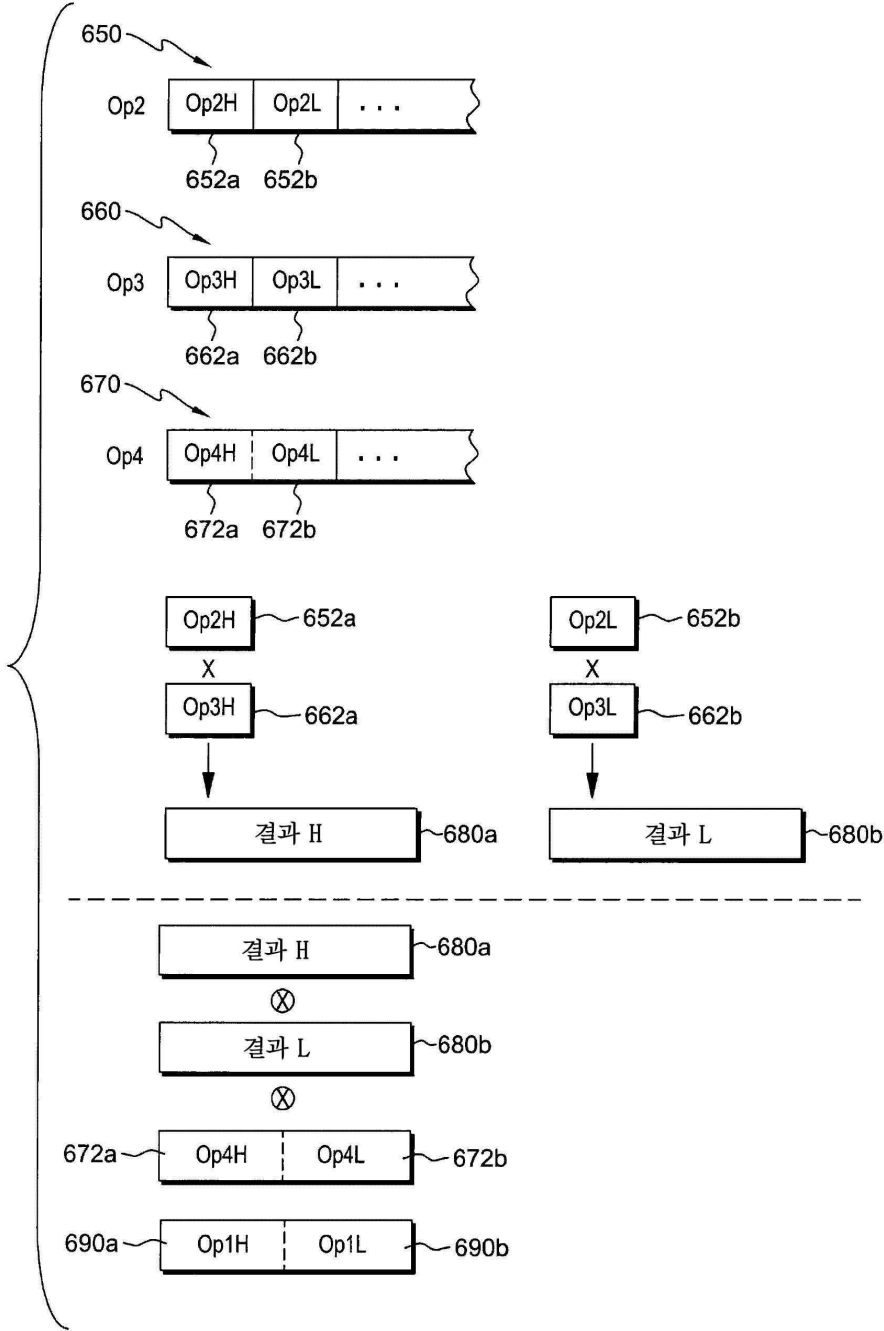




도면6b



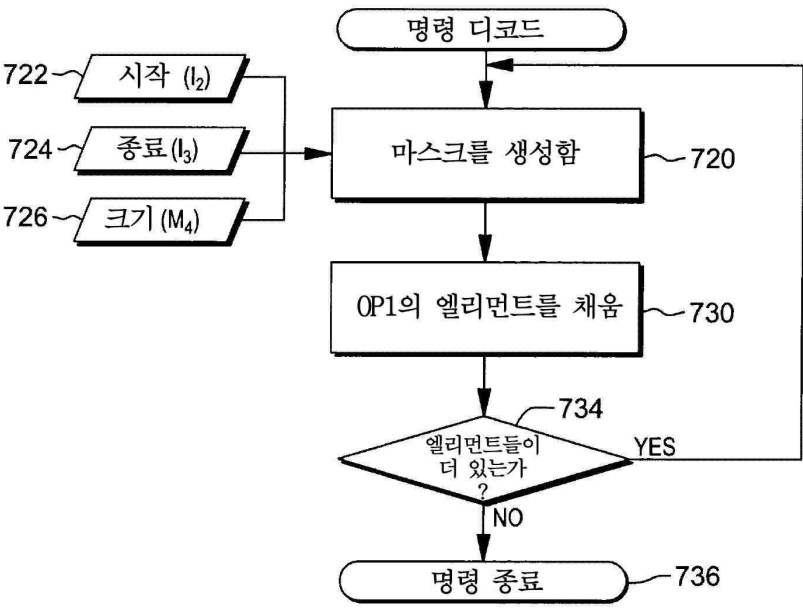
도면6c



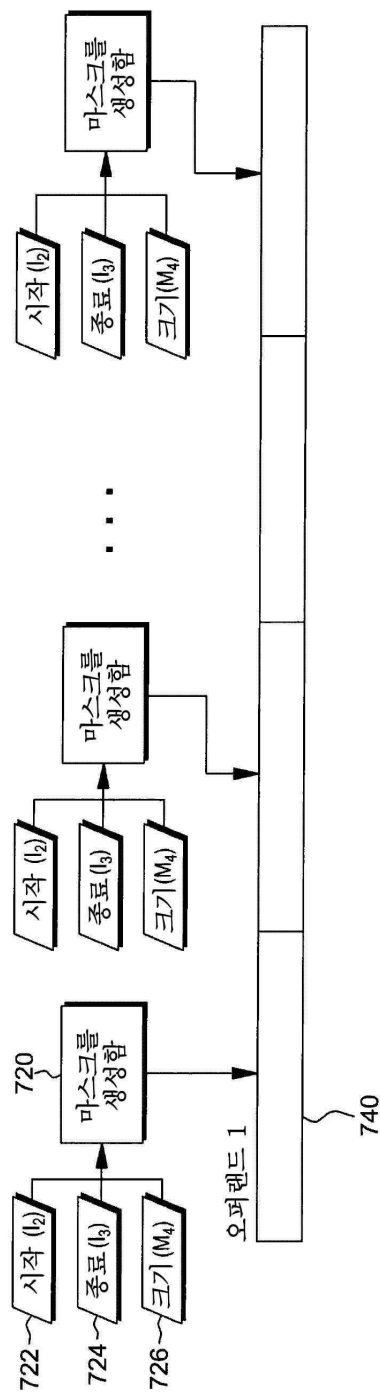
도면7a



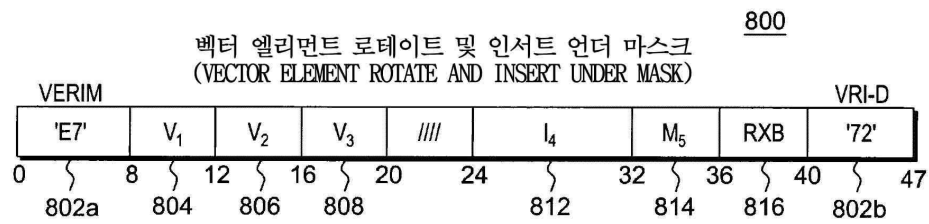
도면7b



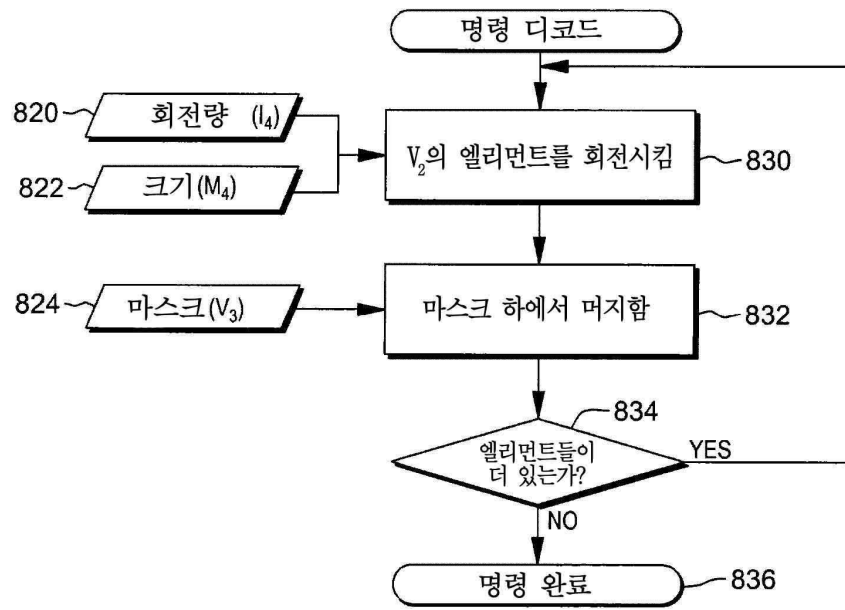
도면7c



도면8a

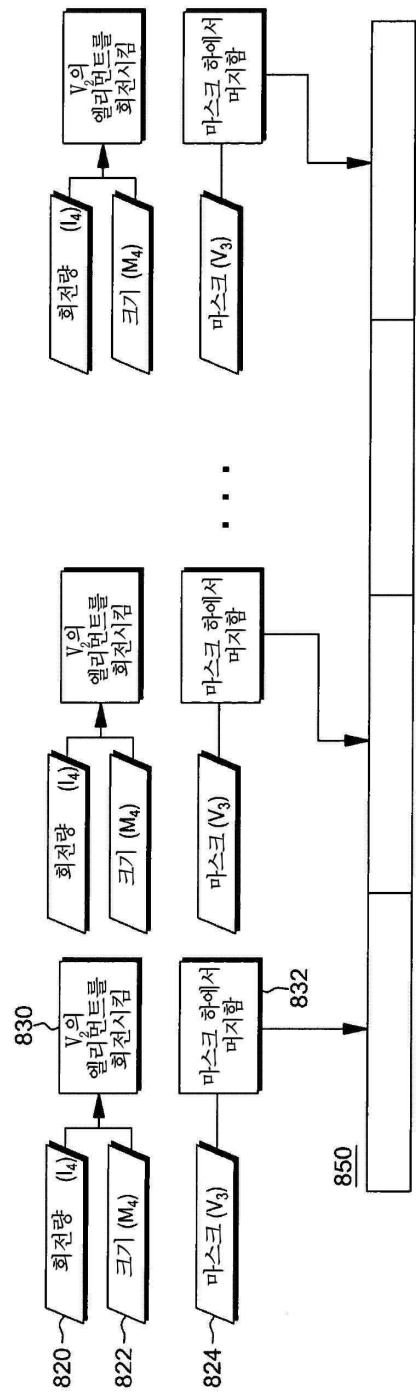


도면8b

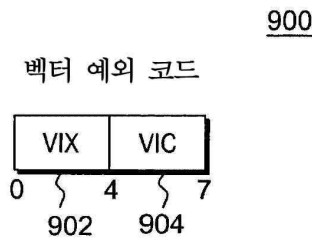




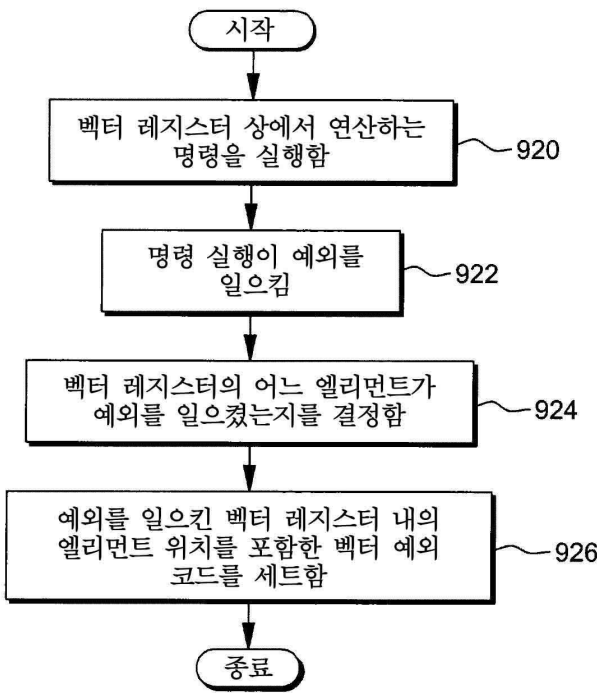
도면8c



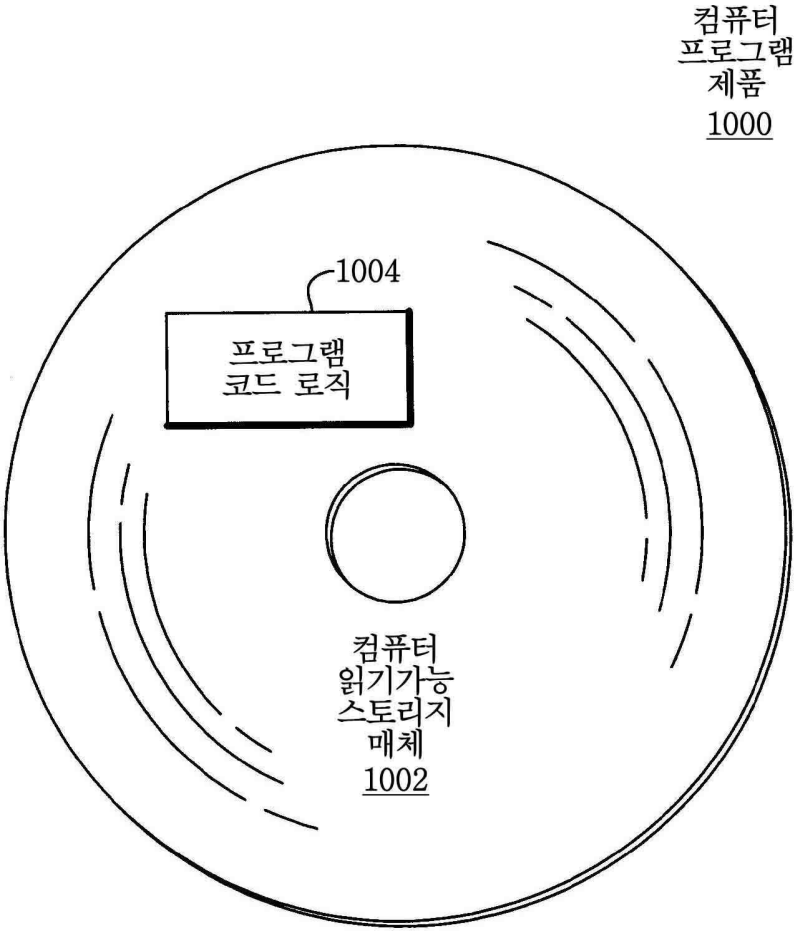
도면9a



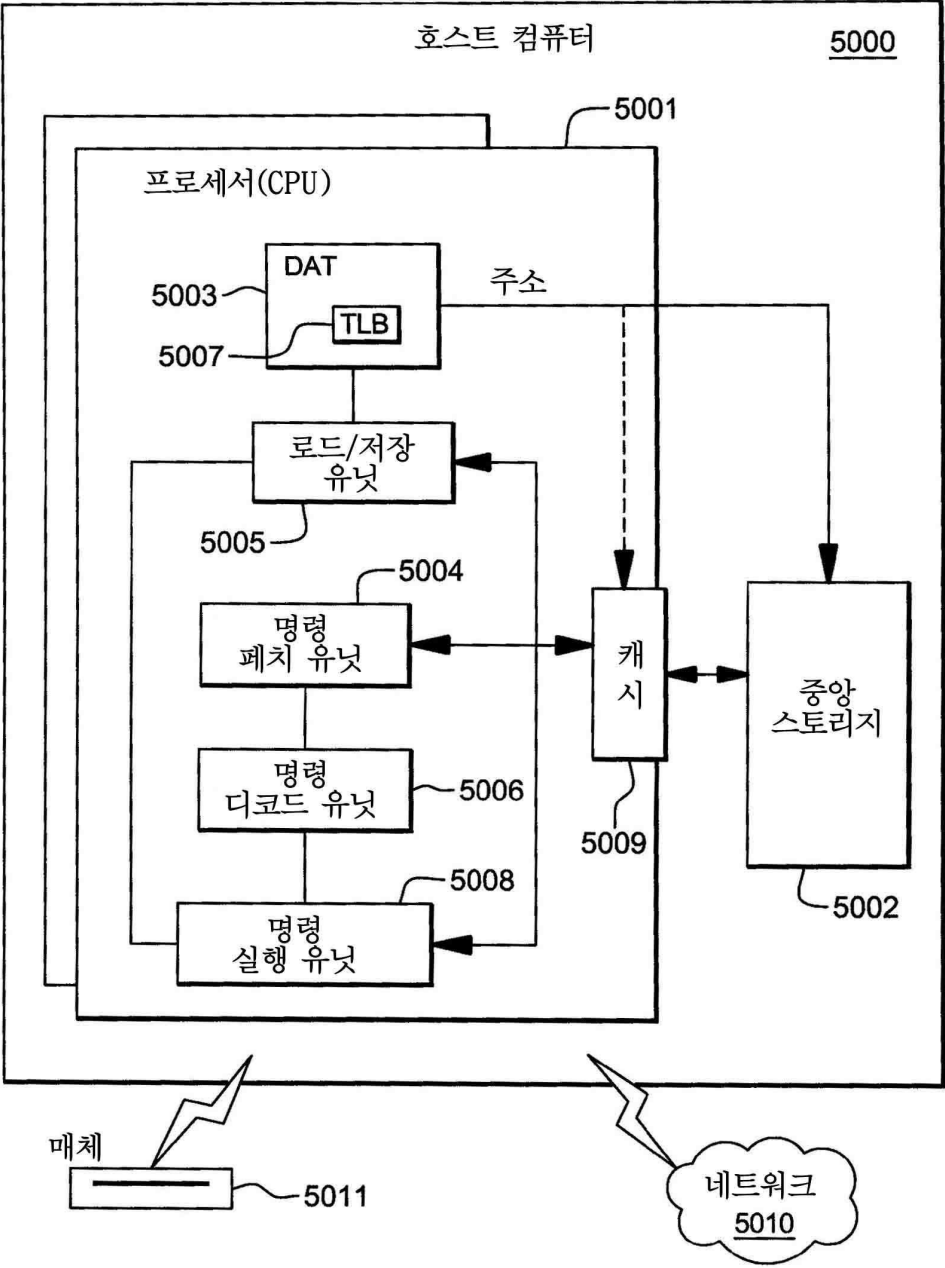
도면9b



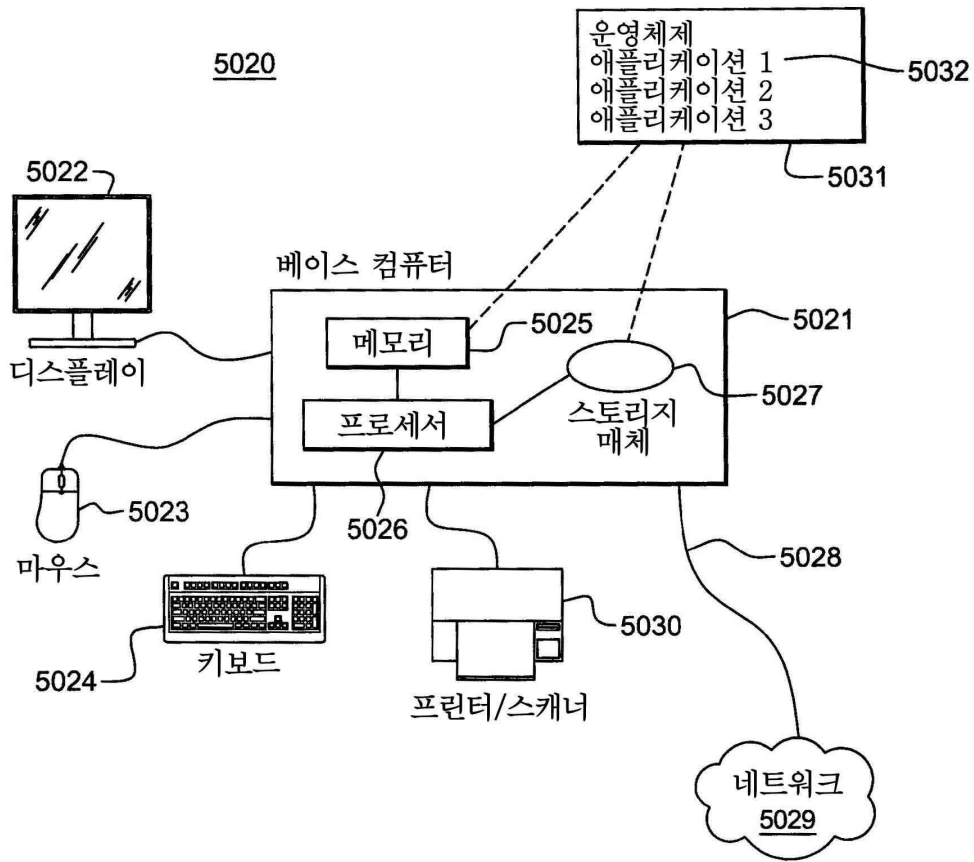
도면10



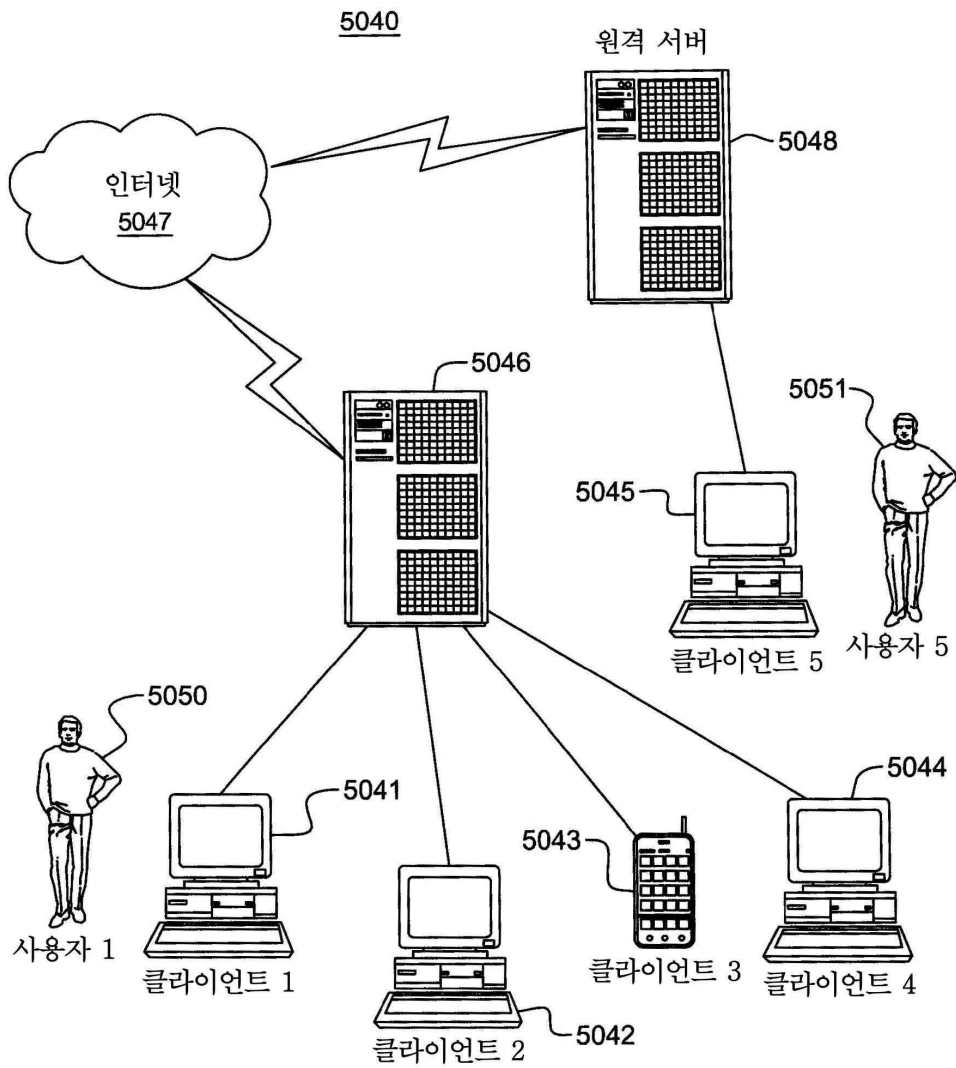
도면11



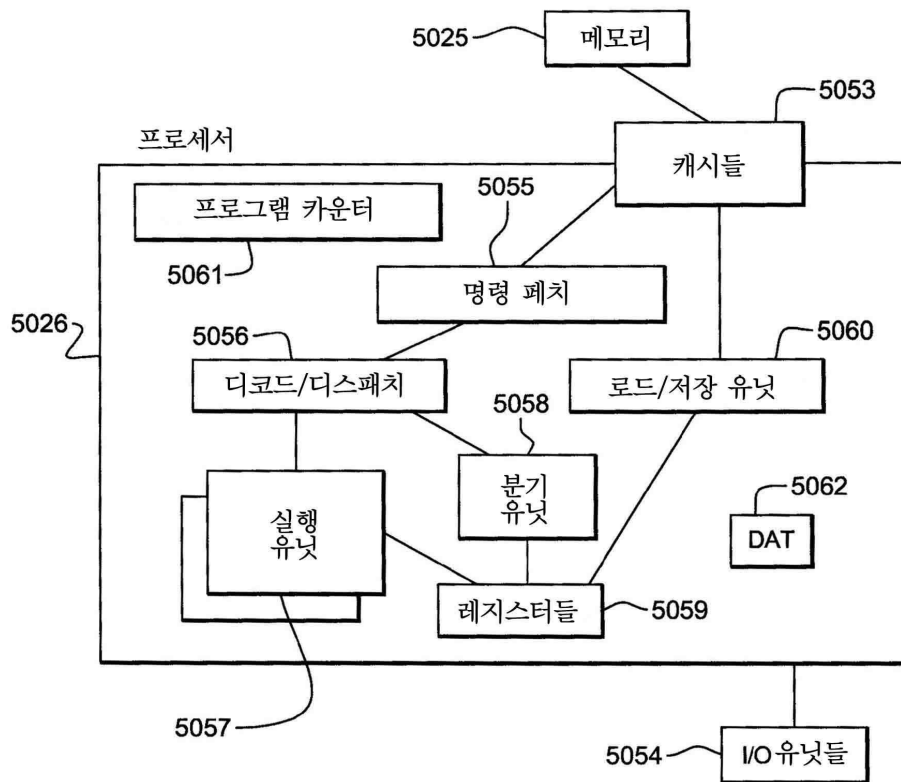
도면12



도면13

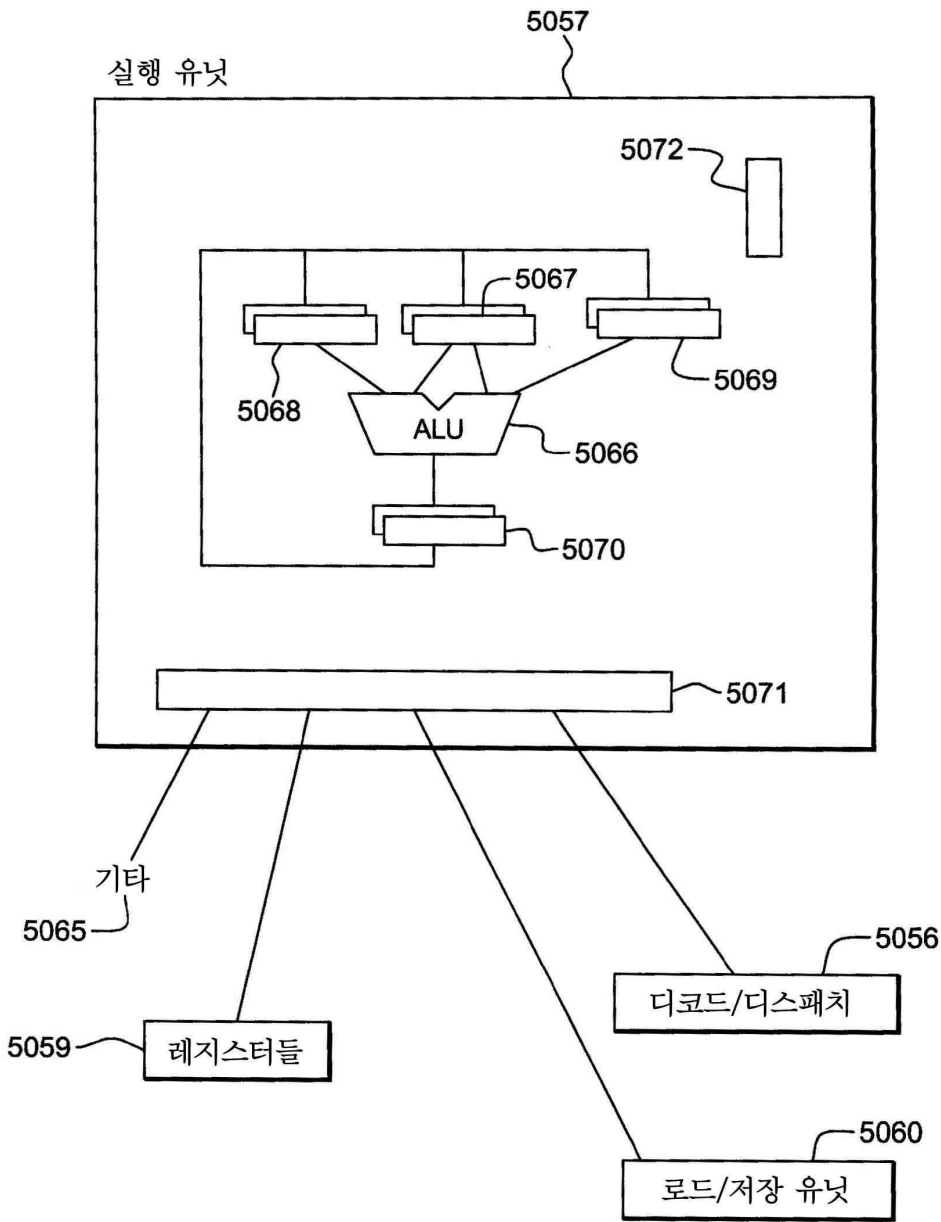


도면14

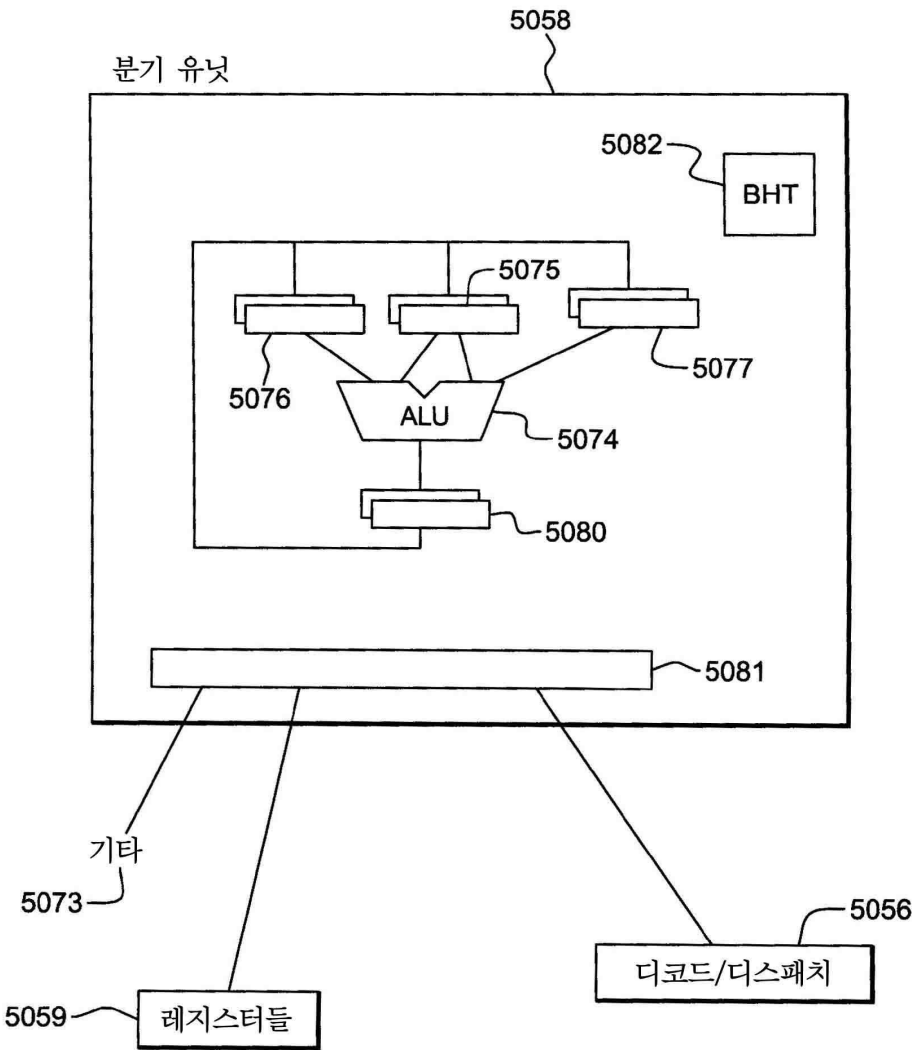




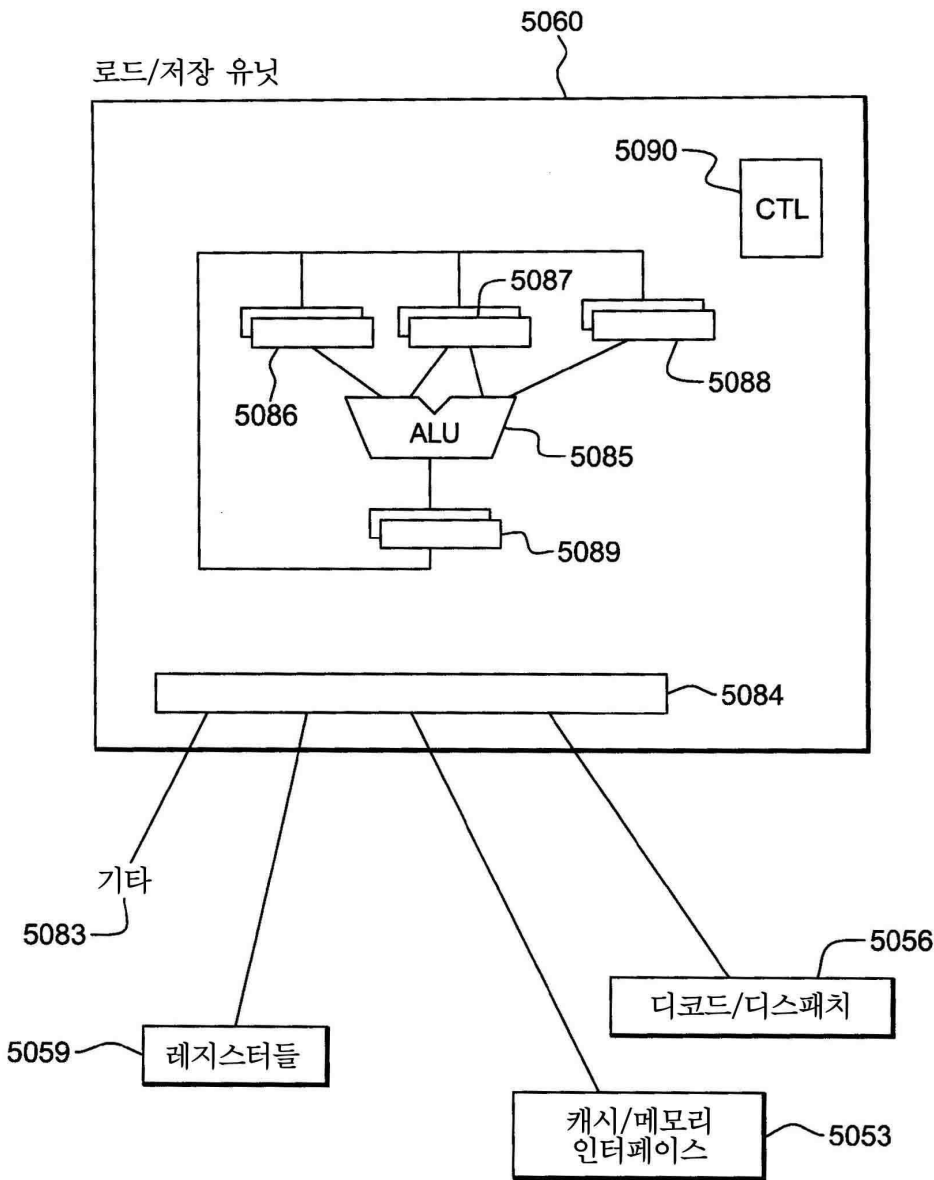
도면15a



도면15b



도면15c



도면16

